

Micro-Architecture et Conception des Microprocesseurs

Cours : Pascal SAINRAT
TP : Thomas CARLE

M1 SECIL
KINN&AH1 : MaCM

Objectifs de l'UE et pré-requis

- Comprendre le fonctionnement interne d'un processeur simple
- Etre capable d'analyser et de concevoir des systèmes matériels informatiques relativement complexes
- Pré-requis
 - Logique combinatoire et séquentielle (vidéos sur moodle)
 - Assembleur ARM (mini-cours sur moodle)
 - VHDL (pour les TP)



Evaluation

Session 1

- Contrôle terminal (70%)
- TP (30%)

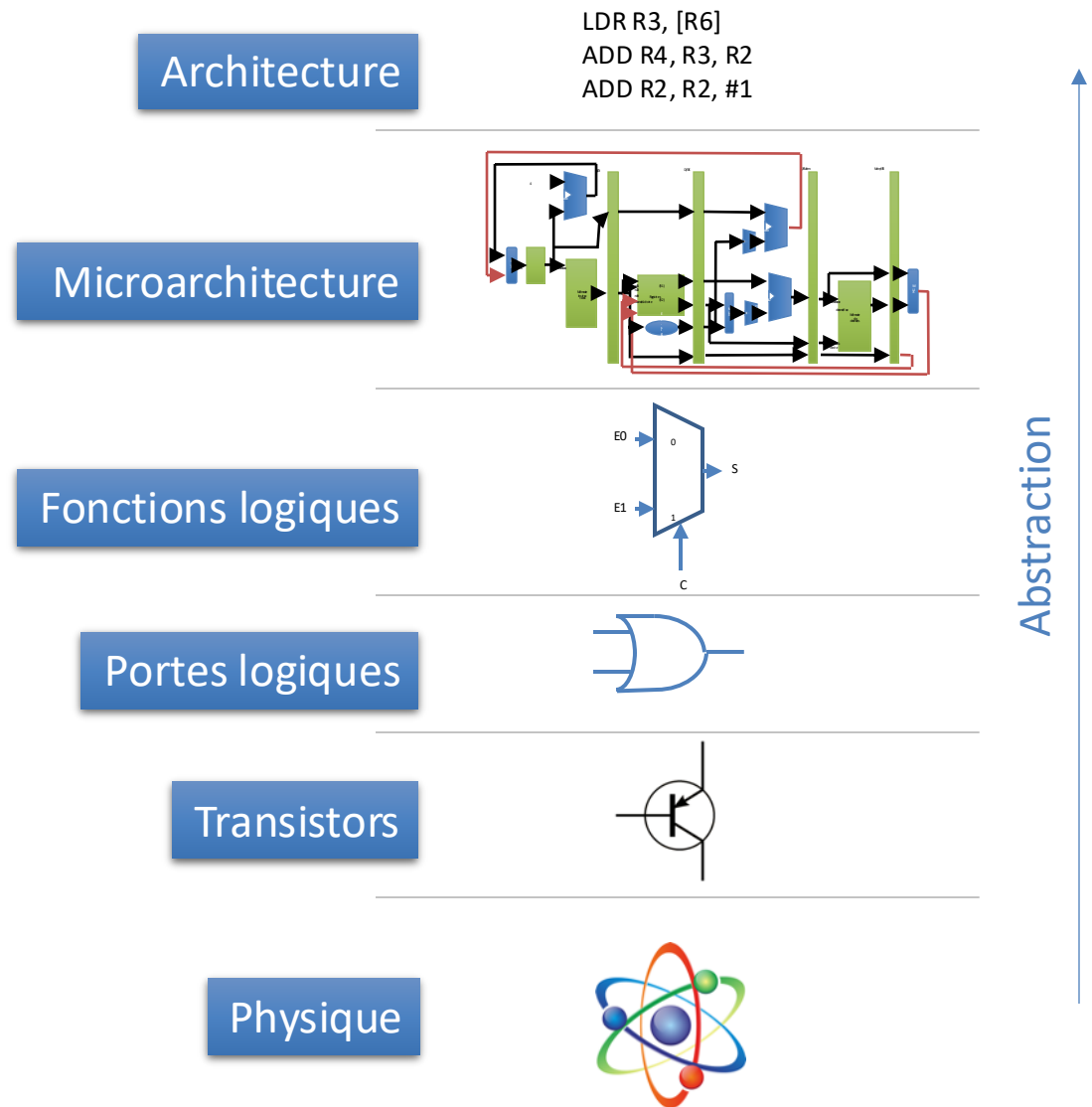
Session 2

- Contrôle terminal (100%)



Abstraction

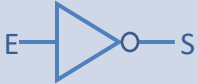
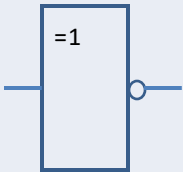
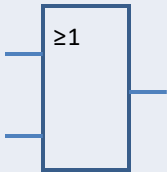
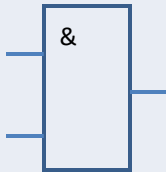
- **Architecture** : vue pour le programmeur
- **Microarchitecture** : Comment c'est fait matériellement
- Il faut connaître les **fonctions logiques** combinatoires et séquentielles
 - Qui sont une abstraction d'un assemblage de **portes logiques**
 - Qui sont une abstraction d'un assemblage de **transistors**
 - Dont le fonctionnement est fondé sur des lois **physiques**

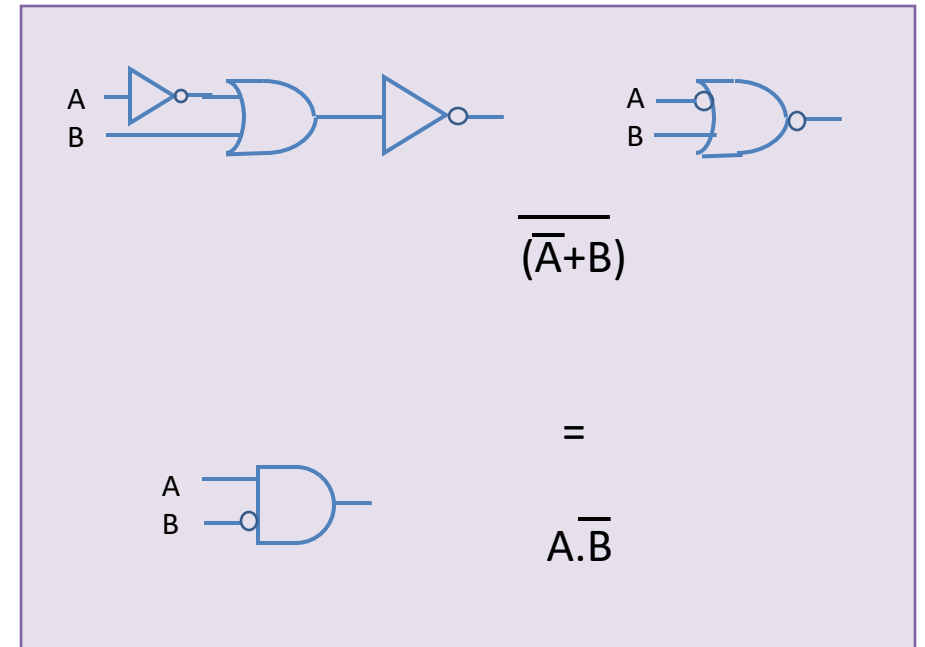


Logique combinatoire

Les portes de base

- Implémentent les fonctions logiques de base et permettent de construire des blocs logiques
- Commutativité + associativité
 - possibilité de portes OU et portes ET à plusieurs entrées

Notation	NON $S = \bar{E}$	OU $S = A + B$	ET $S = A \cdot B$
Américaine			
Européenne			



Logique combinatoire

La porte XOR et la porte XNOR

La porte XOR (ou exclusif)

$S = A \oplus B$ vaut 1 si l'une des deux entrées vaut 1 et pas l'autre.

$$S = (A \cdot \neg B) + (\neg A \cdot B)$$

Contrairement aux portes précédentes, une porte à 3 entrées (ou plus de 3) ne réalise pas la même fonction...

XOR		
A	B	S
0	0	0
0	1	1
1	0	1
1	1	0



La porte XNOR (équivalence)

$$S = \neg (A \oplus B)$$

$$S = (\neg A \cdot \neg B) + (A \cdot B)$$

XNOR		
A	B	S
0	0	1
0	1	0
1	0	0
1	1	1

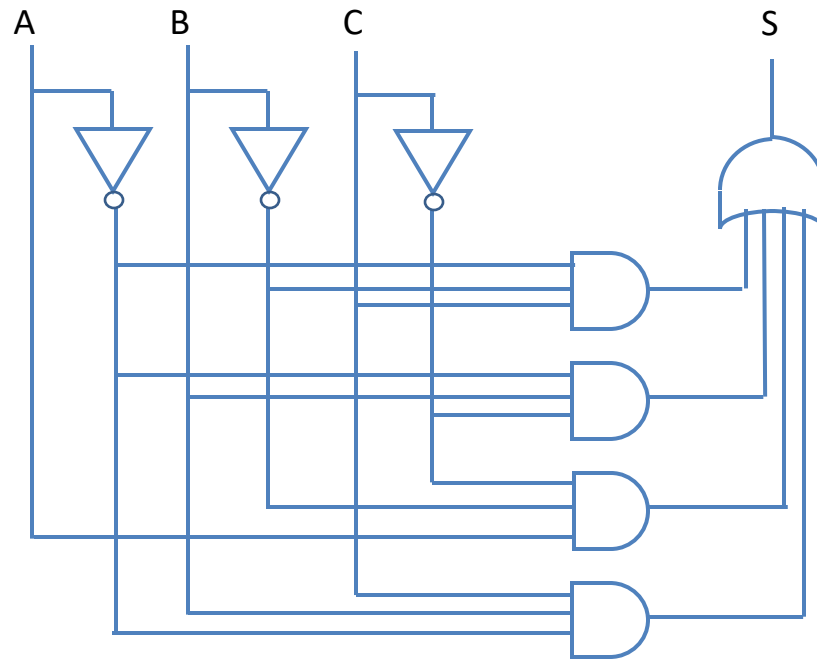


La porte XOR à 3 entrées

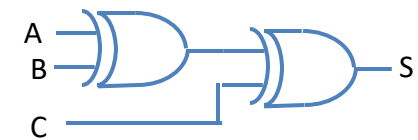
- 1 si un nombre impair d'entrées est à 1

XOR			
A	B	C	S
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$$S = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$



$$S = A \oplus B \oplus C = (A \oplus B) \oplus C$$



$$S = A \oplus B \oplus C$$



Exemples de blocs de logique combinatoire (1)

Décodeurs

- n entrées, 2^n sorties
- Une sortie vaut 1 si la combinaison en entrée représente son numéro

Exemple : un décodeur 2 vers 4

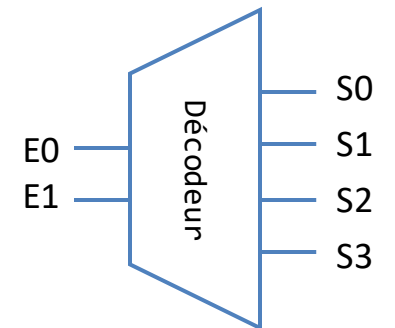
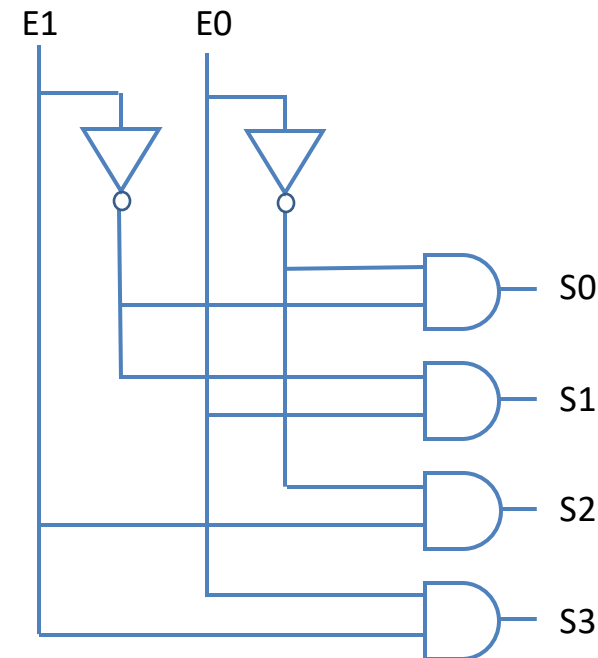
Entrées		Sorties			
E1	E0	S3	S2	S1	S0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

$$S0 = \neg E1 \cdot \neg E0$$

$$S1 = \neg E1 \cdot E0$$

$$S2 = E1 \cdot \neg E0$$

$$S3 = E1 \cdot E0$$



Exemples de blocs de logique combinatoire (2)

Multiplexeurs

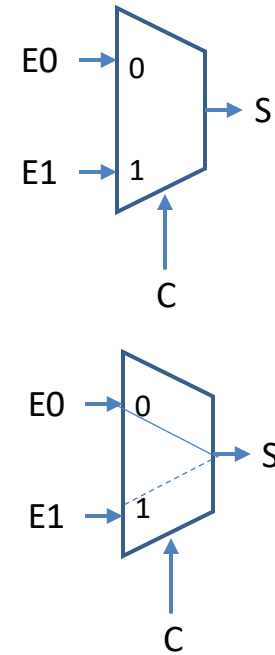
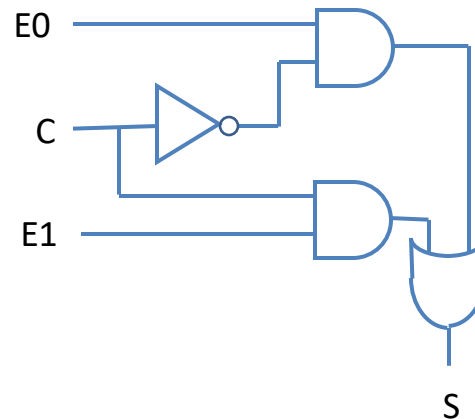
- Un ensemble de signaux C indiquent le numéro de l'entrée E à diriger vers la sortie S

Exemple : un multiplexeur 2 vers 1

Entrées			Sorties
E1	E0	C	S
x	0	0	0
x	1	0	1
0	x	1	0
1	x	1	1

x signifie 0 ou 1

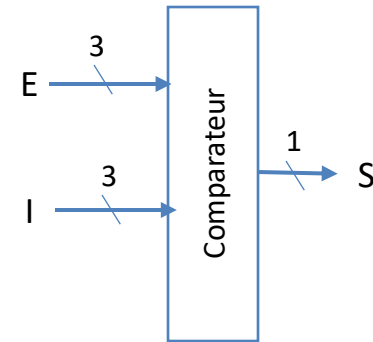
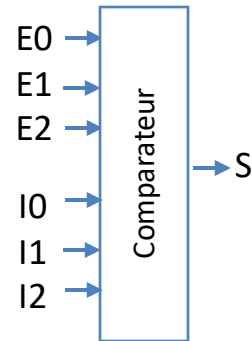
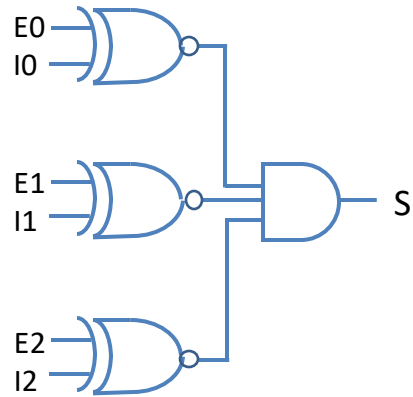
$$S = (E0 \cdot \neg C) + (E1 \cdot C)$$



Exemples de blocs de logique combinatoire (3)

Comparateur (égalité)

- $S=1$ si $E0==I0$ et $E1==I1$ et $E2==I2$



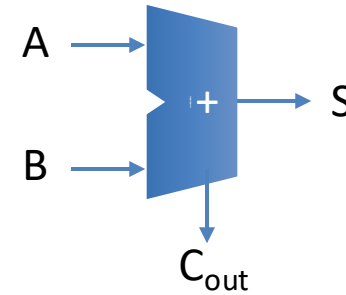
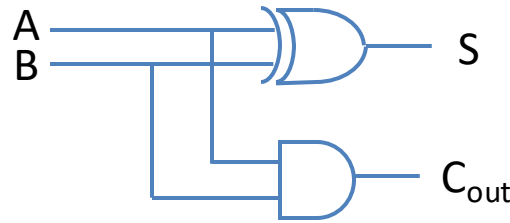
Exemples de blocs de logique combinatoire (4)

Demi-additionneur 1 bit

Entrées		Sorties	
A	B	C _{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$

$$C_{out} = A.B$$

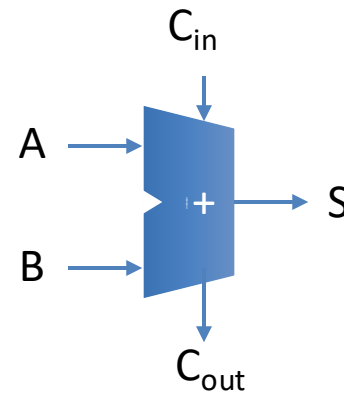


Pour réaliser un additionneur complet, besoin d'une entrée Cin pour la retenue de l'addition du bit inférieur

Exemples de blocs de logique combinatoire (5)

Additionneur complet 1 bit

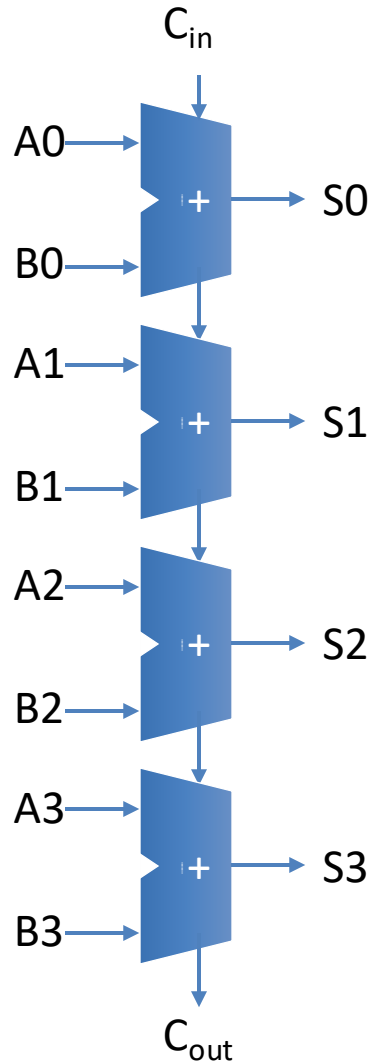
Entrées			Sorties	
A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	1	0	0	1
1	0	0	0	1
1	1	0	1	0
0	0	1	0	1
0	1	1	1	0
1	0	1	1	0
1	1	1	1	1



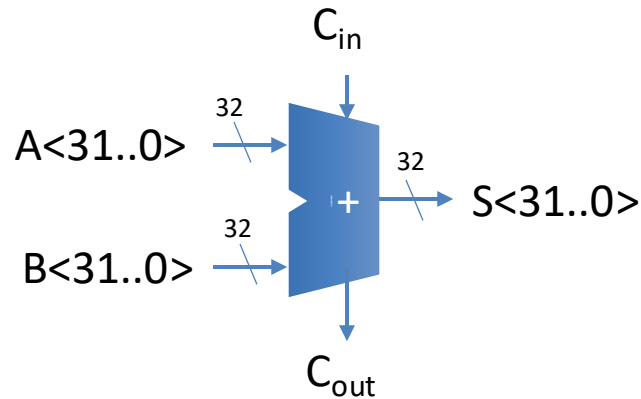
$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = A.B + A.C_{in} + B.C_{in}$$

Exemples de blocs de logique combinatoire (6)

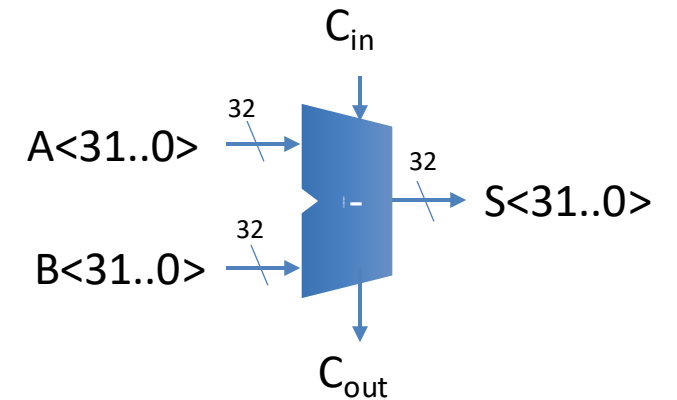
Addition et soustraction multi-bits



Et si on en met 32, on a un additionneur 32 bits.

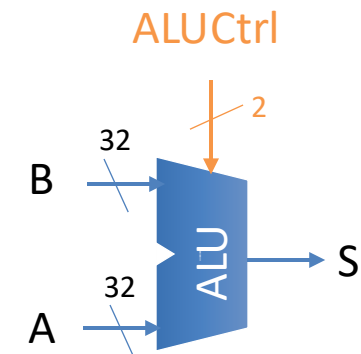
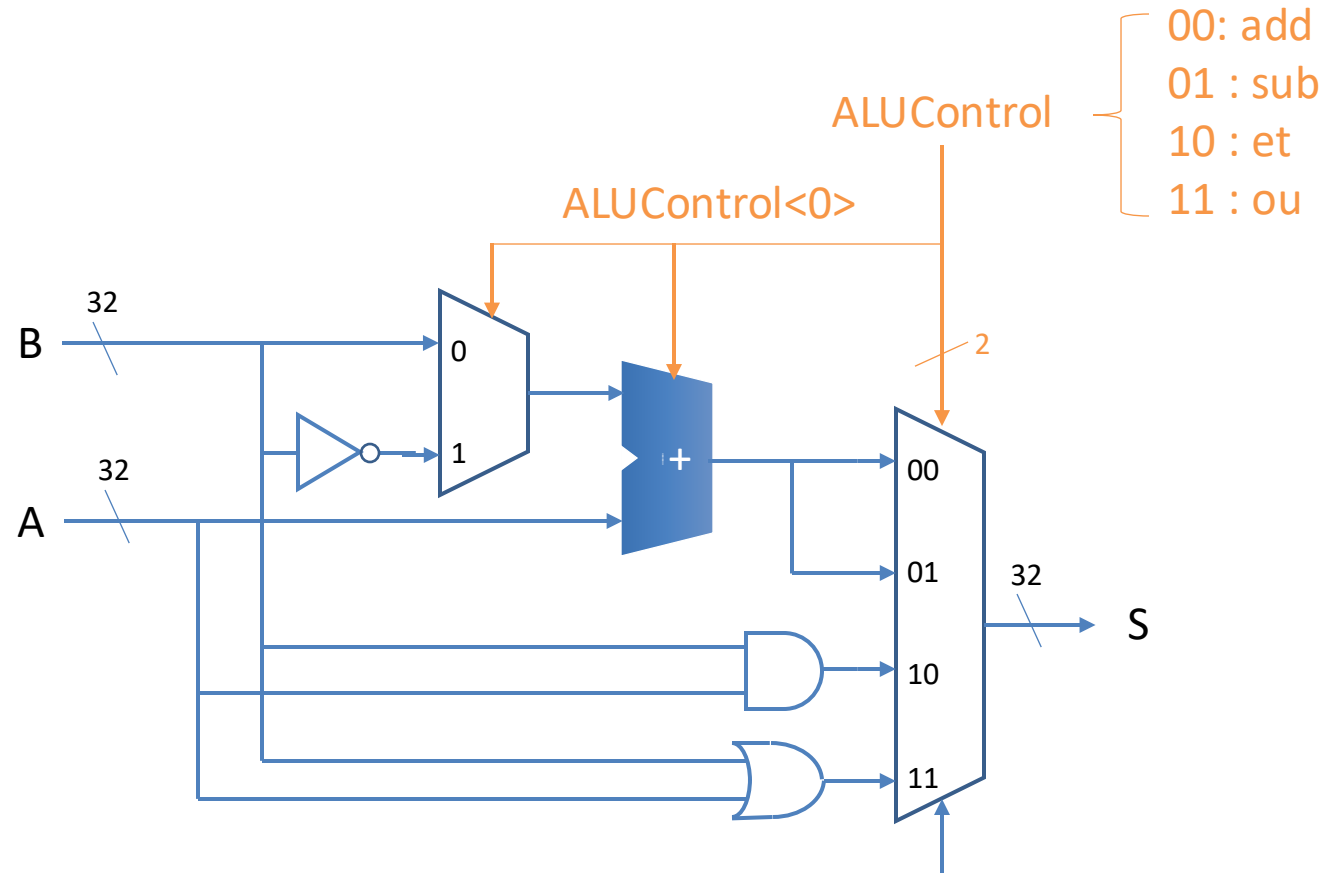


Pour faire une soustraction sur 32 bits, il suffit d'inverser les bits de B et de mettre C_{in} à 1.



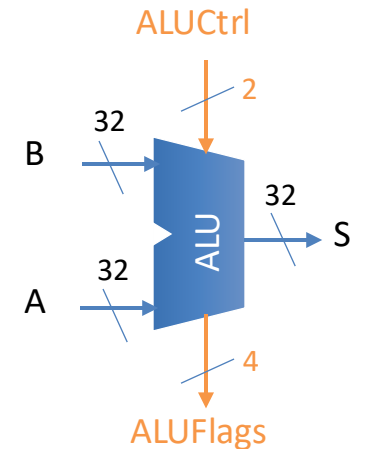
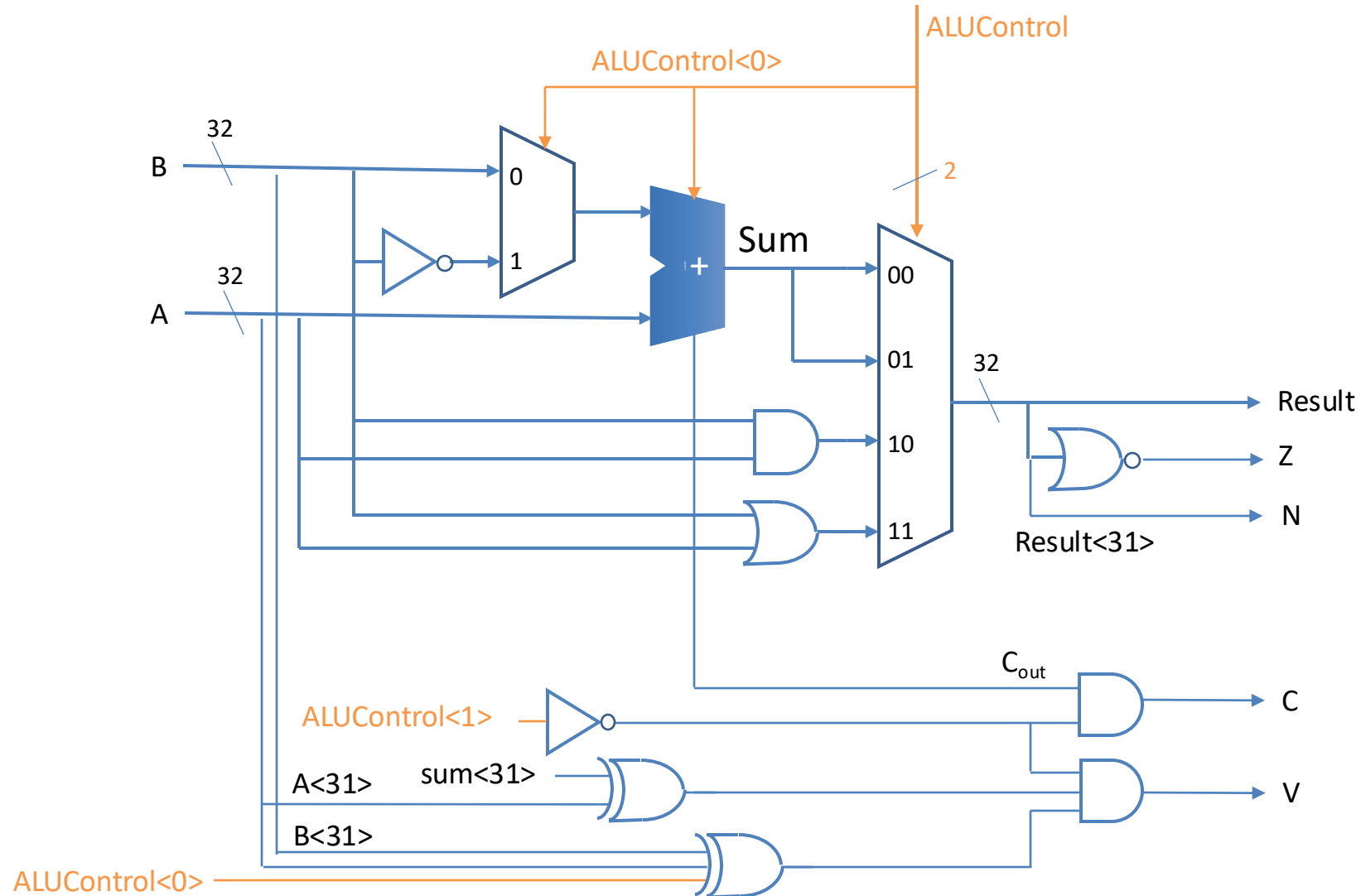
Exemples de blocs de logique combinatoire (7)

Exemple d'ALU (sans indicateurs)



Exemples de blocs de logique combinatoire (8)

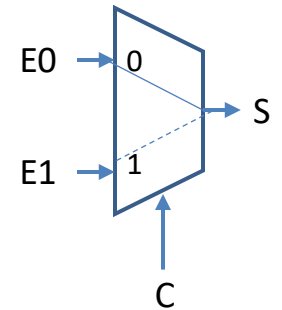
Exemple d'ALU avec indicateurs



Représentation en VHDL de la logique combinatoire

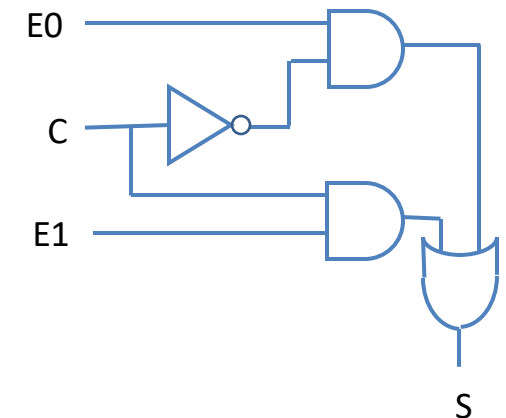
- En VHDL, la description la plus naturelle de circuits logiques combinatoires **simples** est la description de type **flots de données**
 - Pour faire très simple, un ensemble d'équations.

```
entity mux2 is                                architecture flots_de_donnees of mux2 is
  port E0, E1, C: in STD_LOGIC;              begin
  port S: out STD_LOGIC;                      S <= E0 when C else E1;
end;                                           end;
```



- La description de circuits logiques combinatoires **complexes** peut être réalisée avec des descriptions de type **structurelles**
 - Un ensemble de composants et la liste des signaux qui les interconnecte.

```
architecture structurelle of mux2 is
  signal i, j, k : STD_LOGIC;
begin
  u1: work.not port map(C, i);
  u2: work.and port map(E0, i, j);
  u3: work.and port map(C, E1, k);
  u4: work.or port map(j, k, S);
end
```



Logique séquentielle - Introduction

- Logique combinatoire
 - La sortie ne dépend que de l'état des entrées
- Logique séquentielle
 - La sortie dépend de l'état des entrées courant et passé
 - La logique séquentielle a de la mémoire : l'état du circuit séquentiel
 - Valeurs des entrées précédemment ou une représentation interne fonction de ces valeurs
 - Logique combinatoire + bascules pour conserver l'état du circuit
- Logique séquentielle synchrone
 - Les blocs séquentiels changent tous d'état en même temps grâce à un signal d'horloge commun

signal d'horloge

changement d'état sur les fronts montants



Logique séquentielle - Bascule RS

Combinatoire

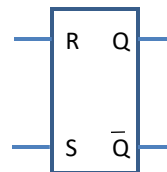
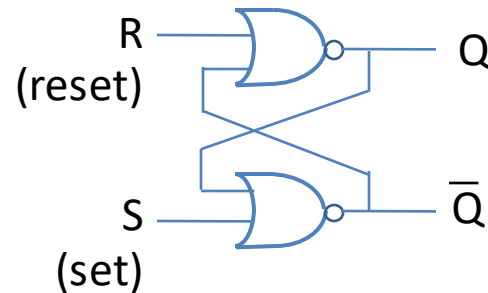


Non-Ou (nor)		
A	B	S
0	0	1
0	1	0
1	0	0
1	1	0

$$1 \text{ nor } x = 0$$

$$0 \text{ nor } x = \neg x$$

Séquentiel

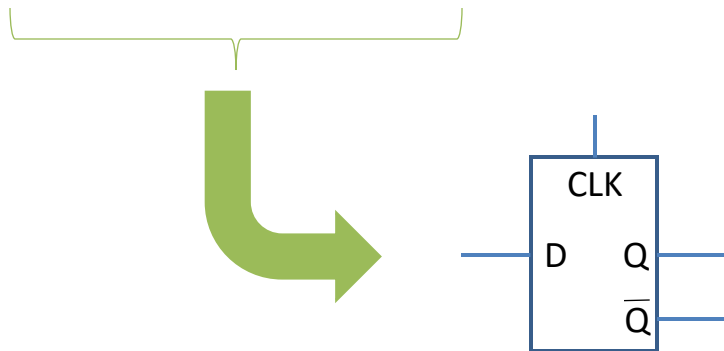
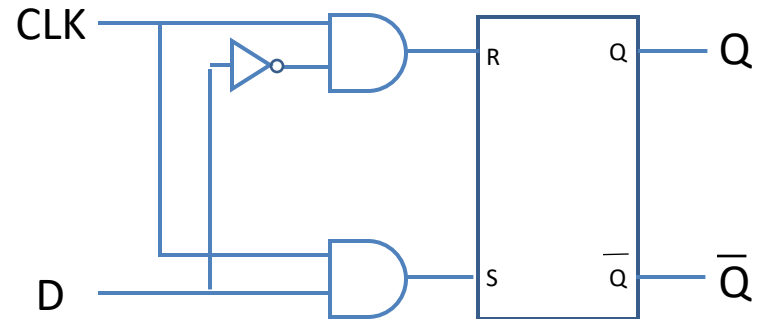


Latch RS			
S	R	Q	$\bar{Q}$
0	0	Q	$\bar{Q}$
0	1	0	1
1	0	1	0
1	1	0	0

Précédentes valeurs

Tant que R et S restent à 0, le circuit reste dans le même état

Logique séquentielle - Verrou



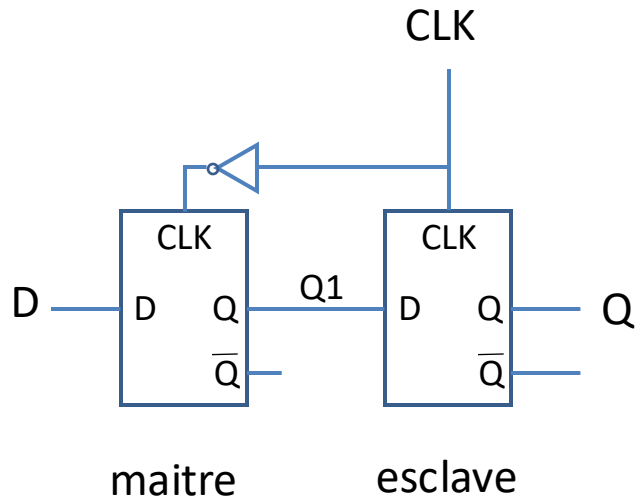
Latch RS						
CLK	D	$\neg D$	S	R	Q	$\neg Q$
0	x	$\neg x$	0	0	Q	$\neg Q$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

Précédentes valeurs

1 sur CLK : D est reproduit sur Q (verrou transparent)

0 sur CLK : la sortie conserve son état (verrou opaque)

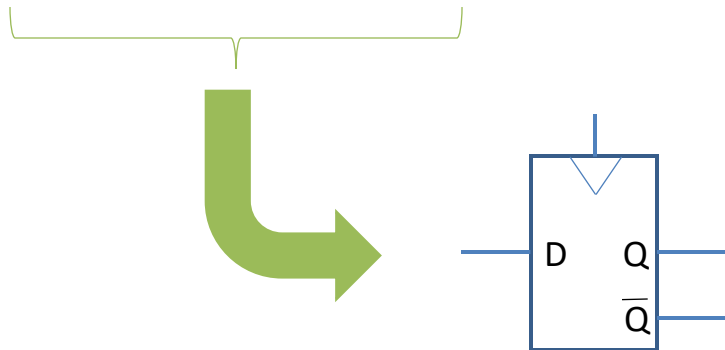
Logique séquentielle - bascule D (maitre-esclave)



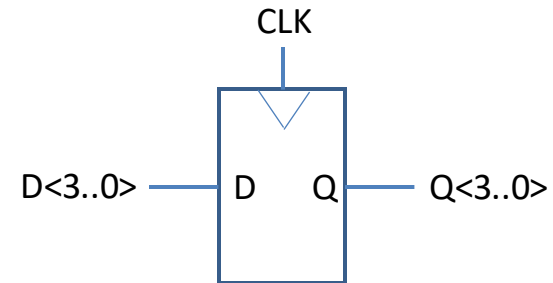
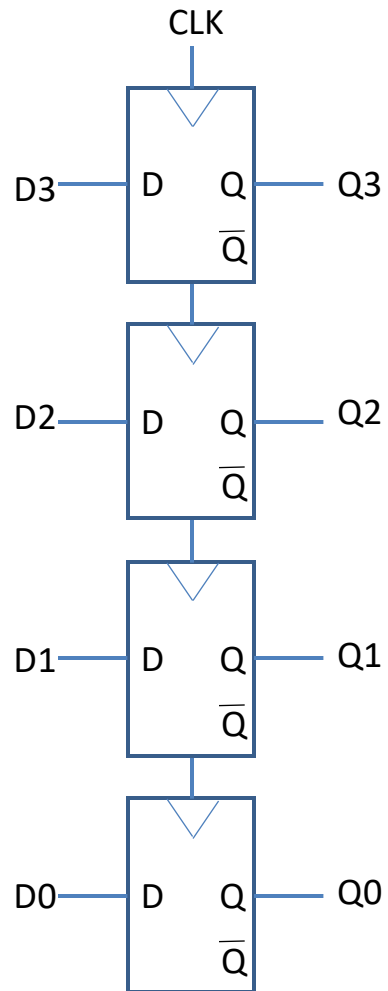
CLK=0 : le maitre est transparent (D est reproduit sur Q1) et l'esclave opaque (conserve sa valeur sur Q quel que soit l'état de D)

CLK passe à 1 : le maitre devient opaque, Q1 mémorise la dernière valeur de D. L'esclave devient transparent, Q affiche cette dernière valeur de D.

La bascule se fait quand CLK passe de 0 à 1 : bascule sensible au front montant.

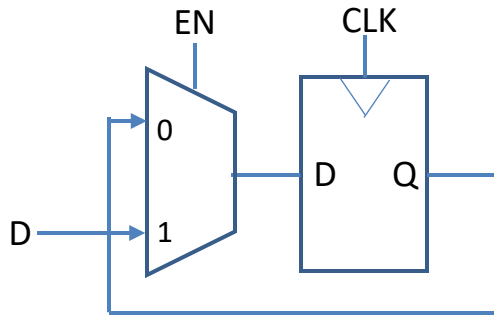


Logique séquentielle - Registre

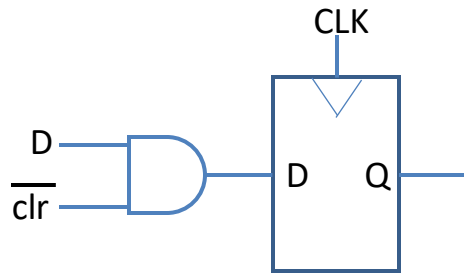


Logique séquentielle

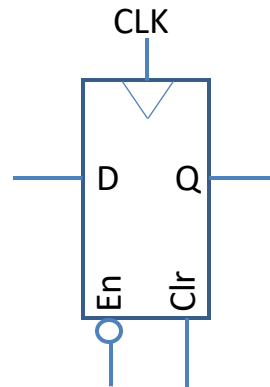
bascule D avec autorisation et reset



La bascule se fait quand CLK passe de 0 à 1 sauf si EN (ENable) est à 0 auquel cas la bascule conserve sa valeur

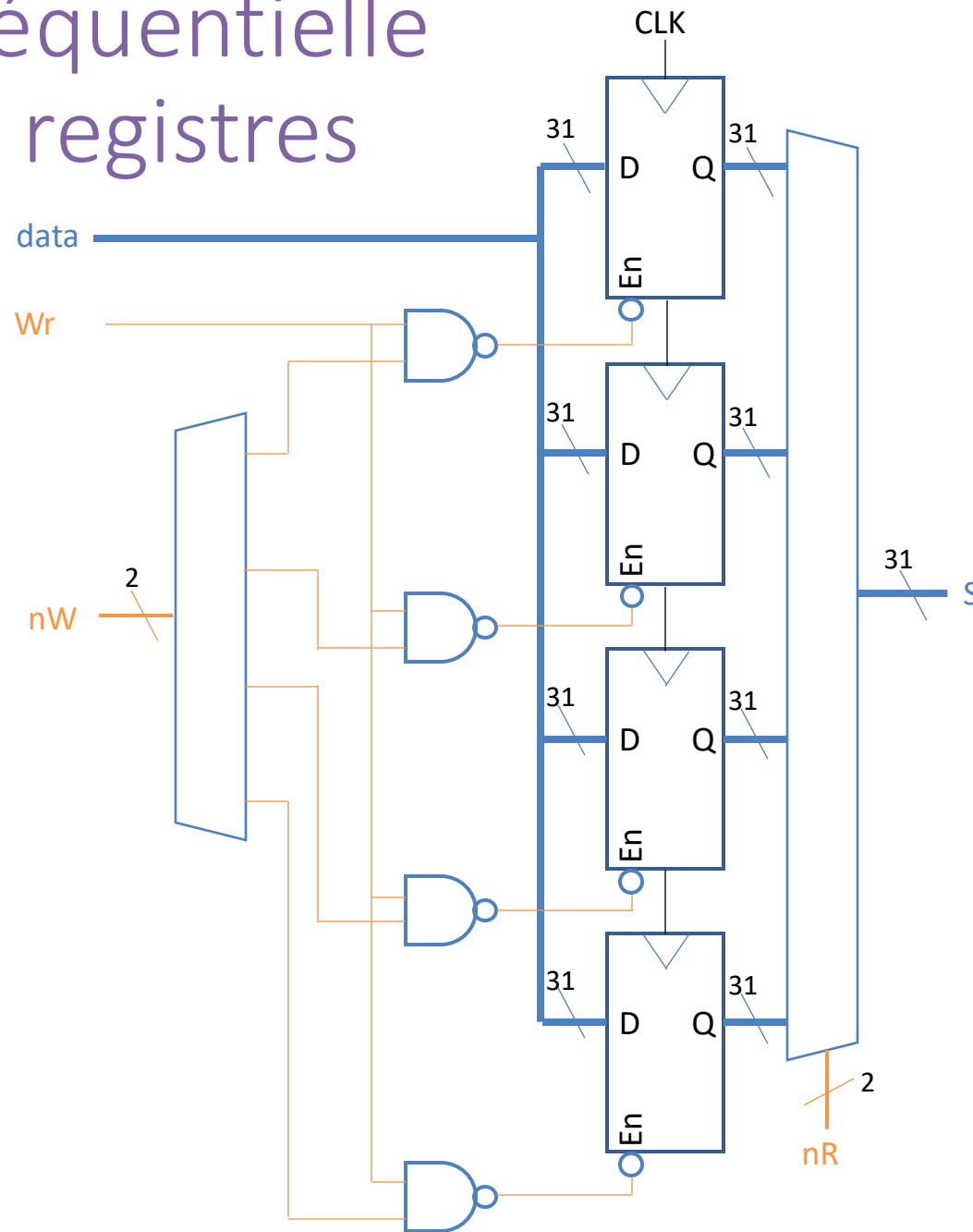


clr=1 : Q=0 sur le front montant de CLK (reset synchrone)



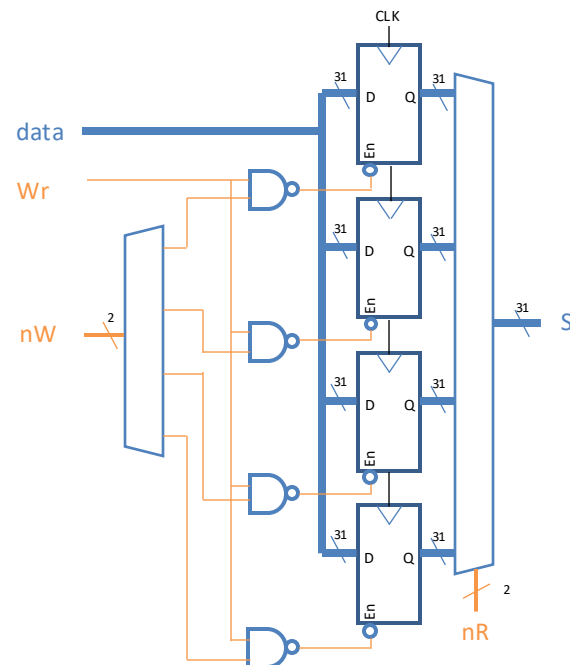
Logique séquentielle

Banc de 4 registres



Implémentation en VHDL de la logique séquentielle

- La logique séquentielle que nous utilisons est une logique séquentielle synchrone de l'horloge (CLK)
- Cela se modélise par un process activé sur le front montant de CLK et éventuellement sur d'autres signaux (reset par exemple) qui est une instruction concurrente comme les autres



```
architecture comportementale of banc_registres is

    type ramtype is array (3 downto 0) of STD_LOGIC_VECTOR(31 downto 0);
    signal mem: ramtype;

begin

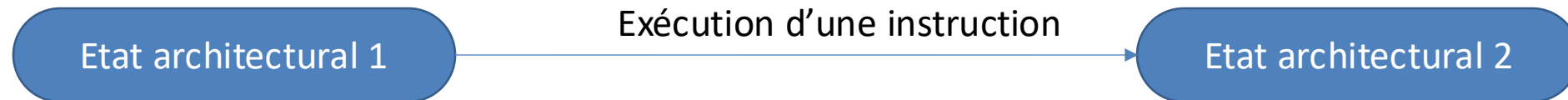
    ecriture: process(clk)
    begin
        if falling_edge(clk) then
            if Wr = '1' then mem(to_integer(nW)) <= data;
            end if;
        end if;
    end process;

    s <= mem(to_integer(nR));

end architecture;
```

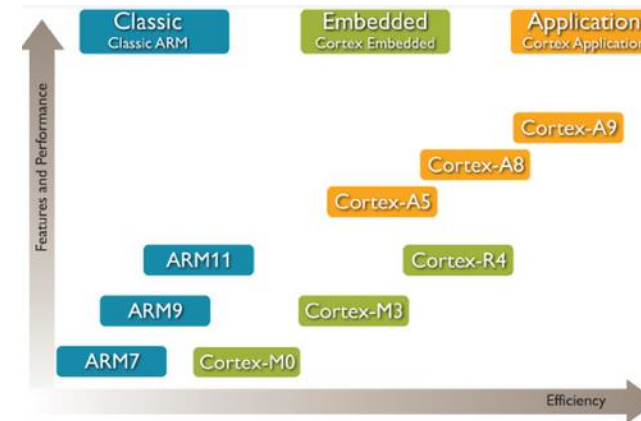
Architecture ARM

- Cf. ressources sur Moodle, module archi3 de L2 info.
 - Architecture load/store : LDR, STR
 - ADD, SUB, AND, ...
 - Bcc
- L'architecture est définie par le jeu d'instructions et l'état architectural (registres généraux, registre d'état, mémoire d'instructions, mémoire de données)



Microarchitecture

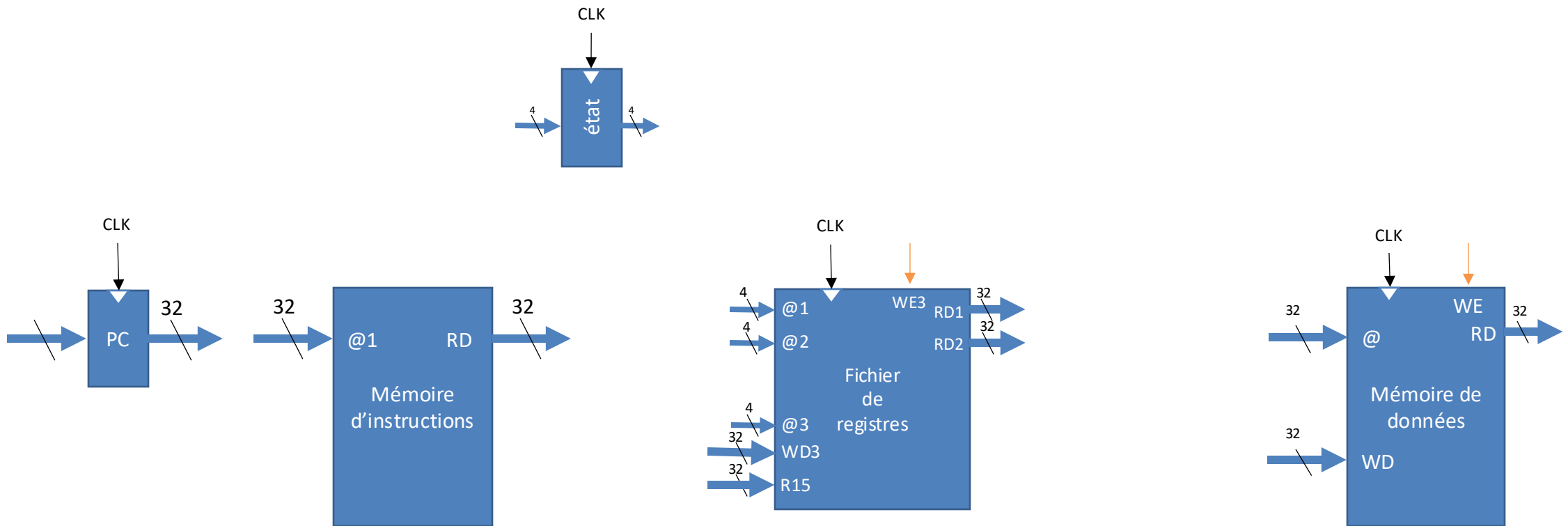
- Connexion entre la logique (combinatoire et séquentielle) et l'architecture
- Agencement des registres, ALUs, mémoires et autre logique
- Plusieurs microarchitectures sont possibles pour une architecture
 - Compromis performance, énergie, coût, ...



- Chemins de données
 - 32 bits pour l'ARM
 - Contient les mémoires, les registres, les ALUs, des multiplexeurs
- Unité de contrôle
 - Contrôle le chemin de données (sens des multiplexeurs, signaux d'écriture des mémoires, contrôle de l'ALU, ...)

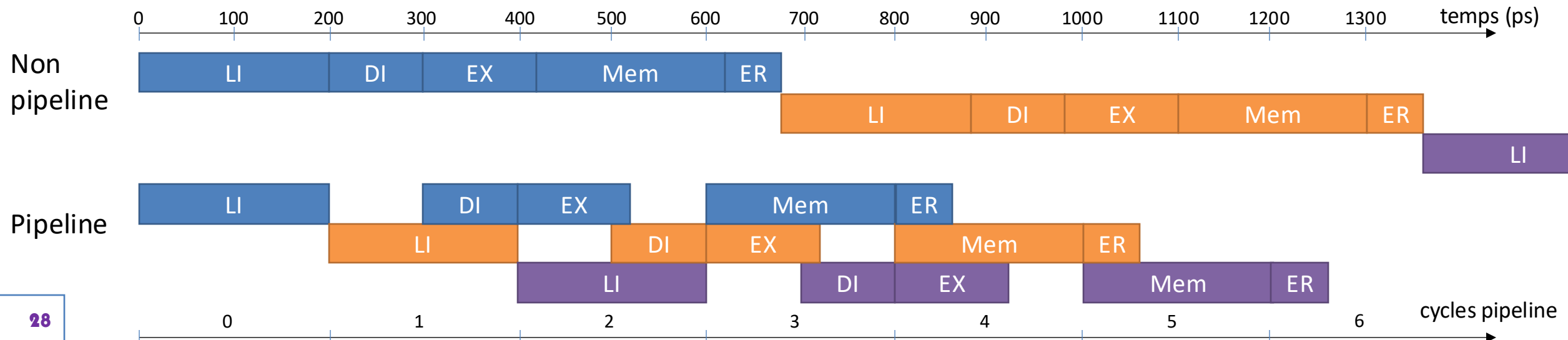
Mémorisation de l'état de la microarchitecture

- Signal noir : l'horloge de cadencement (CLK)
- Signaux **verts** : signaux de contrôle
- Signaux **bleus** : chemin de données



Processeur pipeline

- Le traitement d'une instruction est divisé en cinq étages. 5 instructions peuvent être en cours de traitement, chacune dans un étage.
 - Chargement de l'instruction depuis la mémoire (LI)
 - Lecture des opérandes source (dans le banc de registre ou dans l'instruction elle-même pour les valeurs immédiates) et décodage pour produire les signaux de contrôle (DI)
 - Calcul dans l'ALU (EX)
 - Lecture ou écriture de la mémoire de données (Mem)
 - Ecriture du résultat dans le banc de registres (ER)



Exercice pipeline idéal et temps de cycle (1)

- On suppose que les étages du chemin de données ont les durées suivantes :

LI	DI	EX	Mem	ER
250 ps	350 ps	150 ps	300 ps	200 ps

- On suppose que les instructions exécutées par le processeur sont décomposées comme suit:

ALU	Branch	LDR	STR
45%	20%	20%	15%

- Quel est le temps de cycle d'un processeur pipeline et d'un processeur non pipeline ? Quelles sont les fréquences respectives ?
- Quelle est la latence totale d'une instruction LDR sur un processeur pipeline et un processeur non pipeline ?
- Supposons que l'on peut couper un étage du pipeline en 2. Lequel couper ? Nouveau temps de cycle du processeur pipeline ? Nouvelle fréquence ?
- Quel est le taux d'utilisation de la mémoire de données ?
 - En supposant que les accès à la mémoire de données durent un cycle
- Quel est le taux d'utilisation du registre en écriture du banc de registres ?

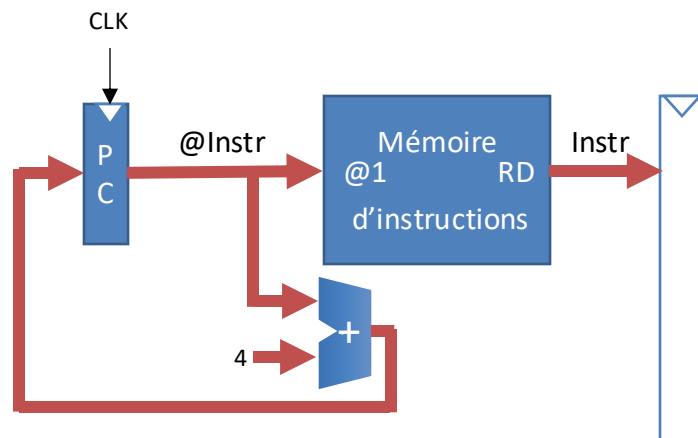
Exercice rendement pipeline idéal

- Nombre de cycles minimum pour exécuter n instructions sur un pipeline de k étages ?

Exécution de LDR R2, [R0, #40] (1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
cond				0	1	0	P	U	B	W	L	Rn				Rd				addr_mode															
1110				0	1	0	1	1	0	0	1	0000				0010				0000 0100 0000															
Always				LDR indexé immédiat								0				2				40															

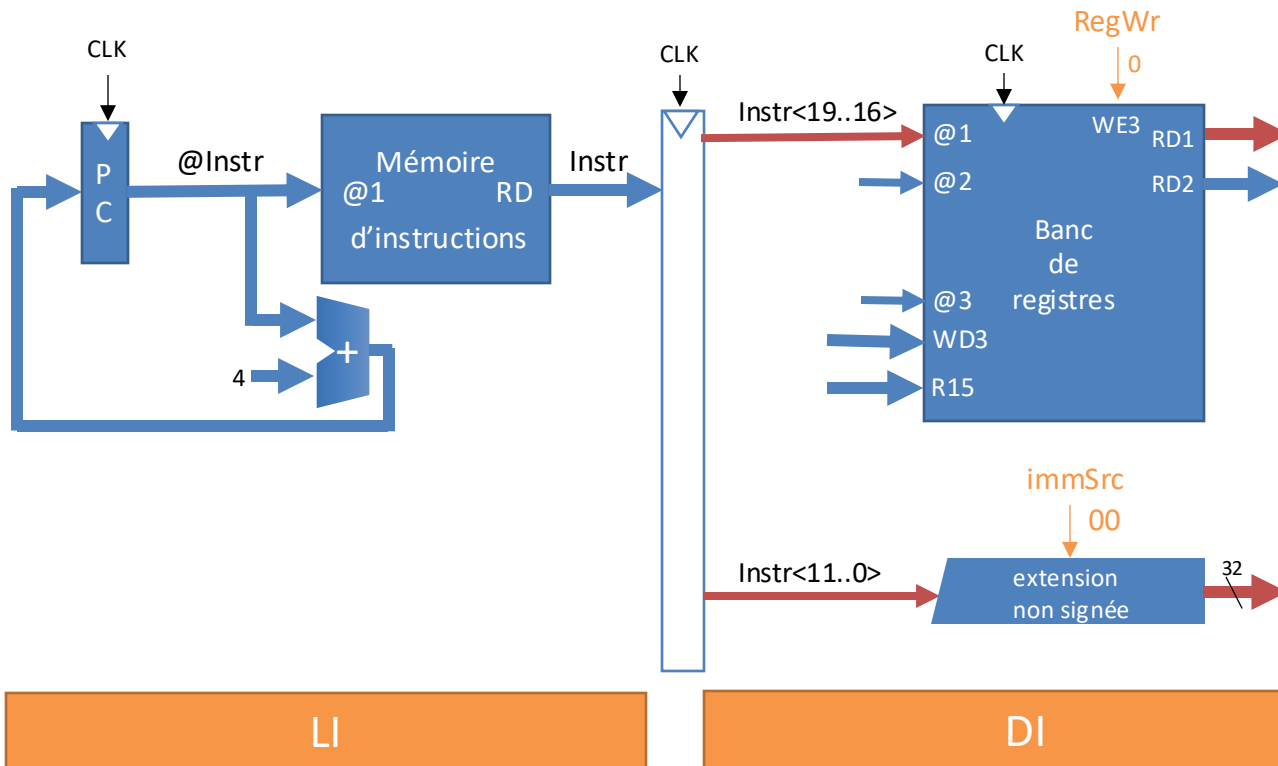
Chargement de l'instruction



Exécution de LDR R2, [R0, #40] (1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1110				0	1	0	1	1	0	0	1	0000				0010				0000 0100 0000													
Always				LDR indexé immédiat								0				2				40													

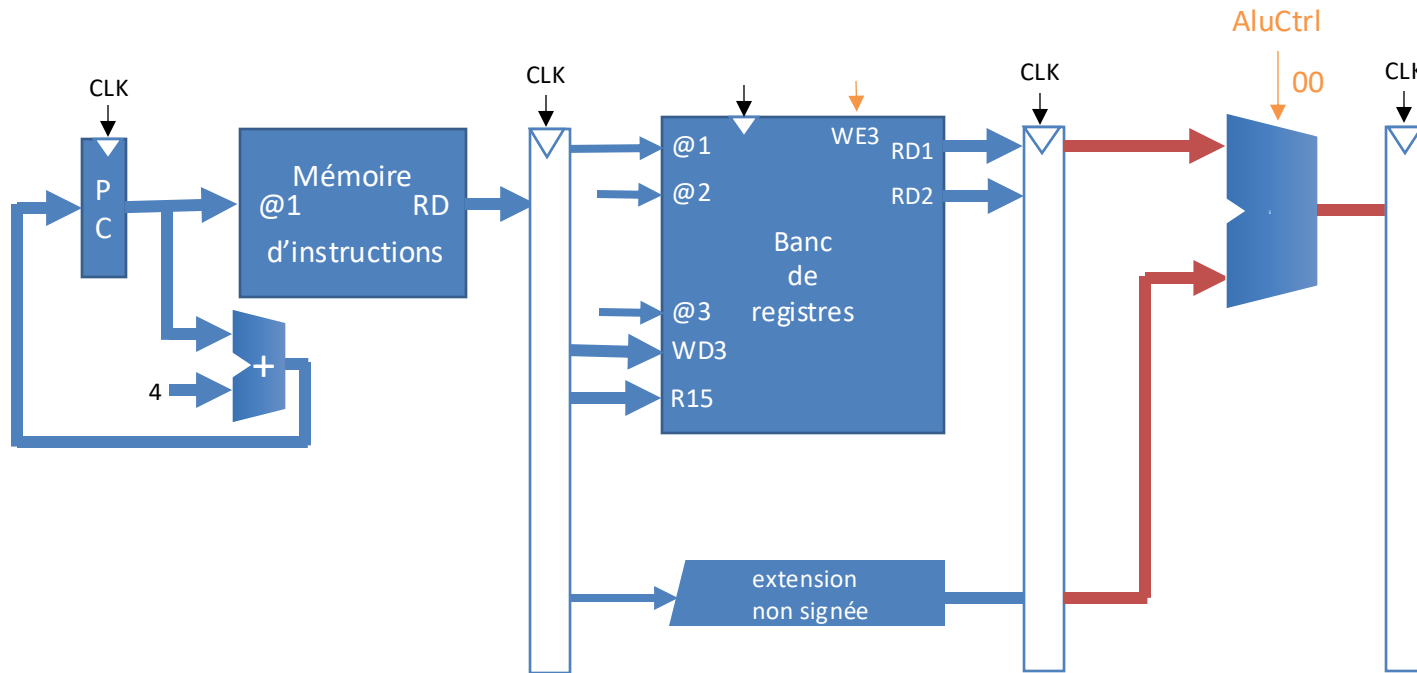
Lecture registre d'adresse
Extension valeur immédiate



Exécution de LDR R2, [R0, #40] (2)

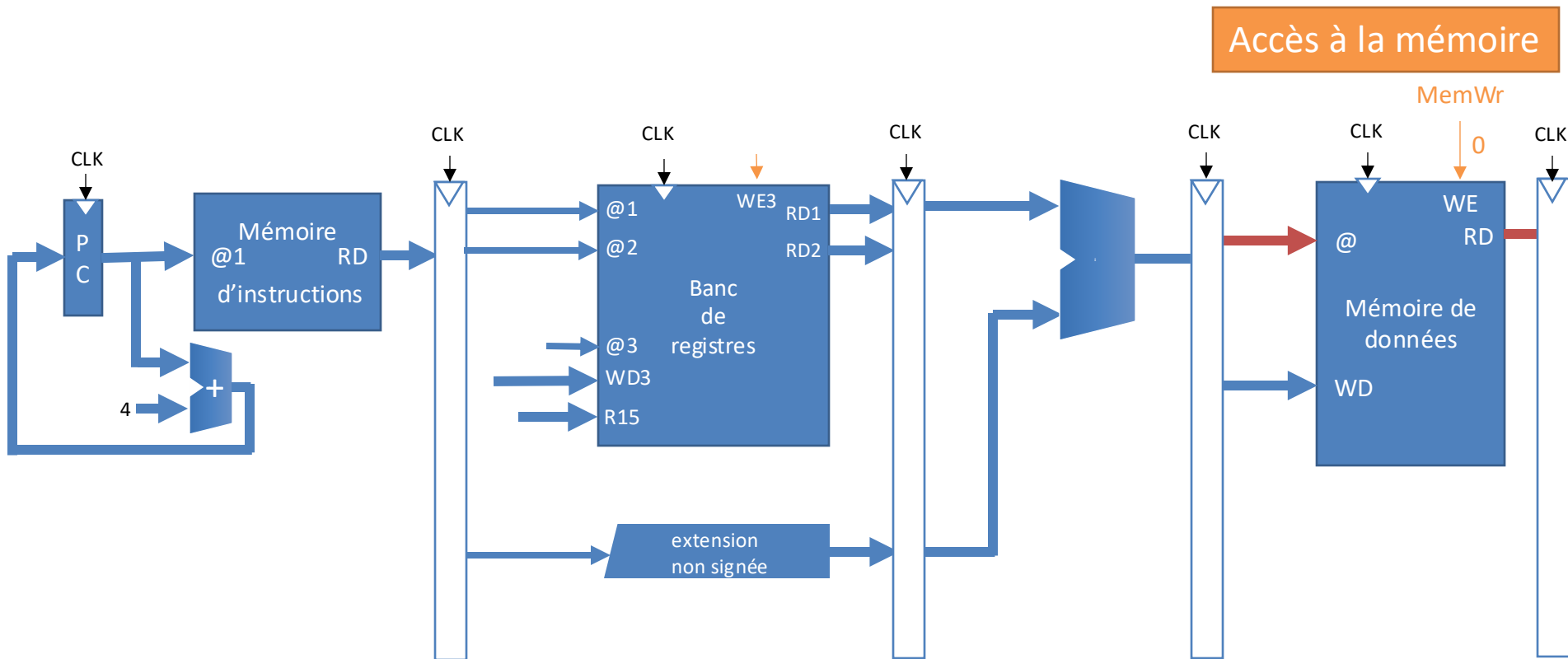
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1110				0	1	0	1	1	0	0	1	0000				0010				0000 0100 0000													
Always				LDR indexé immédiat								0				2				40													

Calcul de l'adresse



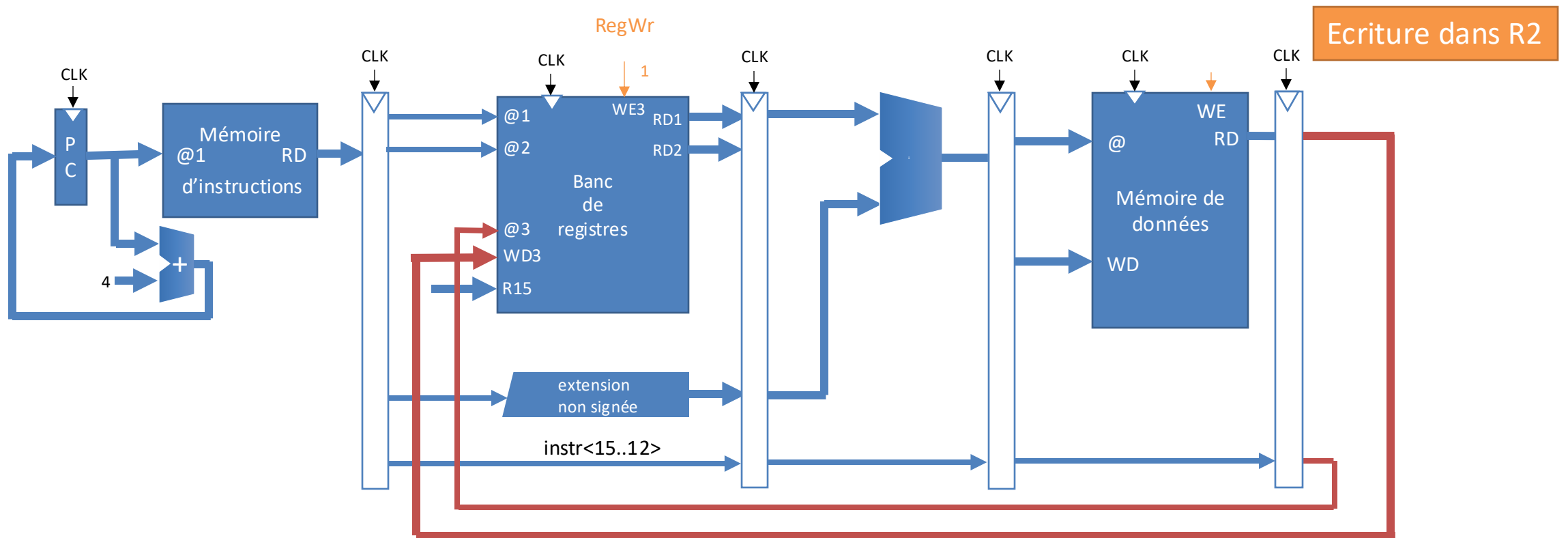
Exécution de LDR R2, [R0, #40] (3)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1110				0	1	0	1	1	0	0	1	0000				0010				0000 0100 0000											
Always				LDR indexé immédiat								0				2				40											



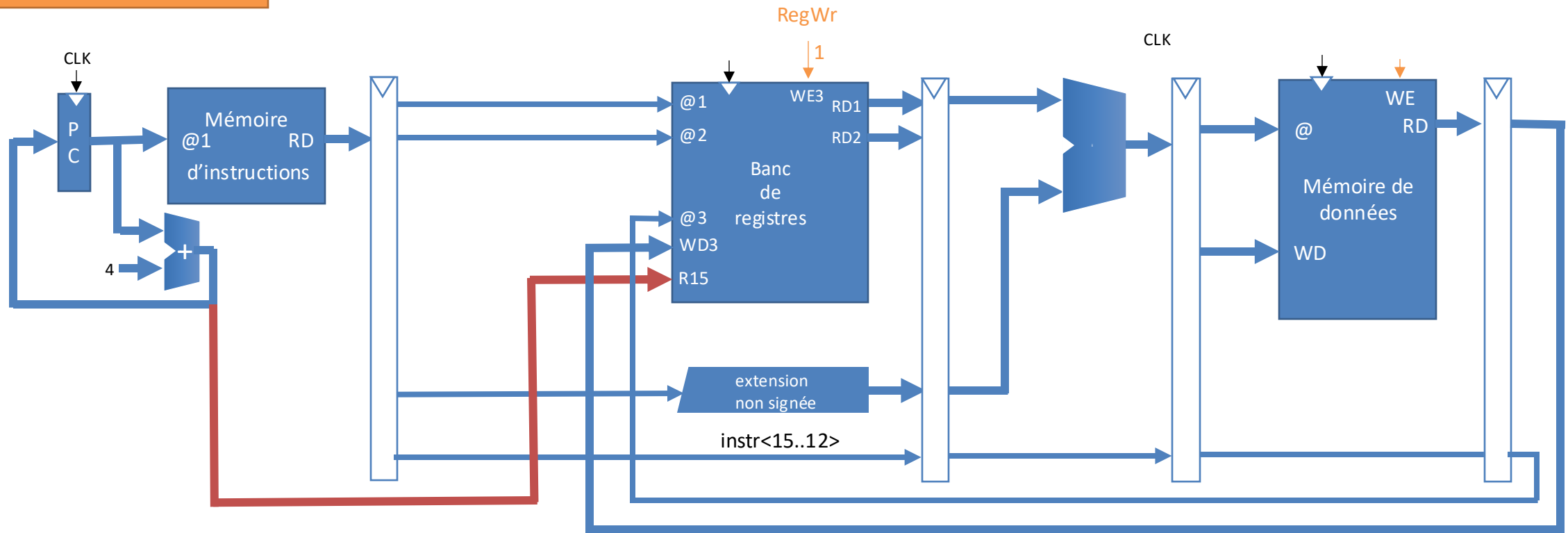
Exécution de LDR R2, [R0, #40] (3)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1110				0	1	0	1	1	0	0	1	0000				0010				0000 0100 0000													
Always				LDR indexé immédiat								0				2				40													

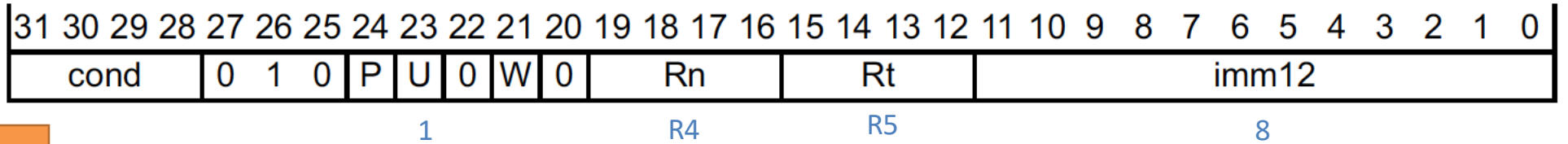


Exécution de LDR R2, [R0, #40] (5)

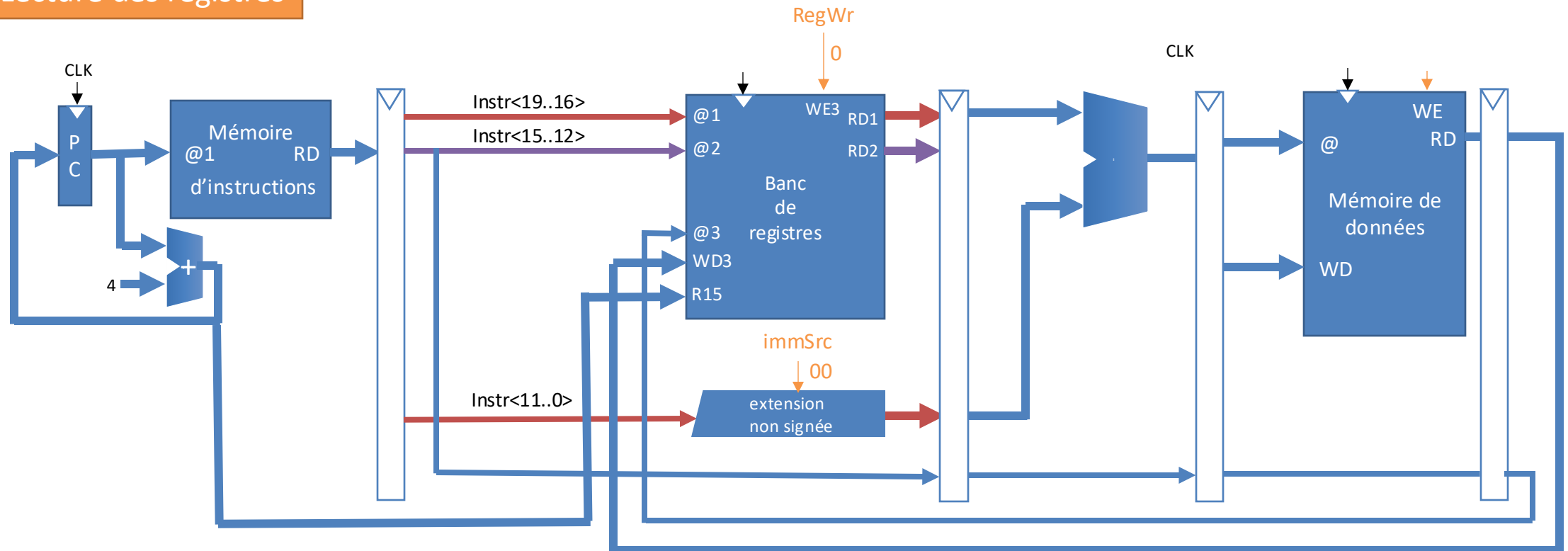
Ecriture de R15 (2)



Exécution d'un STR (1) – Ex : STR R5, [R4, #8]



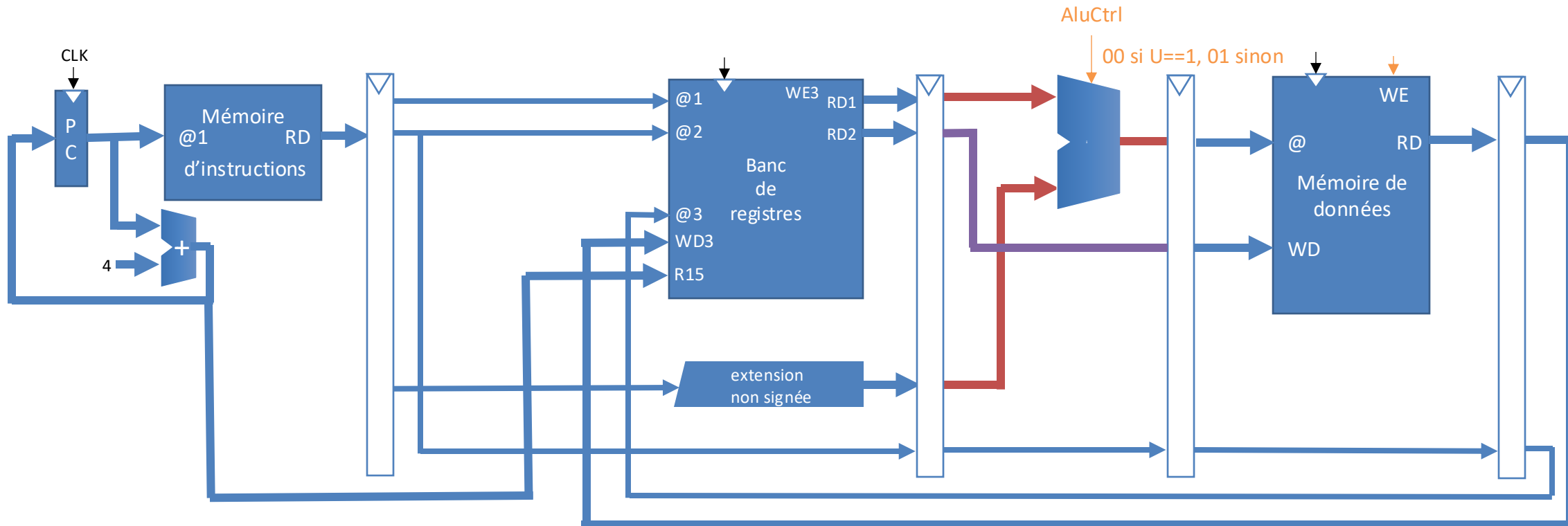
Lecture des registres



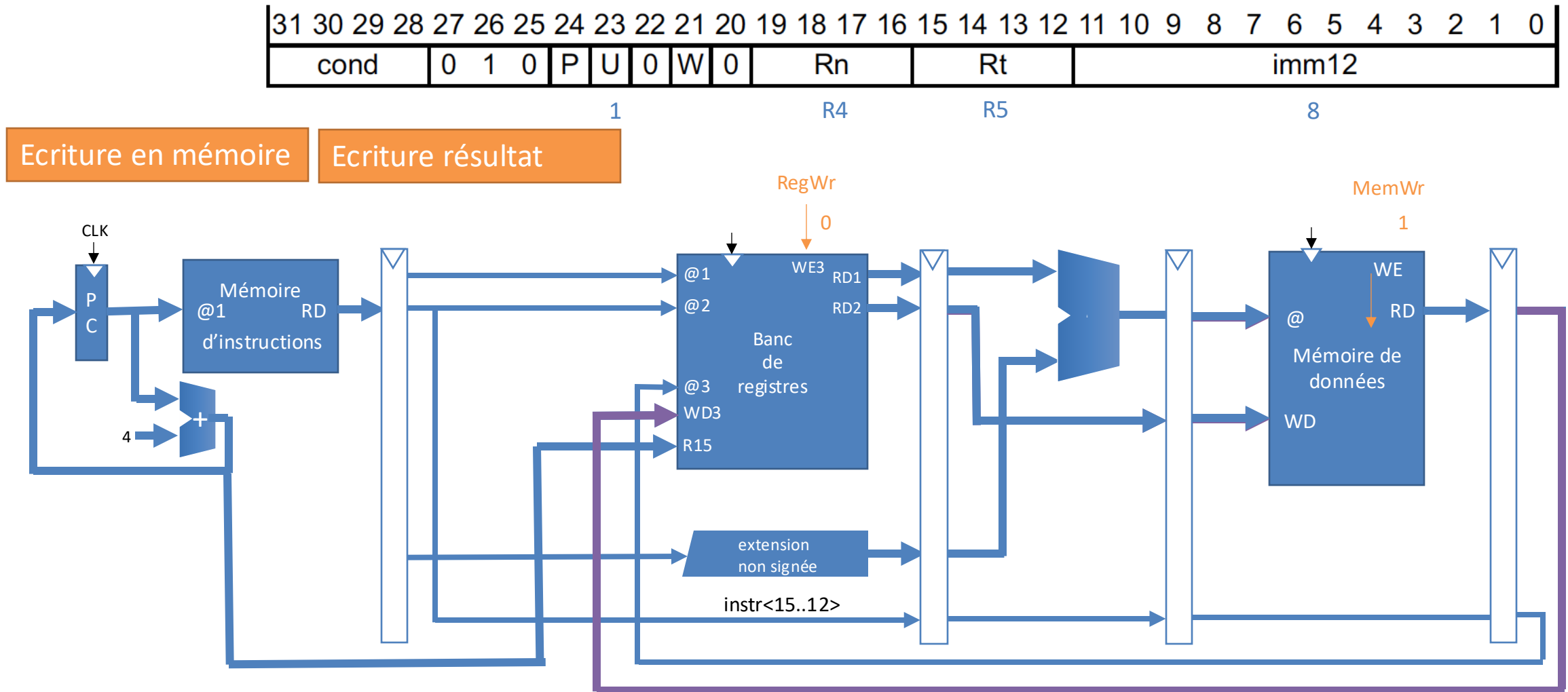
Exécution d'un STR (2) – Ex : STR R5, [R4, #8]

Calcul

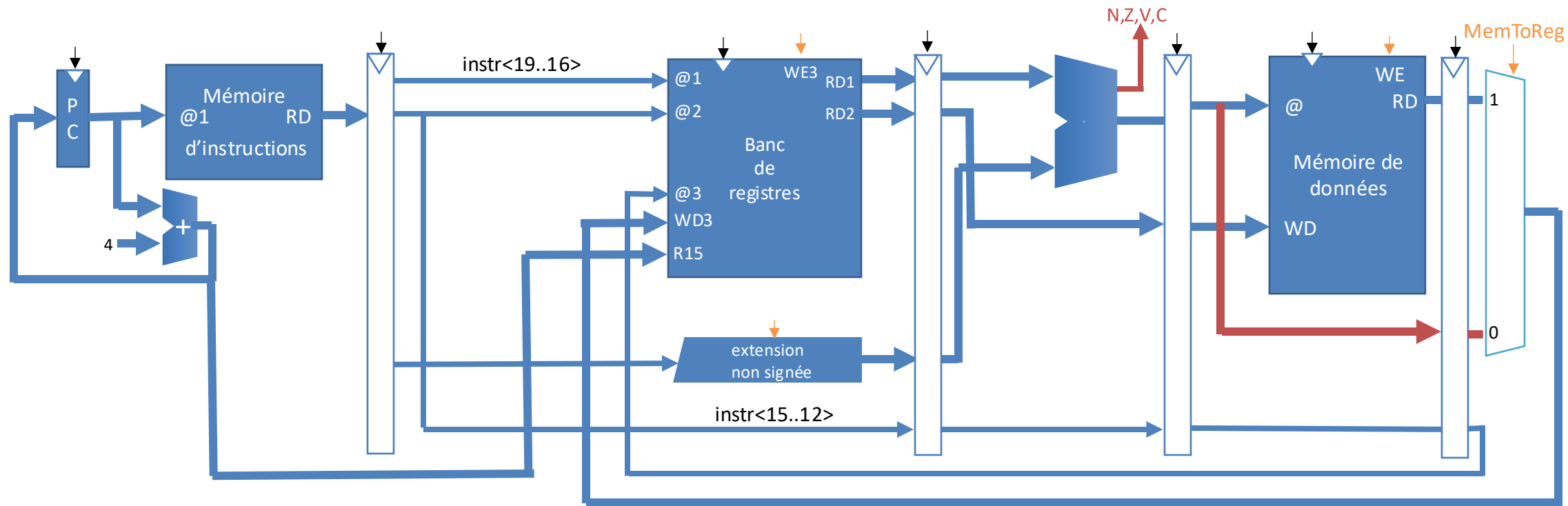
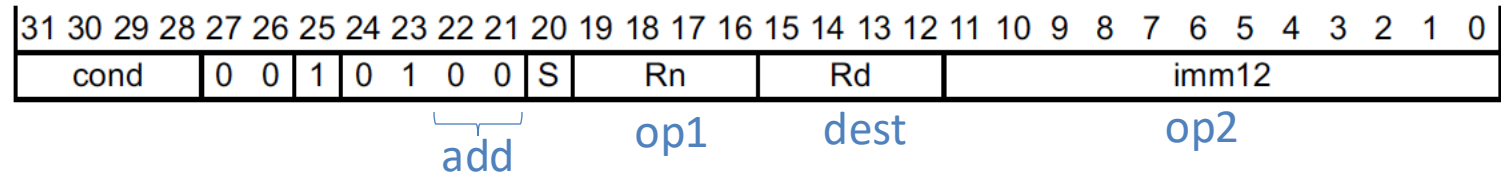
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	1	0	P	U	0	W	0	Rn				Rt				imm12											
1											R4				R5				8												



Exécution d'un STR (3) – Ex : STR R5, [R4, #8]

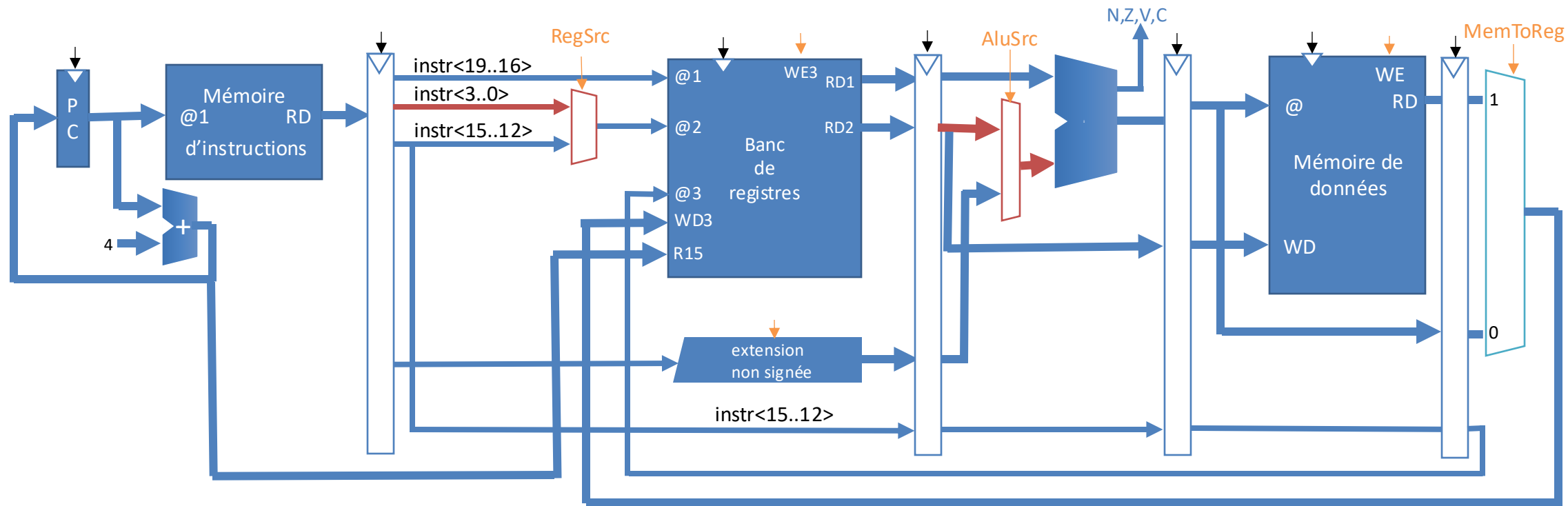


Instruction de calcul registre/immédiat



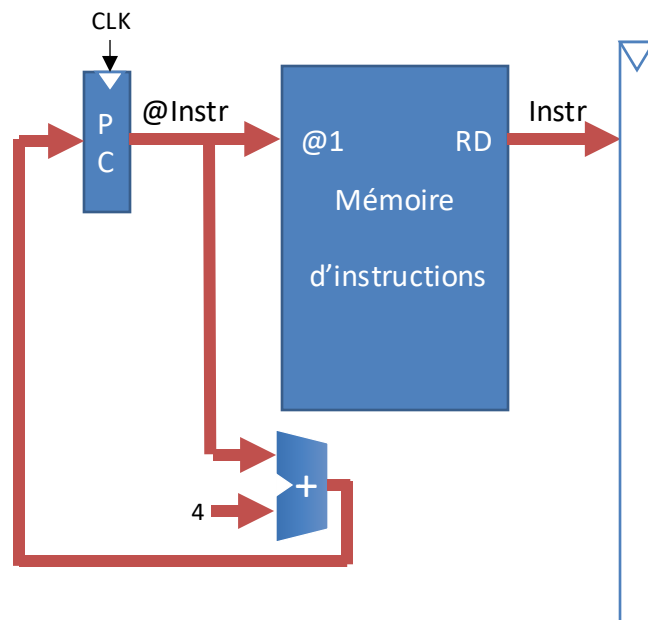
Instruction de calcul registre/registre

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
cond		0		0		0		0		1		0		0		S		Rn				Rd												Rm					
												add				op1				dest																op2			



Instruction de branchement (étage LI)

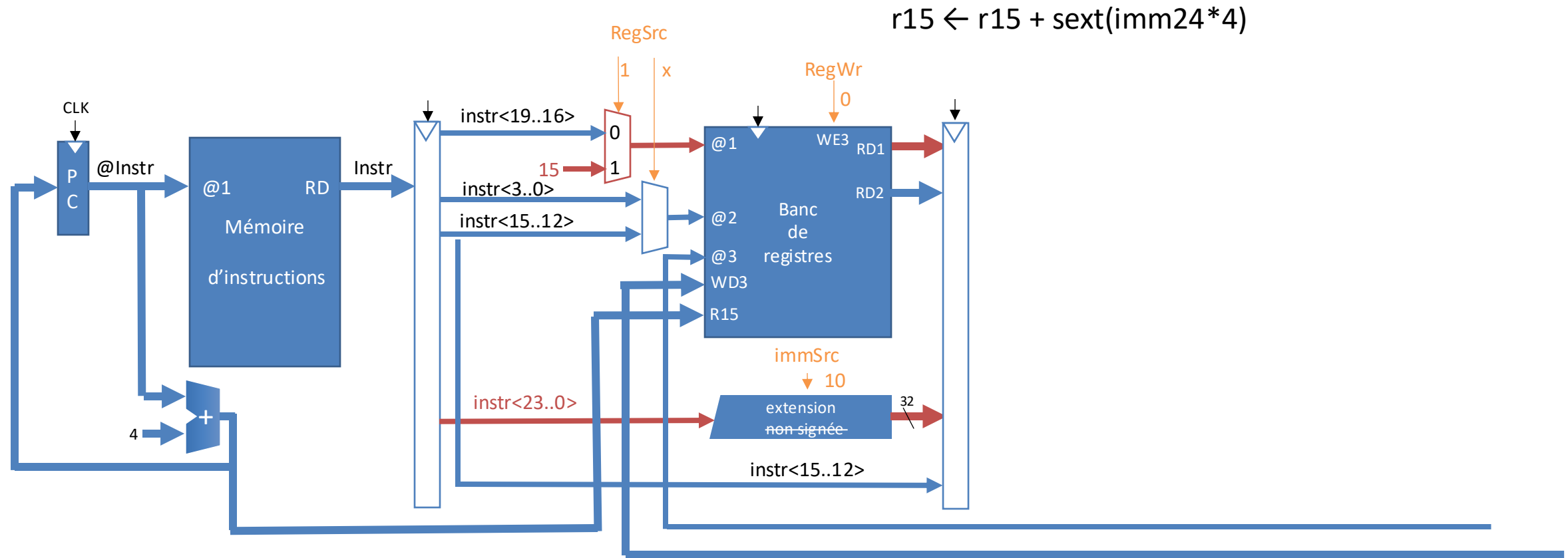
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				1	0	1	0	imm24																							



MemToReg

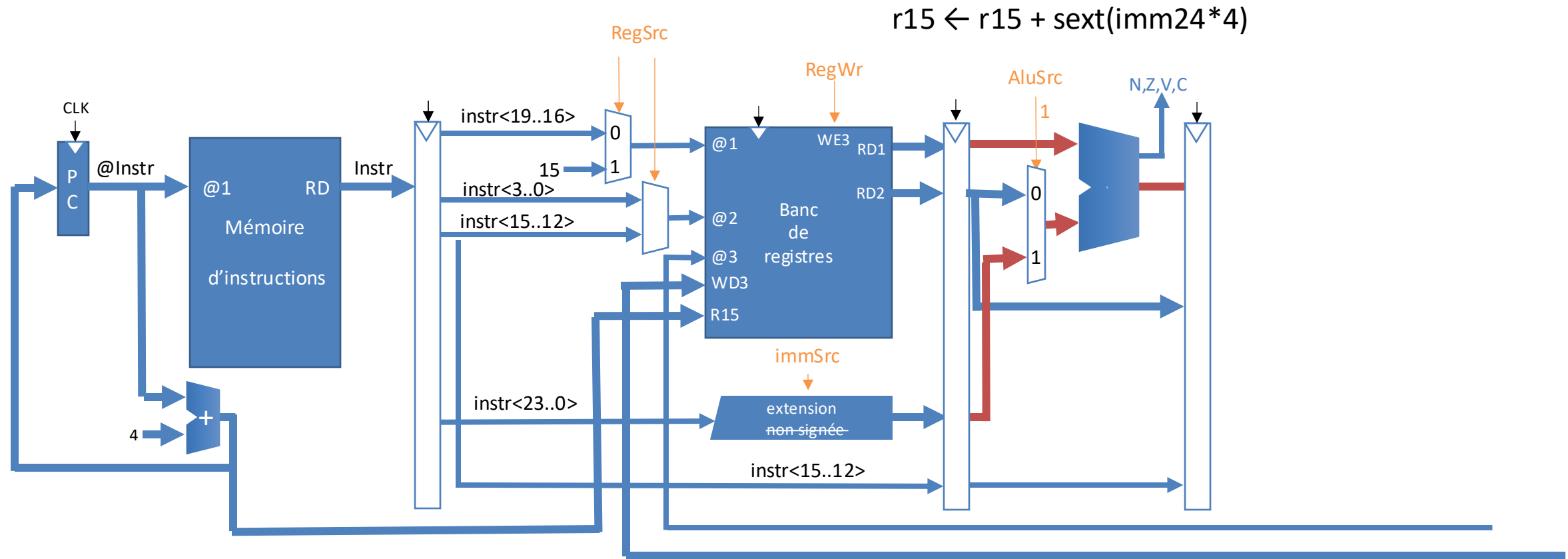
Instruction de branchement (étage DI)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				1	0	1	0	imm24																							

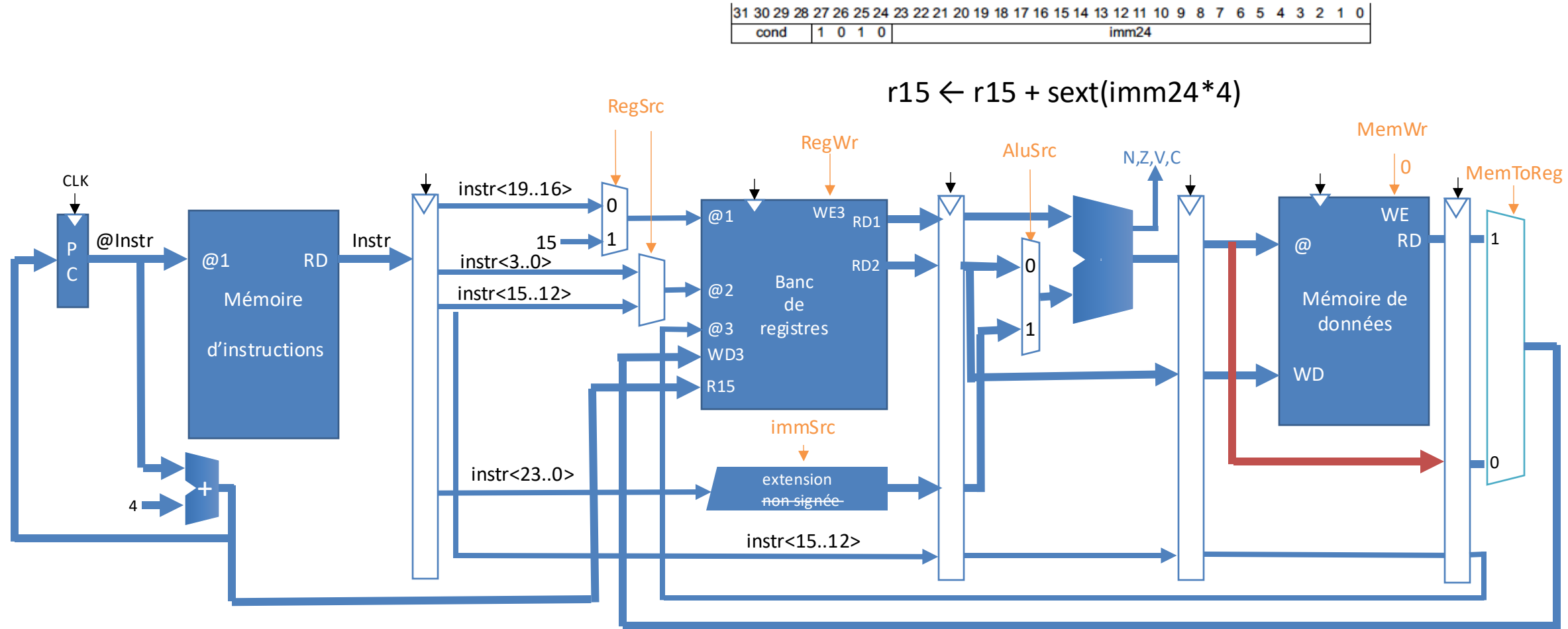


Instruction de branchement (étage EX)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				1	0	1	0	imm24																							

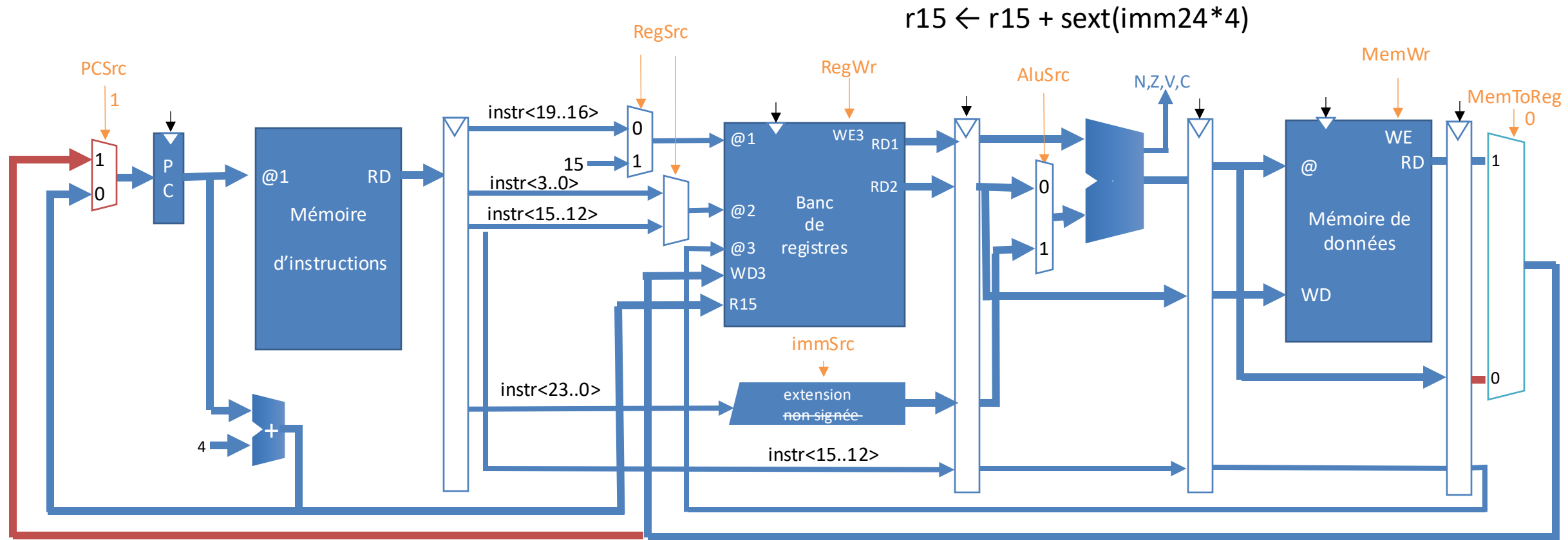


Instruction de branchement (étage Mem)



Instruction de branchement (étage ER)

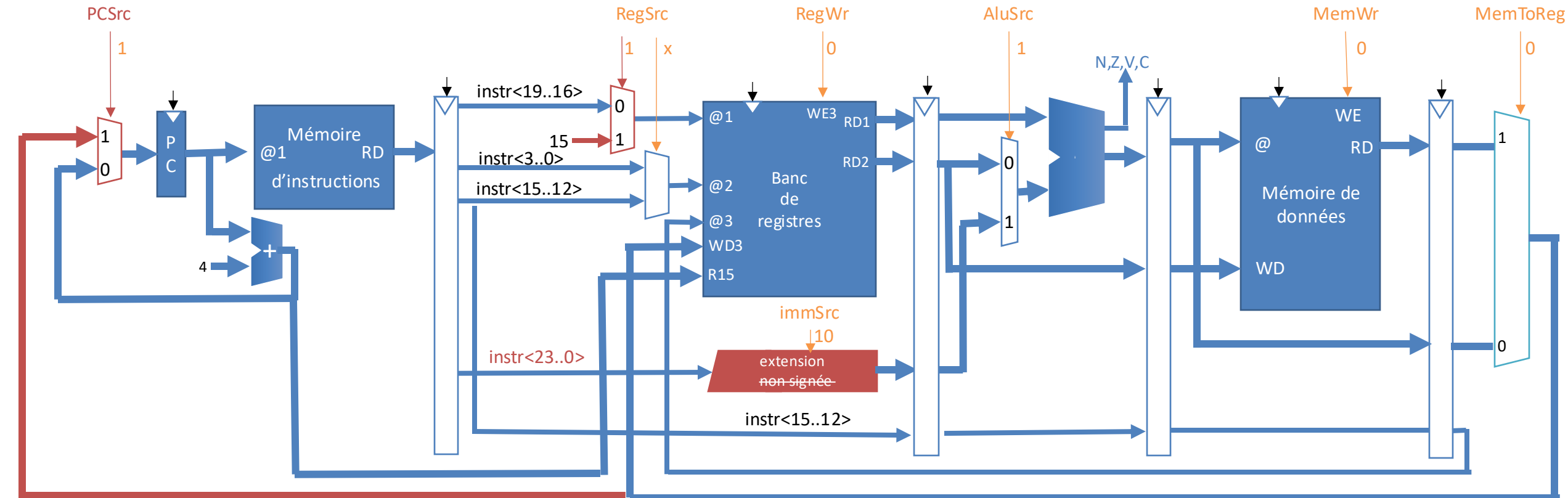
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				1	0	1	0	imm24																							



Instruction de branchement (résumé des ajouts)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				1	0	1	0	imm24																							

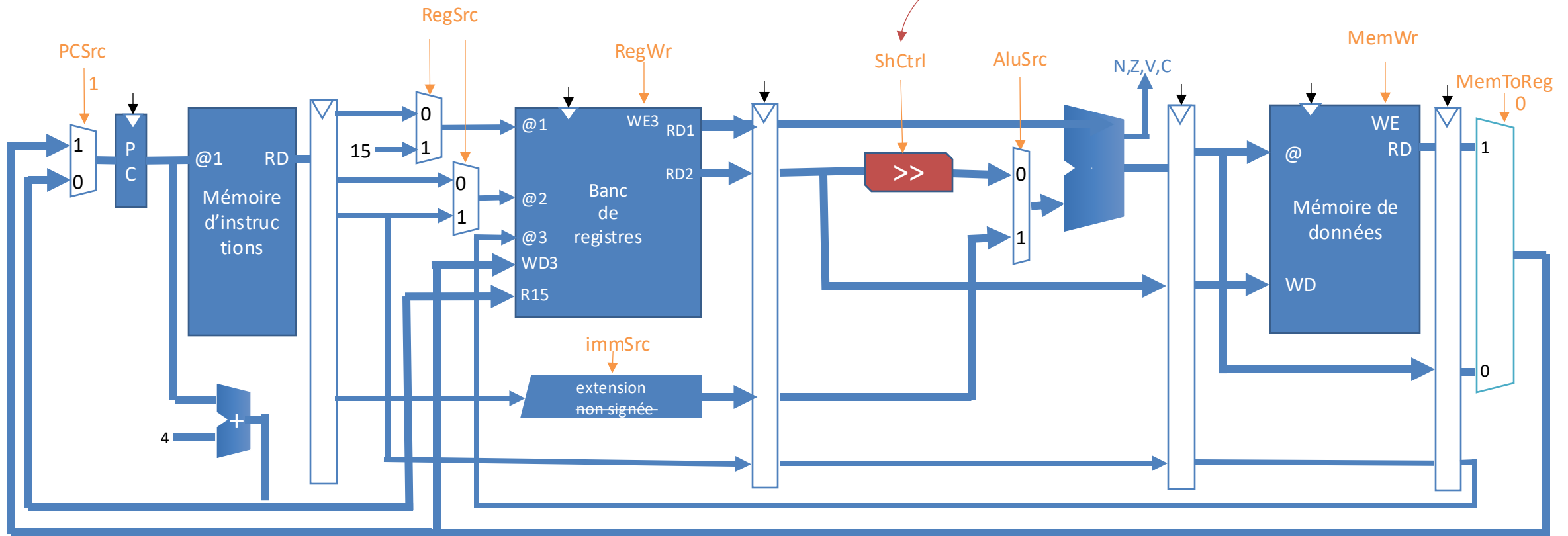
$$r15 \leftarrow r15 + \text{sext}(\text{imm24} * 4)$$



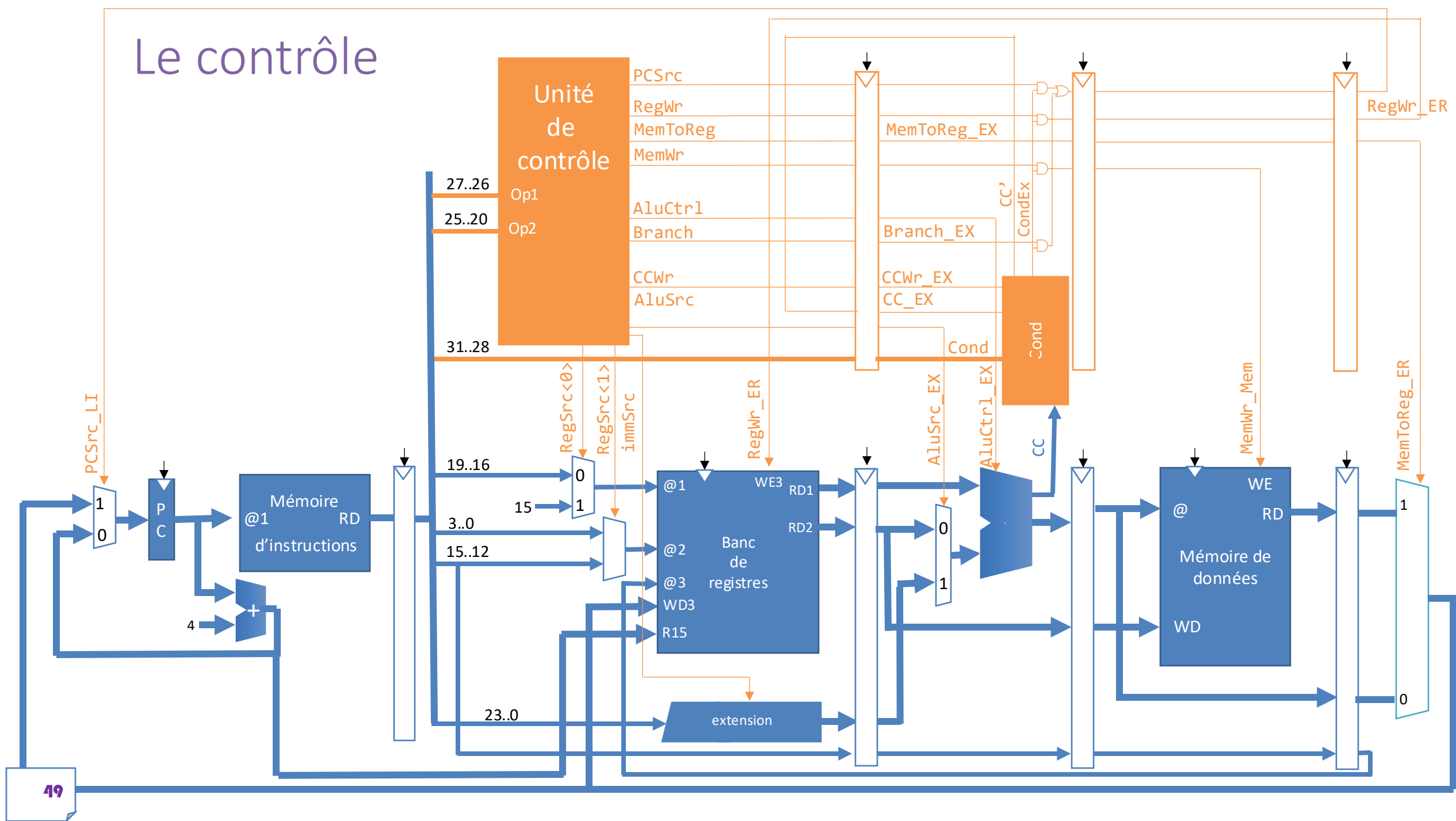
Ajout d'un mode d'adressage : ADD R3, R2, R1, LSL #2

$R3 \leftarrow R2 + (R1 \ll 2)$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond		0	0	0	0	1	0	0	S	Rn			Rd			imm5					type	0	Rm								



Le contrôle



Génération du signal AluCtrl

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond		0	0	0	0	1	0	0	S	Rn				Rd												Rm					

Calcul reg/reg

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond		0	0	1	0	1	0	0	S	Rn			Rd			imm12															

Calcul reg/imm

→ Le bit S indique si les flags (N, Z, V, C) doivent être mémorisés.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	0	1	1	0	1	0	1	Rn				(0)(0)(0)(0)				imm12											

CMP

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	1	0	P	U	0	W	0	Rn				Rt				imm12											

STR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	1	0	P	U	0	W	1	Rn				Rt				imm12											

LDR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				1	0	1	0	imm24																							

Branchement

→ sauf si c'est un branchement.

Les bits 26 et 27 indiquent que c'est une opération de calcul.
Le bit 25 indique le mode d'adressage (reg+reg ou reg+imm).
Les bits 21 à 24 indiquent l'opération que doit faire l'ALU :
0100: ADD – 0010: SUB – 0000: AND – 1100: ORR

Les bits 26 et 27 indiquent que c'est un STR ou un LDR.
Le bit 20 indique si c'est un LDR ou un STR.
L'UAL doit faire un ADD si U vaut 1, un SUB si U vaut 0.

Les bits 26 et 27 indiquent que c'est un branchement.
L'UAL doit faire un ADD.

Instr<27..26>	Instr<24..21>	AluCtrl<1..0>
10 (Branch)	xxxx	00 (+)
00 (calcul)	0100	00 (+)
	0010	01 (-)
	0000	10 (Et)
	1100	11 (Ou)
	1010	01(-)

LDR ou STR		
Instr<27..26>	Instr<23>	AluCtrl<1..0>
01	1	00 (+)
01	0	01 (-)

Génération des autres signaux (1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	0	0	0	1	0	0	S	Rn				Rd								Rm							

Calcul reg/reg

$Rn+Rm \rightarrow \text{RegSrc}<1>=0, \text{RegSrc}<0>=0, \text{RegWr}=1$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	0	1	0	1	0	0	S	Rn				Rd				imm12											

Calcul reg/imm

$Rn+Rm \rightarrow \text{RegSrc}<1>=0, \text{RegSrc}<0>=0, \text{RegWr}=1$
 $\text{ImmSrc}<1..0>=00$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	0	0	1	0	1	0	1	Rn				(0)(0)(0)(0)				imm5				type		0	Rm				

CMP

$Rn-Rm \rightarrow \text{RegSrc}<1>=0, \text{RegSrc}<0>=0$
 $\text{RegWr}=0$

AluSrc

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	1	0	P	U	0	W	0	Rn				Rt				imm12											

STR

$Rn+imm \rightarrow \text{RegSrc}<0>=0$
 $Rt \text{ à écrire en mémoire} \rightarrow \text{RegSrc}<1>=1$
 $\text{RegWr}=0$

MemWr

$\text{ImmSrc}<1..0>=01$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				0	1	0	P	U	0	W	1	Rn				Rt				imm12											

LDR

$Rn+imm \rightarrow \text{RegSrc}<0>=0$
 $\text{RegWr}=1$

MemToReg

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cond				1	0	1	0	imm24																							

Branchement

$R15+ext(imm24) \rightarrow \text{RegSrc}<0>=1$
 $\text{ImmSrc}<1..0>=10, \text{RegWr}=0$

Branch, $\text{RegSrc}<0>$

	Instr<27..26>	Instr<25>	Instr<20>	Branch	MemToReg	MemWr	AluSrc	ImmSrc<1..0>	RegWr	RegSrc<1..0>
Calcul reg/reg	00	0	x	0	0	0	0	xx	1	00
Calcul reg/imm	00	1	x	0	0	0	1	00	1	x0
LDR	01	x	1	0	1	0	1	01	1	x0
STR	01	x	0	0	x	1	1	01	0	10
B	10	x	x	1	0	0	1	10	0	x1

Génération des autres signaux (2)

- Il manque PCSrc et tous les signaux pour gérer les codes conditions...
 - On a pas le temps mais rien ne vous empêche de tenter de le faire.
- Vous pouvez vous amuser à prendre une autre instruction de l'ARM (ou un autre mode d'adressage de LDR, STR) et voir comment modifier le pipeline pour que votre processeur soit capable de l'exécuter.

Vue (un peu) plus abstraite du pipeline

LDR R2, [R0, #40]

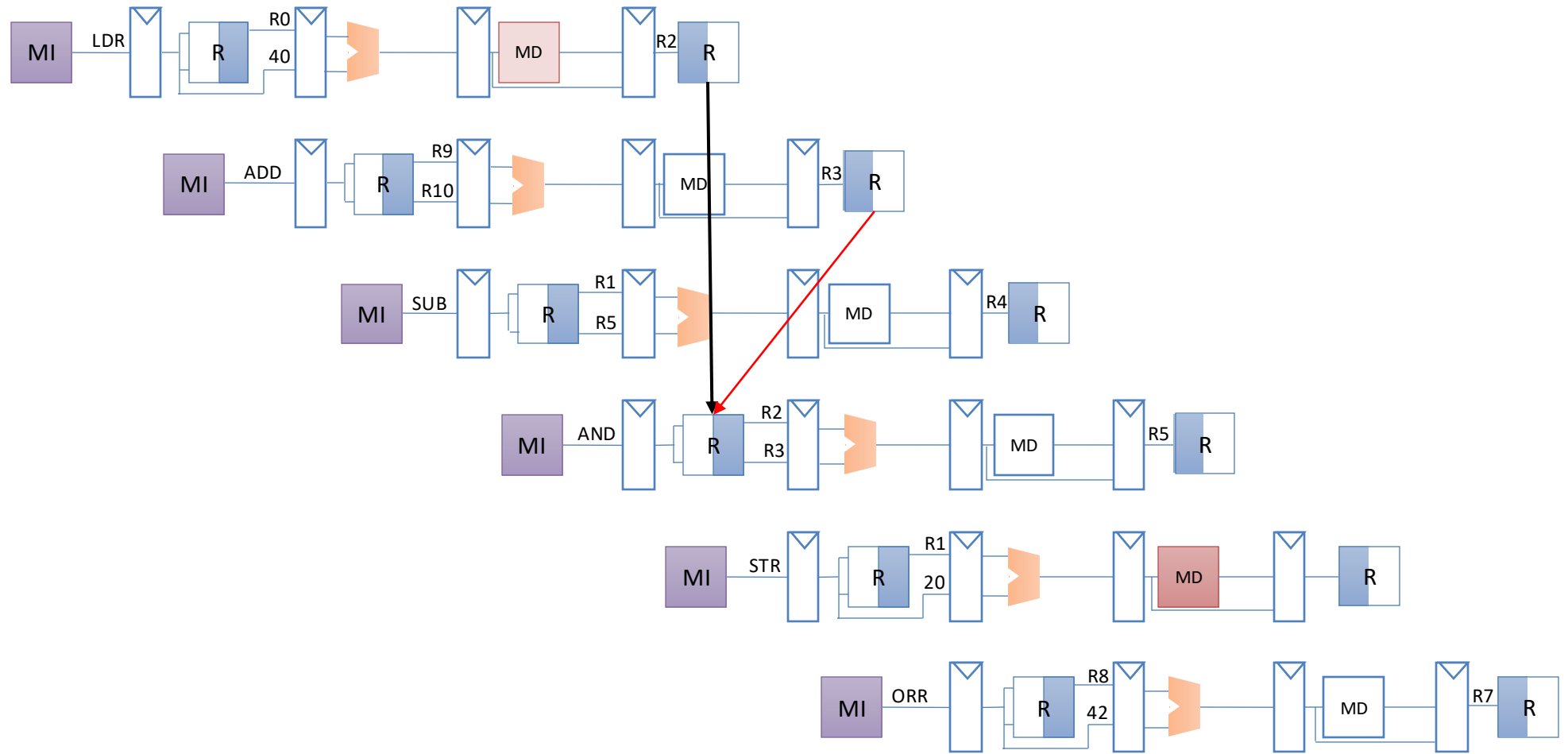
ADD R3, R9, R10

SUB R4, R1, R5

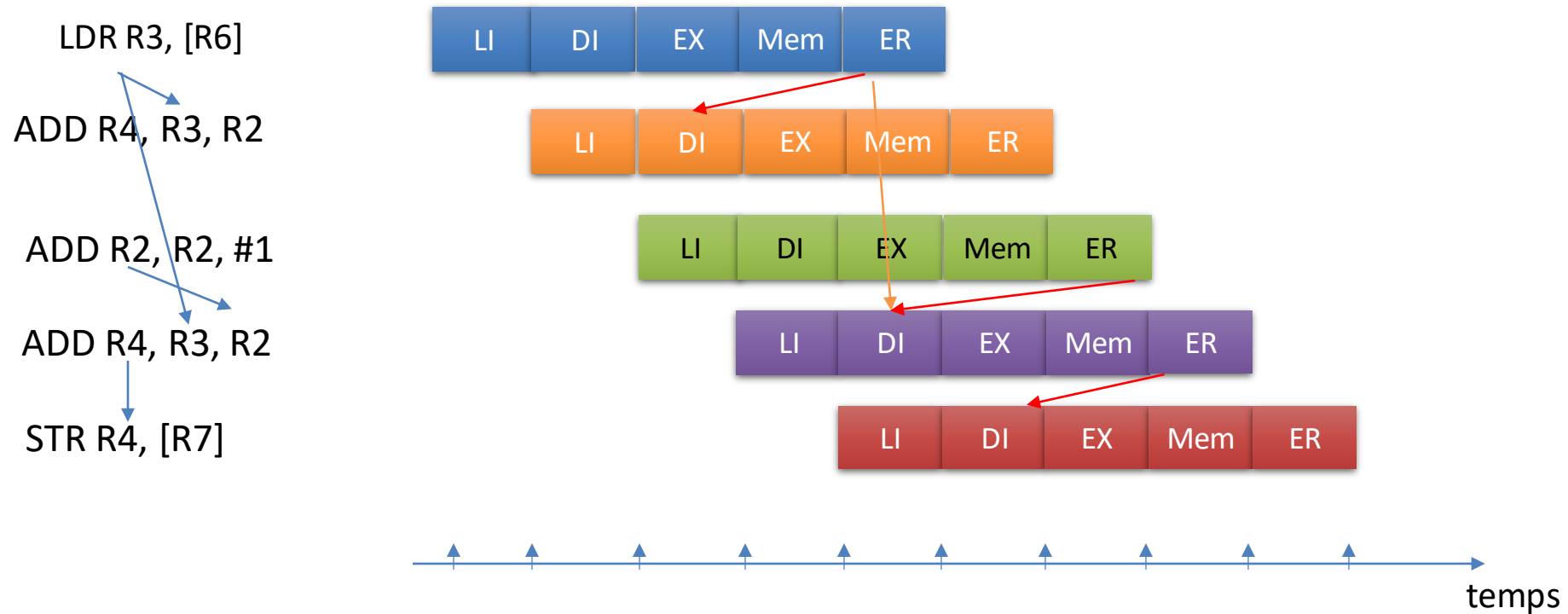
AND R5, R2, R3

STR R6, [R1, #20]

ORR R7, R8, #42



Dépendances et pipeline – Les aléas de données



Les données ne peuvent pas remonter le temps !

Exercice problème des dépendances

- Supposons que R1 et R3 contiennent 11 et que R2 contient 22.
- Supposons que l'on exécute le code suivant sur le processeur défini au transparent précédent. Quelle est la valeur finale des registres R3 et R4 ?

```
ADD R1, R2, #5  
ADD R3, R1, R2  
ADD R4, R1, #15  
ADD R3, R1, R3
```

Solution logicielle aux aléas de données

LDR R3, [R6]

ADD R4, R3, R2

ADD R2, R2, #1

ADD R4, R3, R2

STR R4, [R7]

LDR R3, [R6]

NOP

NOP

ADD R4, R3, R2

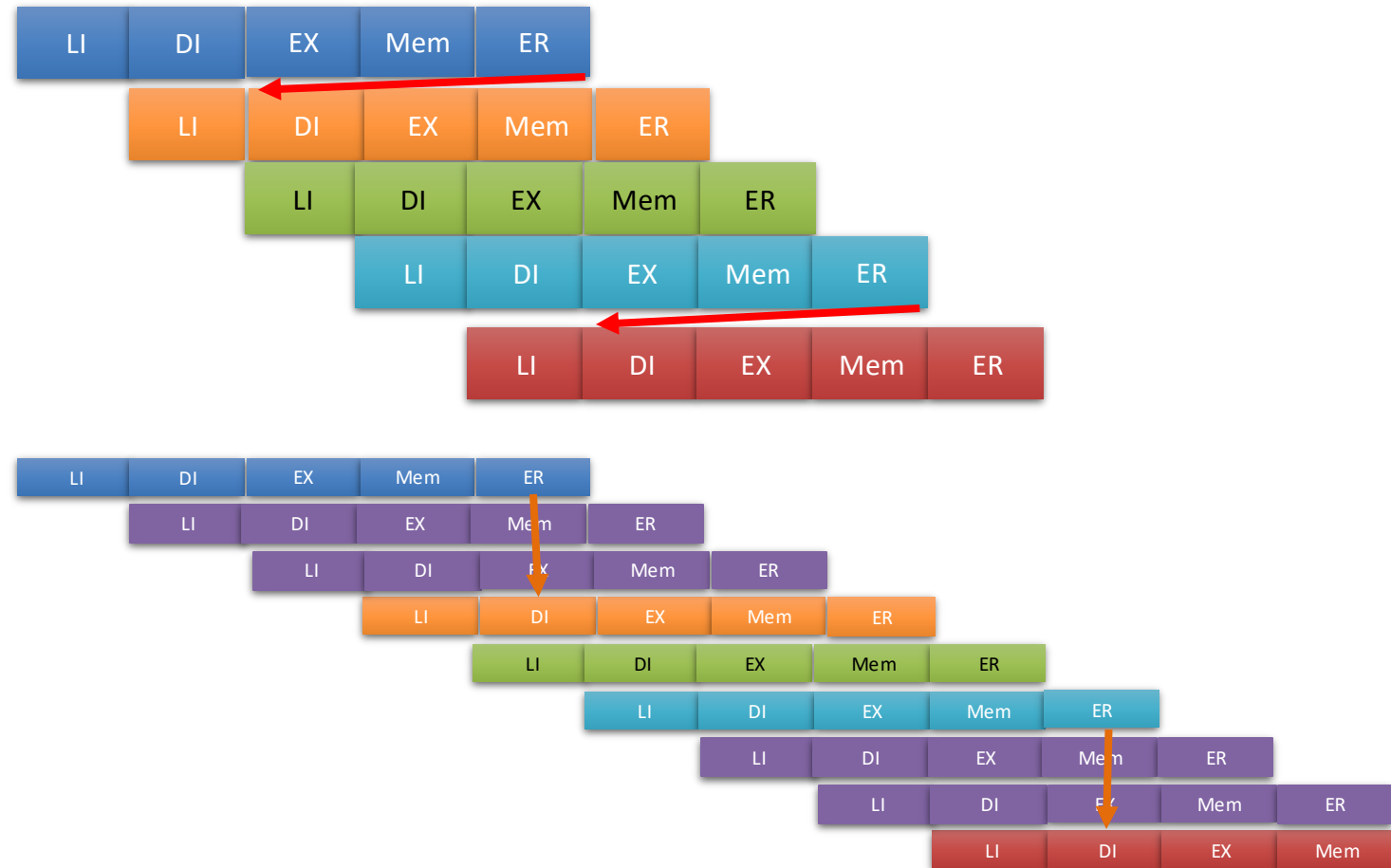
ADD R2, R2, #1

ADD R4, R3, R2

NOP

NOP

STR R4, [R7]



Exercice

Peut-on écrire le programme ci-dessous de sorte à n'avoir ni NOP ni aléa dans le pipeline dû à la dépendance sur R3 ?

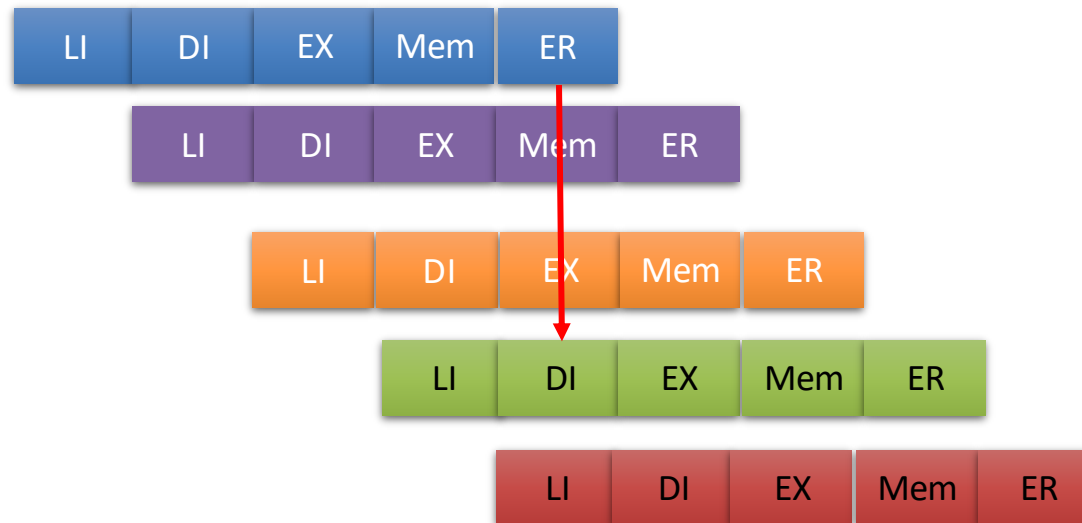
LDR R3, [R6]

NOP

ADD R5, R6, R7

ADD R4, R3, R2

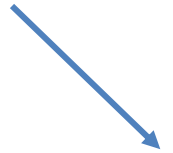
ADD R8, R7, R2



Solution matérielle pour éviter les ALEAS (1)

- Unité d'envoi (ou de court-circuit)

ADD R5, R6, R7



ADD R4, R3, R5



Besoin d'envoyer la sortie de l'ALU
vers l'entrée 2 de l'ALU

ADD R5, R6, R7

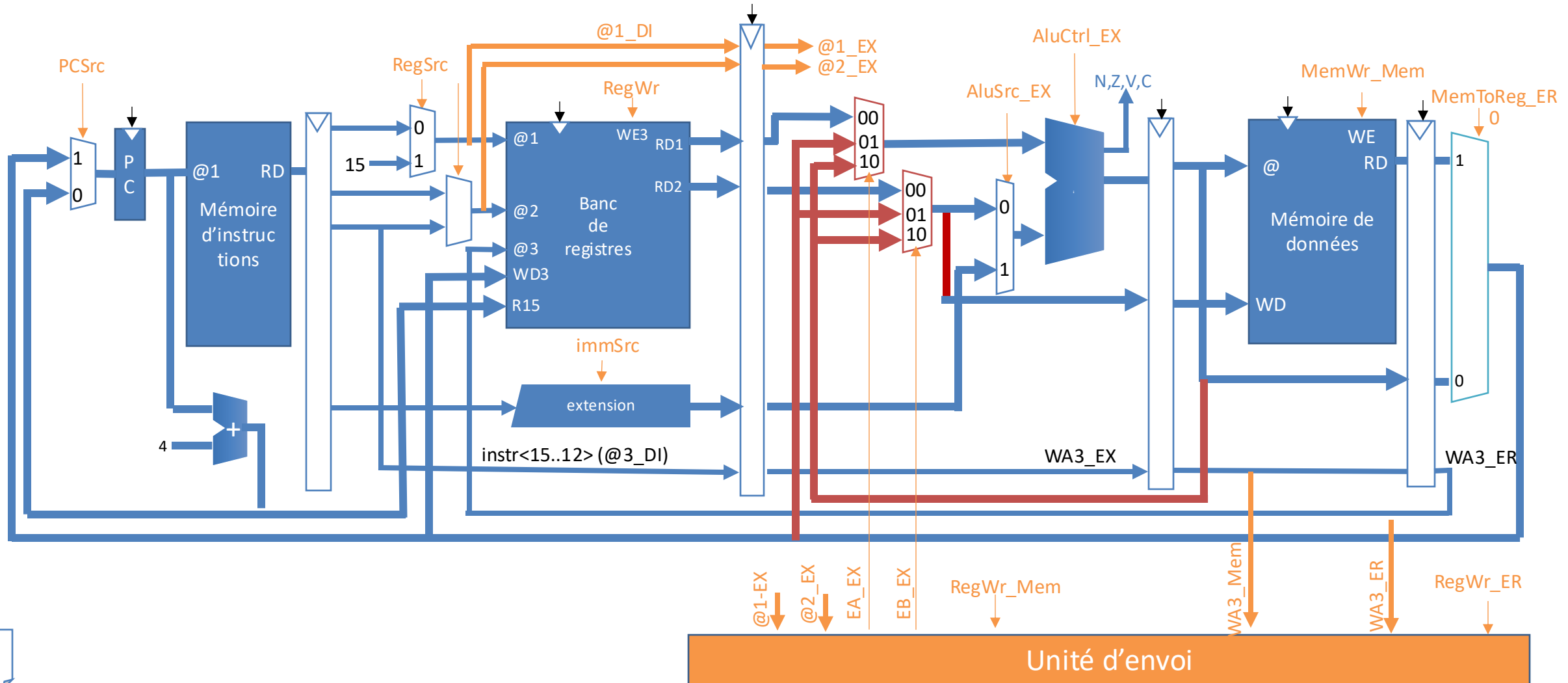


ADD R4, R5, R3



Besoin d'envoyer la sortie de l'ALU
vers l'entrée 1 de l'ALU

Solution matérielle pour éviter les ALEAS (2)



Solution matérielle pour éviter les ALEAS (3)

- Gel du pipeline

LDR R3, [R6]



ADD R5, R3, R7



ADD R4, R7, R2

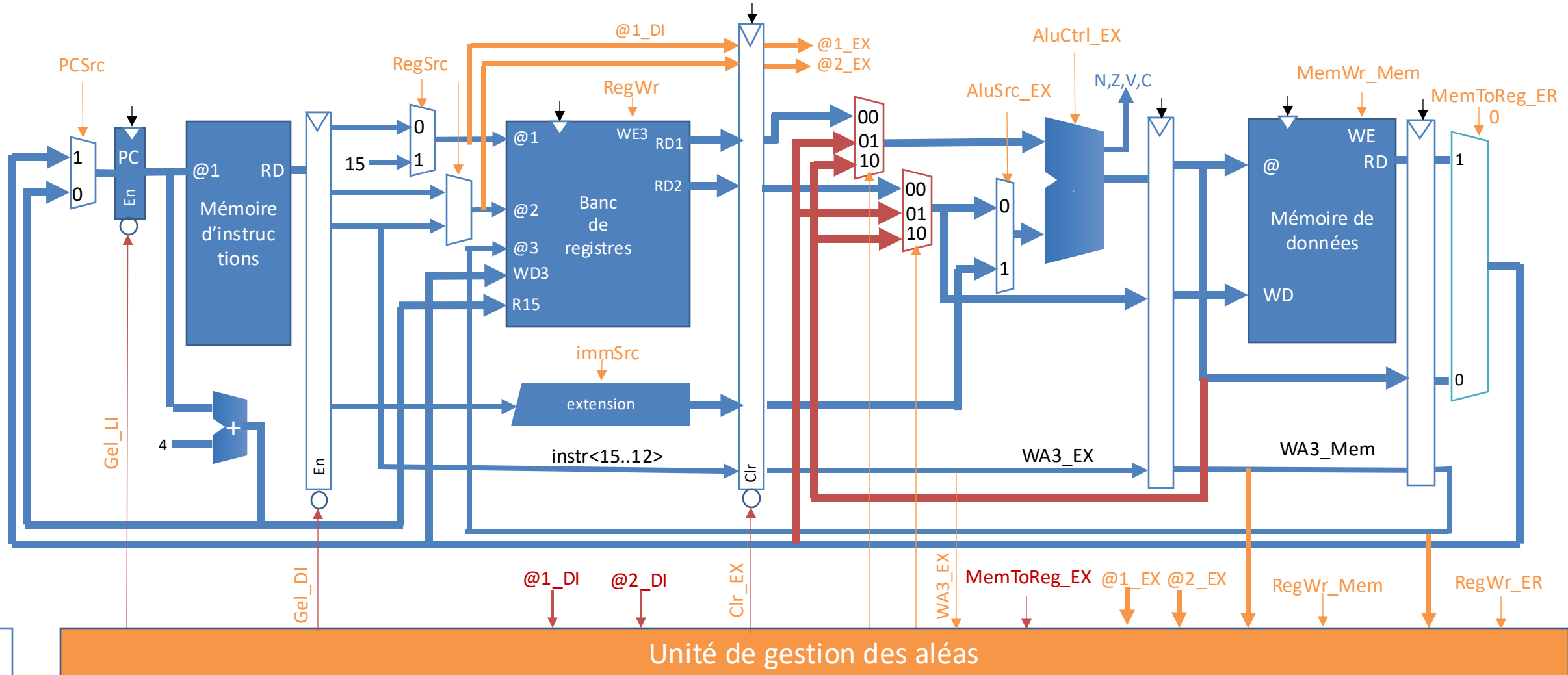


ADD R8, R7, R9



Solution matérielle pour éviter les ALEAS (4)

- Insérer 1 NOP de manière dynamique



Exercice – simulation pipeline 1 (1)

- Soit le pipeline à 5 étages vu précédemment et en supposant les latences suivantes (valables pour tous les exercices à venir sauf si indication contraire)
- Opérations entières : 0 cycle, MUL : 6 cycles, DIV : 24 cycles
- Soit la séquence de code suivante :

LDR R1, [R2] @ $R1 \leftarrow \text{mem}_{32}(R2)$

ADD R1, R1, #1 @ $R1 \leftarrow R1 + 1$

STR R1, [R2] @ $\text{mem}_{32}(R2) \leftarrow R1$

ADD R2, R2, #4 @ $R2 \leftarrow R2 + 4$

SUB R4, R3, R2 @ $R4 \leftarrow R3 - R2$

Exercice – simulation pipeline 1 (2)

Représenter l'exécution de cette séquence pour le processeur avec gestion des aléas

Brouillon		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	LDR R1, [R2]	LI	DI	EX	M	ER												
	ADD R1, R1, #1																	
	STR R1, [R2]																	
	ADD R2, R2, #4																	
	SUB R4, R3, R2																	

Solution		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	LDR R1, [R2]	LI	DI	EX	M	ER												
	ADDI R1, R1, #1																	
	STR R1, [R2]																	
	ADD R2, R2, #4																	
	SUB R4, R3, R2																	

Exercice – simulation pipeline 2 (1)

- Soit le pipeline à 5 étages vu précédemment et en supposant les latences suivantes (valables pour tous les exercices à venir sauf si indication contraire)
- Opérations entières : 0 cycle, MUL : 6 cycles, DIV : 24 cycles
- Soit la séquence de code suivante :

LDR R2, [R0] @ R2 $\leftarrow$ mem(R0)

LDR R3, [R0, #4] @ R3 $\leftarrow$ mem(R0+4)

MUL R5, R3, R2 @ R5 $\leftarrow$ R3 * R2

LDR R4, [R1] @ R4 $\leftarrow$ mem(R1)

ADD R5, R5, R4 @ R5 $\leftarrow$ R5 + R4

STR R5, [R1] @ mem(R1) $\leftarrow$ R5

SUB R0, R0, #2 @ R0 $\leftarrow$ R0 - 2

Exercice – simulation pipeline 2 (2)

Brouillon

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
LDR R2, [R0]	LI	DI	EX	M	ER													
LDR R3, [R0, #4]																		
MUL R5, R3, R2																		
LDR R4, [R1]																		
ADD R5, R5, R4																		
STR R5, [R1]																		
SUB R0, R0, #2																		

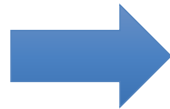
Solution

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
LDR R2, [R0]	LI	DI	EX	M	ER													
LDR R3, [R0, #4]																		
MUL R5, R3, R2																		
LDR R4, [R1]																		
ADD R5, R5, R4																		
STR R5, [R1]																		
SUB R0, R0, #2																		

Supprimer la suspension - Réordonnancement

A la charge du compilateur

LDR R3, [R6]
ADD R4, R3, R2
ADD R5, R5, #1



LDR R3, [R6]
ADD R5, R5, #1
ADD R4, R3, R2

Pas toujours possible à cause des dépendances

Dépendance
vraie

LDR R3, [R6]
ADD R4, R3, R2
ADD R2, R2, #1
ADD R4, R3, R2
STR R4, [R7]

Dépendance
de
sortie

Anti-
dépendance

Il ne faut violer aucune dépendance.
Le compilateur peut modifier les registres pour
supprimer des fausses dépendances. Encore
faut-il que des registres soient disponibles...

Renommage de registres

LDR R3, [R6]
ADD R4, R3, R2
ADD R8, R2, #1
ADD R9, R3, R8
STR R9, [R7]

Exercice – simulation pipeline 2 (3)

Réordonner le code pour minimiser les suspensions

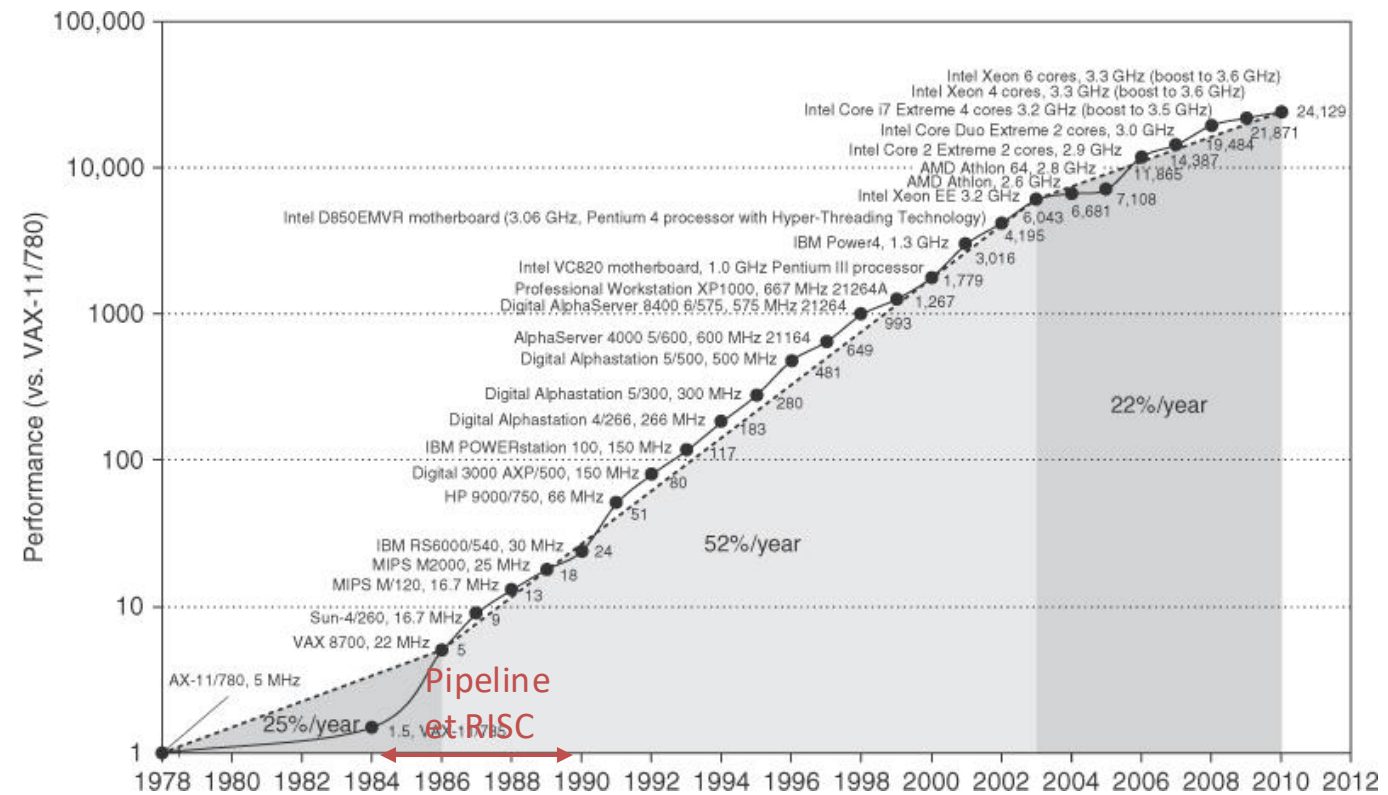
Brouillon		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	LDR R2, [R0]																	
Solution		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

En résumé

- Le pipeline permet d'augmenter la fréquence de fonctionnement et le débit d'instructions

La performance ne se mesure plus en CPI mais en IPC.

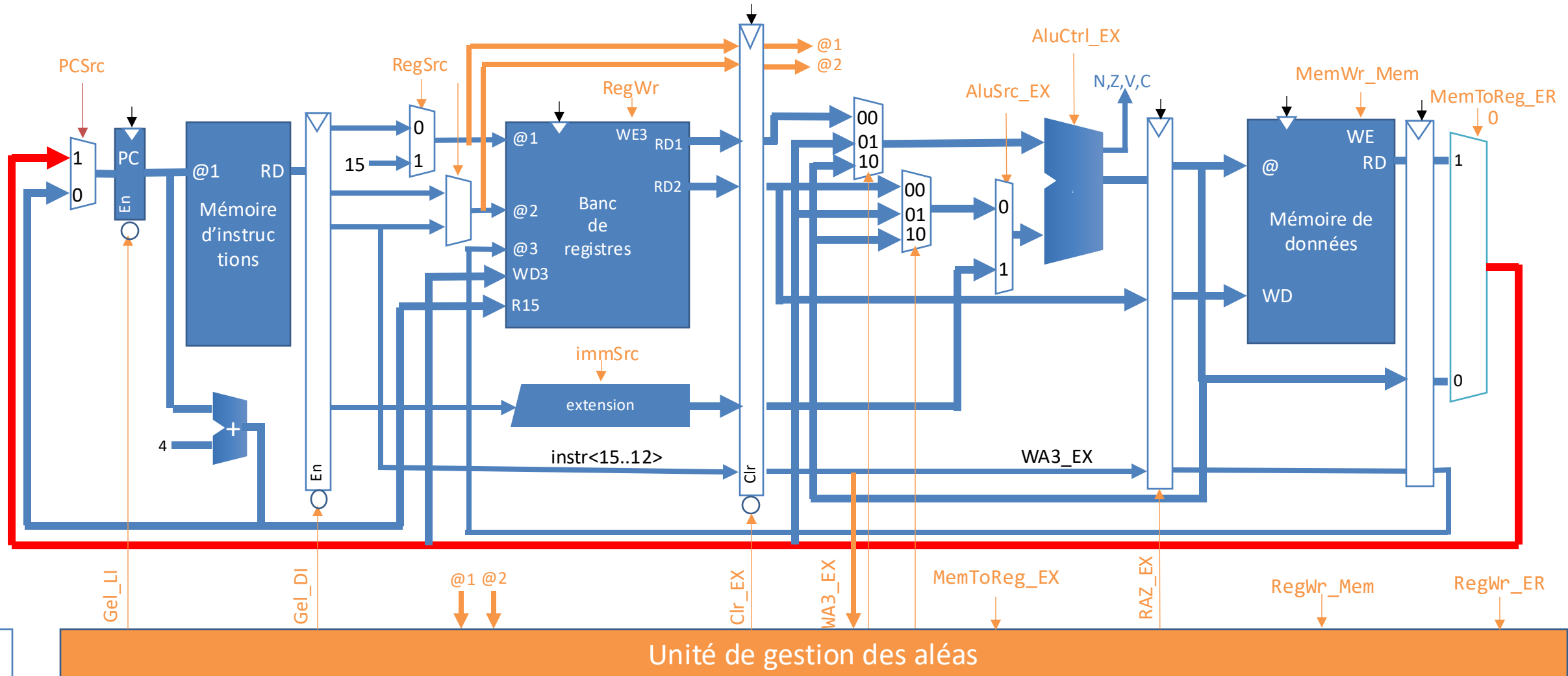
Pipeline et RISC datent de 1985 (début de la pente à 52%).



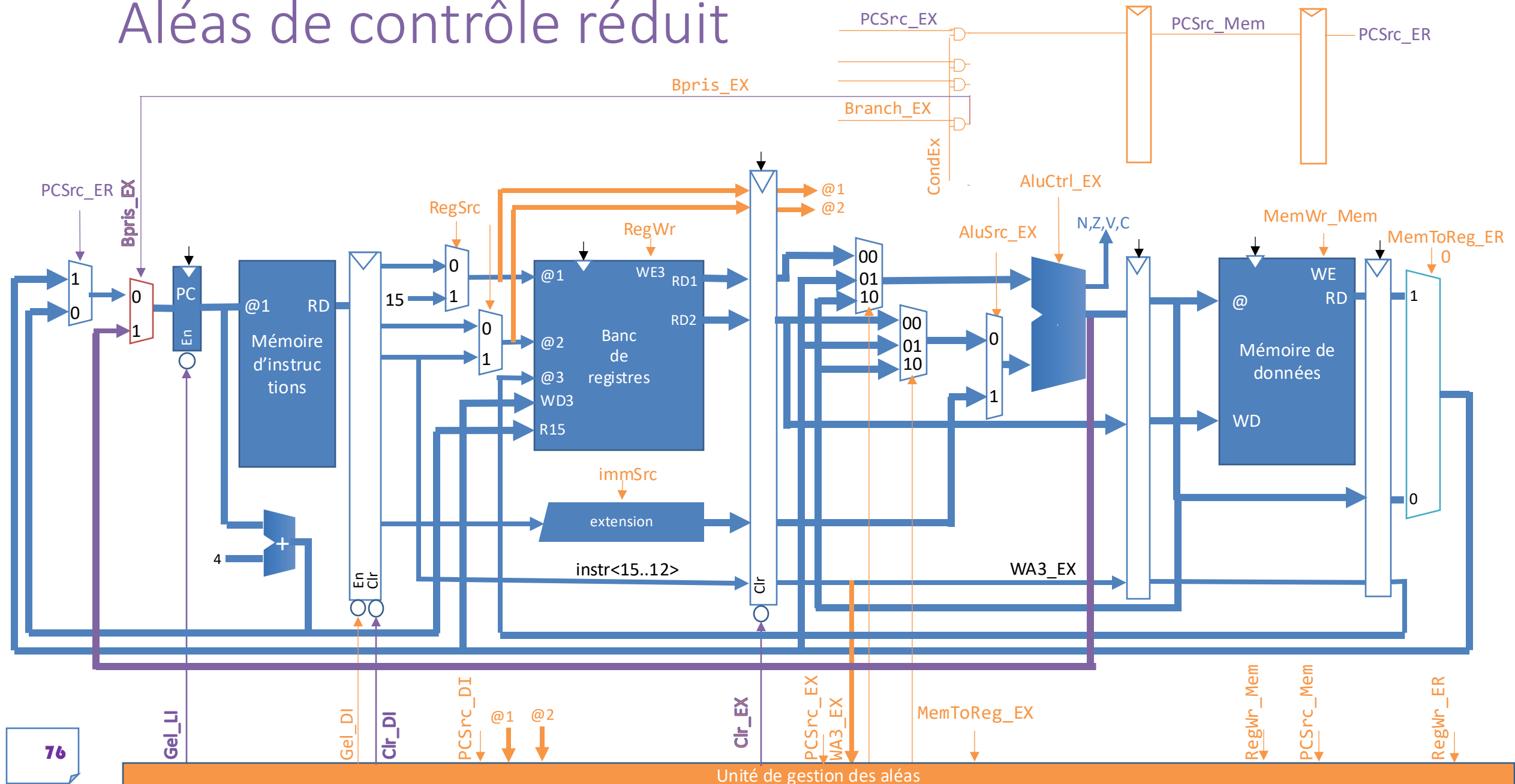
Rappels sur les ruptures du flot de contrôle

1. Par défaut, le processeur doit exécuter l'instruction qui suit
2. 2 types de branchement
 - Branchement non conditionnels
B, BL, écriture dans R15 (PC)
 - Branchements conditionnels
BEQ, BNE, BHI, BLO, ... Le branchement est pris si le registre CPSR indique que la condition est vérifiée.
CPSR est mis à jour par CMP, TST, xxxS

Aléa de contrôle



Aléas de contrôle réduit

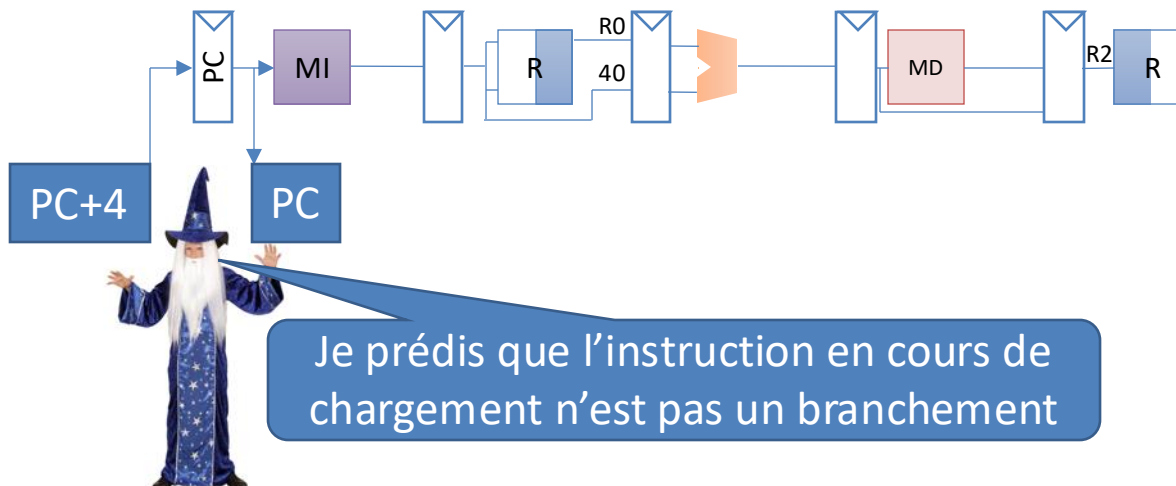


Quelques exemples de pipeline

- Cortex A5
 - 8 étages
- Cortex A8
 - LI sur 3 étages, DI sur 5 étages, EX sur 5 étages, ER sur 1 étage

Prédiction de branchement

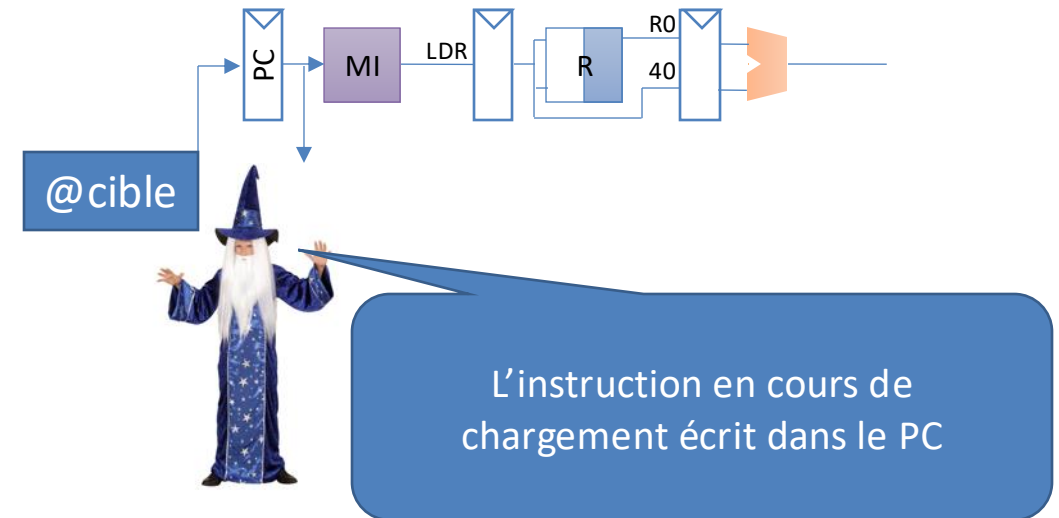
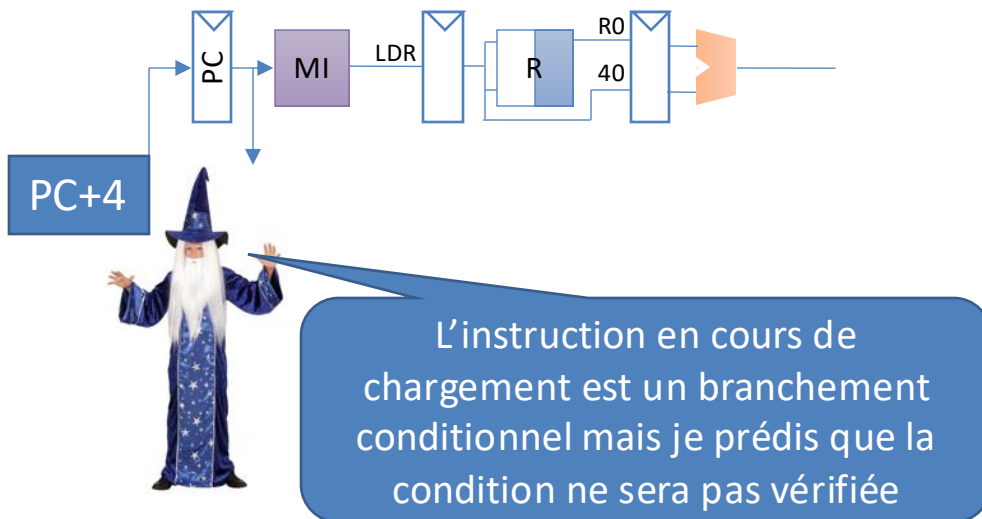
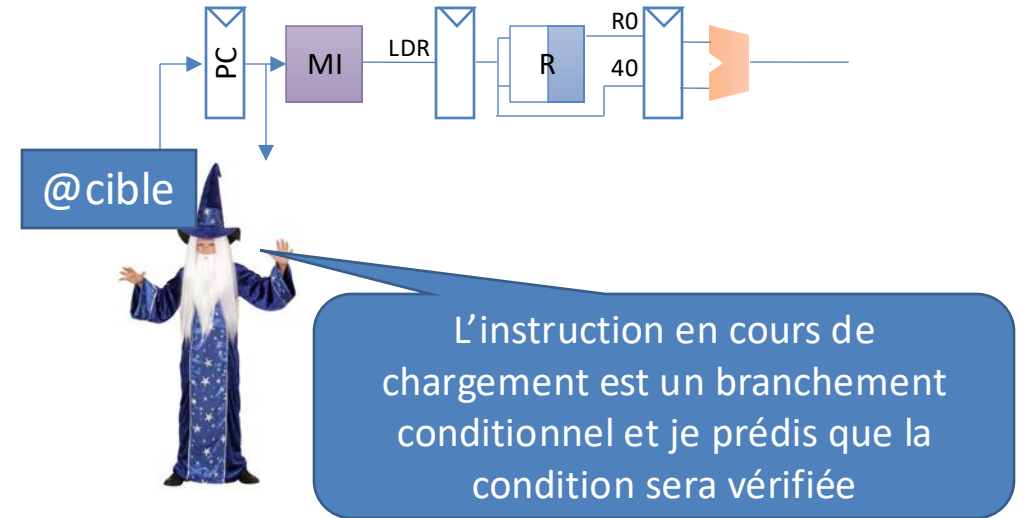
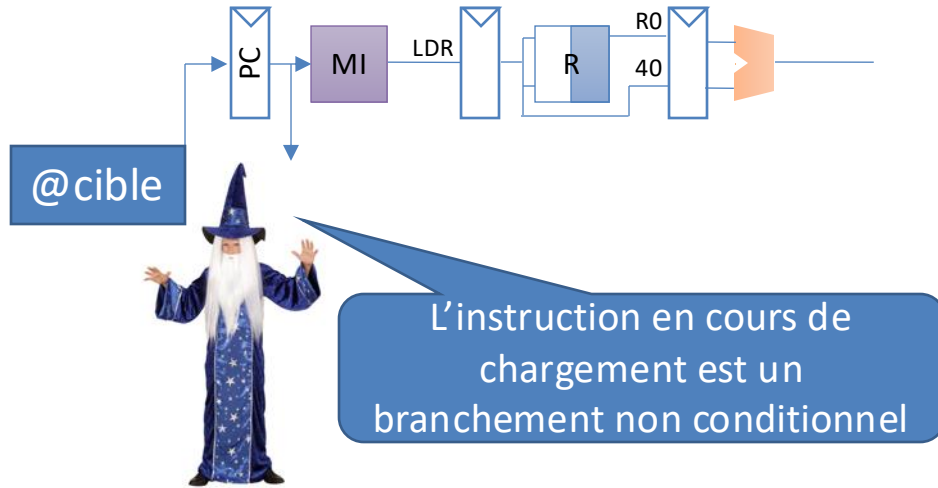
- Le pipeline précédent prédit systématiquement que les branchements ne sont pas pris et on a une pénalité de 3 cycles chaque fois qu'un branchement est pris.
- Pour éviter cette pénalité, il faudrait calculer les branchements dès le premier cycle
 - Impossible
- A défaut, prédire les branchements dès le premier cycle, vérifier à l'exécution que la prédiction était correcte. On ne paye la pénalité que si la prédiction est incorrecte.



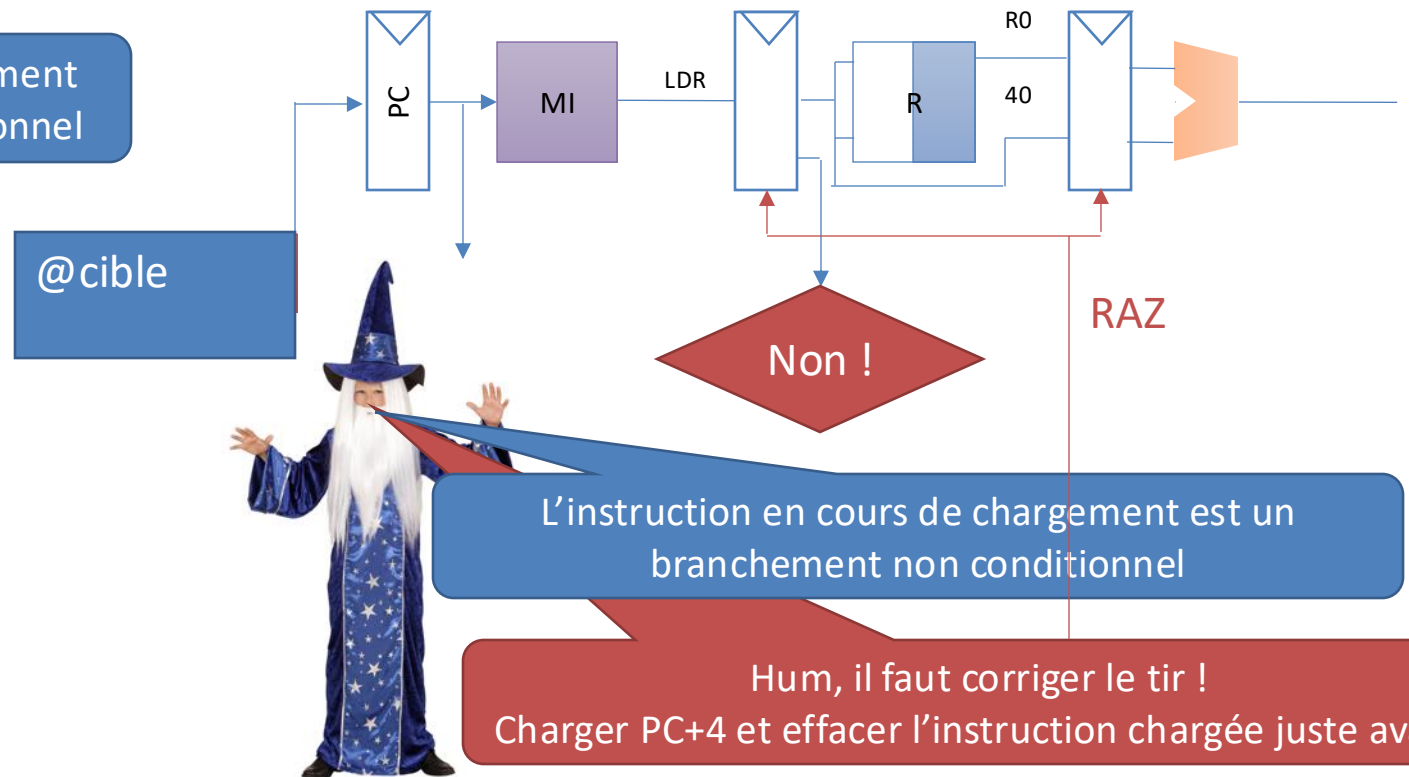
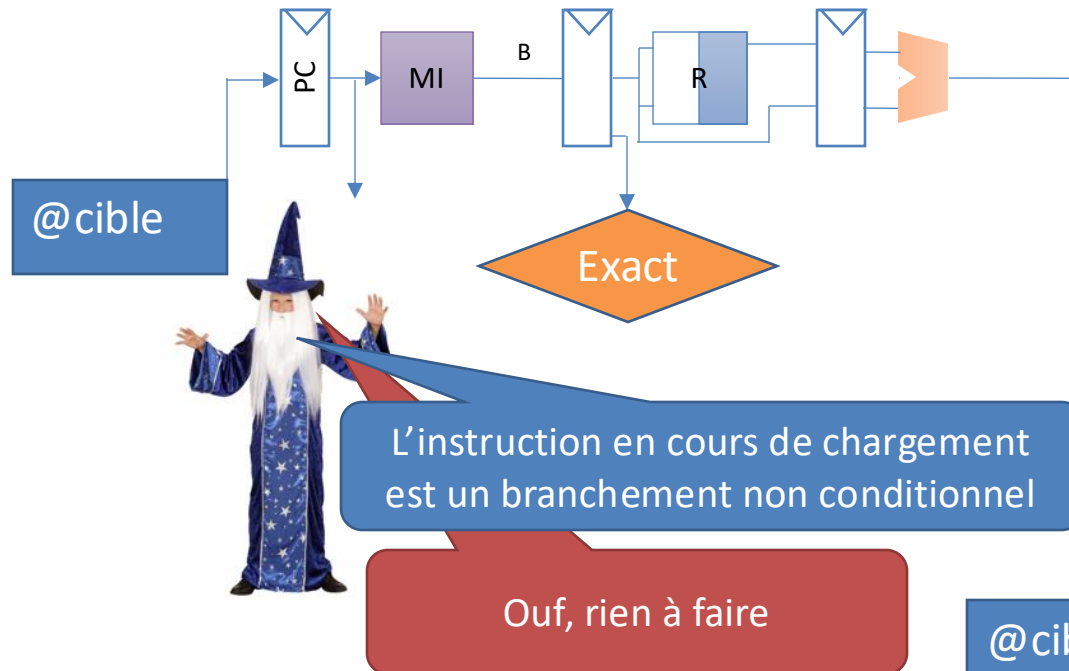
Prédiction de branchement: exercice 1

- Le processeur non pipeline a un débit d'une instruction toutes les 680ps.
- Le processeur pipeline (5 étages) a un temps de cycle de 200ps.
 - Chaque branchement pris implique une pénalité de 3 cycles.
- Un programme de 100 instructions comporte 20 branchements pris.
- Quel est le débit du processeur pipeline ?

Prédiction de branchement



Prédiction de branchement



Prédiction statique de la direction des branchements

```
for (i=0; i<256; i++) {  
    if (x==y) ...  
    else ...  
}
```

for:	mov	r2, #0	
	cmp	r2, #256	
	beq	finfor	← Branchement en avant : généralement non pris
	cmp	r3, r4	
	bne	sinon	← Branchement en avant : ? généralement non pris
	...		
	b	suite	← Branchement en avant : toujours pris
sinon:	...		
suite:	add	r2, r2, #1	← Branchement en arrière : toujours pris
	b	for	
finfor:	...		

Problème de la prédiction statique

- Il faut savoir que l'instruction que l'on est en train de charger est un branchement
 - On ne le sait qu'à l'étape de décodage
 - Perte d'un cycle chaque fois qu'un branchement est pris.
- NB : la prédiction de la direction ne concerne que les branchements conditionnels
 - Une fois décodés, on sait que les branchements non conditionnels sont pris

Prédiction statique de la direction des branchements

```
for:    mov    r2, #0
        cmp    r2, #256
        beq    finfor
        cmp    r3, r4
        bne    sinon
        ...           // 4 cycles
        b      suite
sinon:
        ...           // 5 cycles
suite:  add    r2, r2, #1
        b      for
finfor:  ...
```

1. Quel est le taux minimal et maximal de mauvaises prédictions de ce programme sans prédiction de branchement (prédiction toujours non pris)
2. Quel est le taux minimal et maximal de mauvaises prédictions de ce programme avec un prédicteur statique BTFNT ?

Prédiction dynamique

- Prédiction qui prend en compte le comportement passé des branchements
- Prédiction dès le premier cycle
 - Prédiction de la présence d'un branchement
 - Prédiction du type du branchement
 - Prédiction de l'adresse cible
- Vérification lors de l'exécution du branchement et annulation si mauvaise prédiction

Prédiction de la direction dynamique

Compteurs 1 bit

- Prédiction du comportement
Table d'historique des branchements:
 - petite mémoire indexée par l'adresse de l'instruction
- Moins d'entrée que la mémoire principale, accès direct
 - interférences entre branchements
- Entrée = un bit qui indique si le branchement était pris ou non la dernière fois qu'on l'a exécuté
 - Si prédiction fausse, inversion du bit
 - Pb : branchement de fin de boucle exécutée plusieurs fois
 - 2 erreurs chaque fois qu'un branchement change de comportement

Boucles imbriquées

BCL1: ...

...

BCL2: ... BCL2

... BCL1

BLS

BLS

Comportement B2	P	P	P	P	NP	P	P	P
Prédiction B2	?	P	P	P	P	NP	P	P

Exercice prédiction statique et dynamique 1 bit

- Soit les 3 branchements ayant les comportements suivants

- B1

P P P N N N N N N P P P P N N N N N N N P P P N P

- B2

P P P P N P P P P N P P P N N N N P N N N P N N N N

- B3

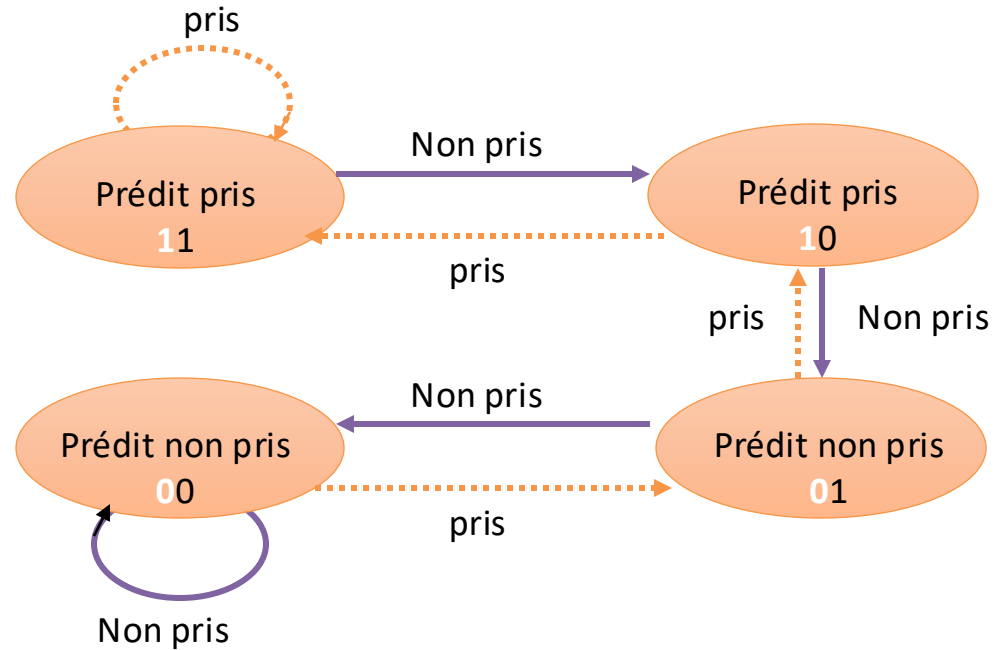
N N N N N P N N N N N P N N N N N P N N N N N P N N

- Donner le nombre d'erreurs de prédiction pour
 - Prédiction statique pris
 - Prédiction statique non pris
 - Prédiction dynamique 1 bit (état initial à 0)

Prédiction de la direction dynamique

compteurs 2 bits

- La prédiction ne change que si un branchement a 2 fois la même prédiction
- Initialisation de la table à 10 ou 01 suivant la direction majoritaire (dépend du compilateur).
- Taille de la table : 4096 entrées apporte une précision quasiment identique à une table infinie (de 0 à 15% de mauvaises prédictions)
- La différence entre un compteur 3 bits et un 2 bits est très faible.



Comportement B2	P	P	P	P	NP	P	P	P
Compteur	10	11	11	11	11	10	11	11
Prédiction	P	P	P	P	P	P	P	P

Exercice1 prédiction dynamique 2 bits

- Soit les 3 branchements ayant les comportements suivants

- B1

P P P N N N N N N P P P P N N N N N N N P P P N P

- B2

P P P P N P P P P N P P P N N N N P N N N P N N N N

- B3

N N N N N P N N N N N P N N N N N P N N N N N P N N

- Donner le nombre d'erreurs de prédiction pour
 - Prédiction dynamique 2 bits, compteurs initialisés à 10

Exercice 2 : prédiction dynamique 2 bits

- On suppose que l'état initial des compteurs 2 bits est 10 et qu'il n'y a pas de conflits dans la table des compteurs.
- Soit le programme suivant

```
for (i=0; i<5; i++)  
    ...  
for:    CMP    MOV    R1, #0  
        R1, #5  
        BEQ    finfor  
        ...  
        B      for  
finfor:
```

- Quel est le taux de bonnes prédictions du BEQ finfor ?

Compteur	10					
Prédiction	P					
Comportement						

Exercice 3 : prédiction dynamique 2 bits

- Soit le programme suivant compilé de la même façon que dans l'exemple précédent

```
for (i=0; i<100; i++) // b1
    for (j=0; j<5; j++) // b2
        tab[i] = 0;
...
```

- Quels sont les taux de bonnes prédictions des branchements conditionnels b1 et b2 ? Quel est le taux global de bonnes prédictions des branchements conditionnels ?

Compteur b1	10					
Prédiction b1	P					
Comportement b1						

Compteur b2	10					
Prédiction b2	P					
Comportement b2						

Exercice 4 : prédiction dynamique 2 bits

- Soit le programme suivant compilé de la même façon que dans l'exemple précédent. On suppose pour b3 et b4 que les branchements sont pris lorsque la condition est fausse.

```
for (i=0; i<100; i++)           // b1
    for (j=0; j<10; j++)         // b2
        if (j%2 == 0)            // b3
            if (i%2 == 0)         // b4
                a[j] = 0;
```

...

- Quels sont les taux de bonnes prédictions des branchements conditionnels b1, b2, b3 et b4 ? Quel est le taux global de bonnes prédictions des branchements conditionnels ?

Compteur b3	10						...
Prédiction b3	P						...
Comportement réel B3							...

Compteur b4	10													...
Prédiction b4	P													...
Comportement réel B4														...

Prédiction de la direction dynamique

Historique global - Introduction

- Un branchement peut avoir un comportement qui dépend des branchements précédents

Exemple de programme

```
if (aa=2)
    aa = 0;
if (bb=2)
    bb = 0;
if (aa != bb)
    { ... }
```

Traduction en assembleur
aa dans R1, bb dans R2

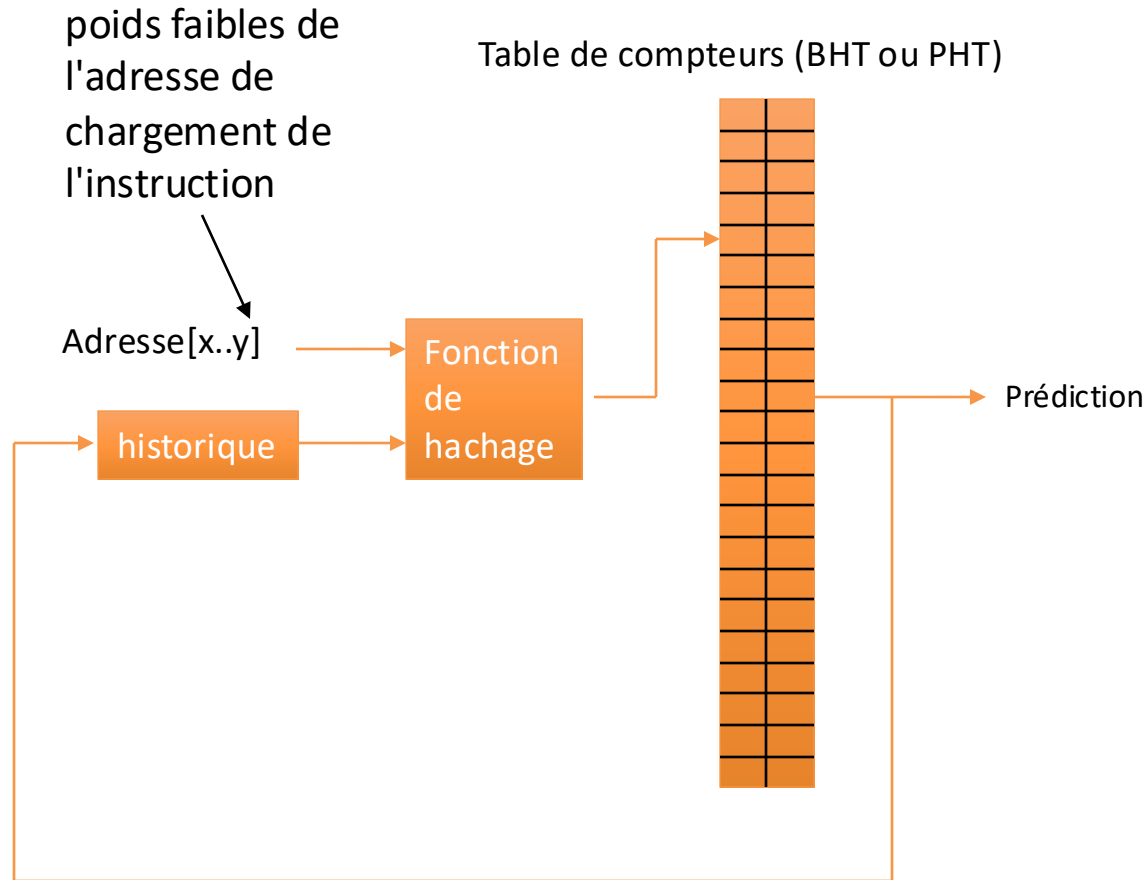
```
          SUBS    R3, R1, #2
b1:        BNE     L1
          MOV     R1, #0
L1:        SUBS   R3, R2, #2
b2:        BNE     L2
          MOV    R2, #0
L2:        SUBS   R3, R1, R2
b3:        BNE     L3
```

Si b1 et b2 ne sont pas pris,
b3 le sera car aa = bb = 0.
b3 est corrélé à b1 et b2.

Prédicteurs à corrélation ou
prédicteurs à deux niveaux.

Prédiction de la direction dynamique

Historique global (1)



Fonction de hachage
Concaténation
XOR

plus souple

plus de bits pris en compte

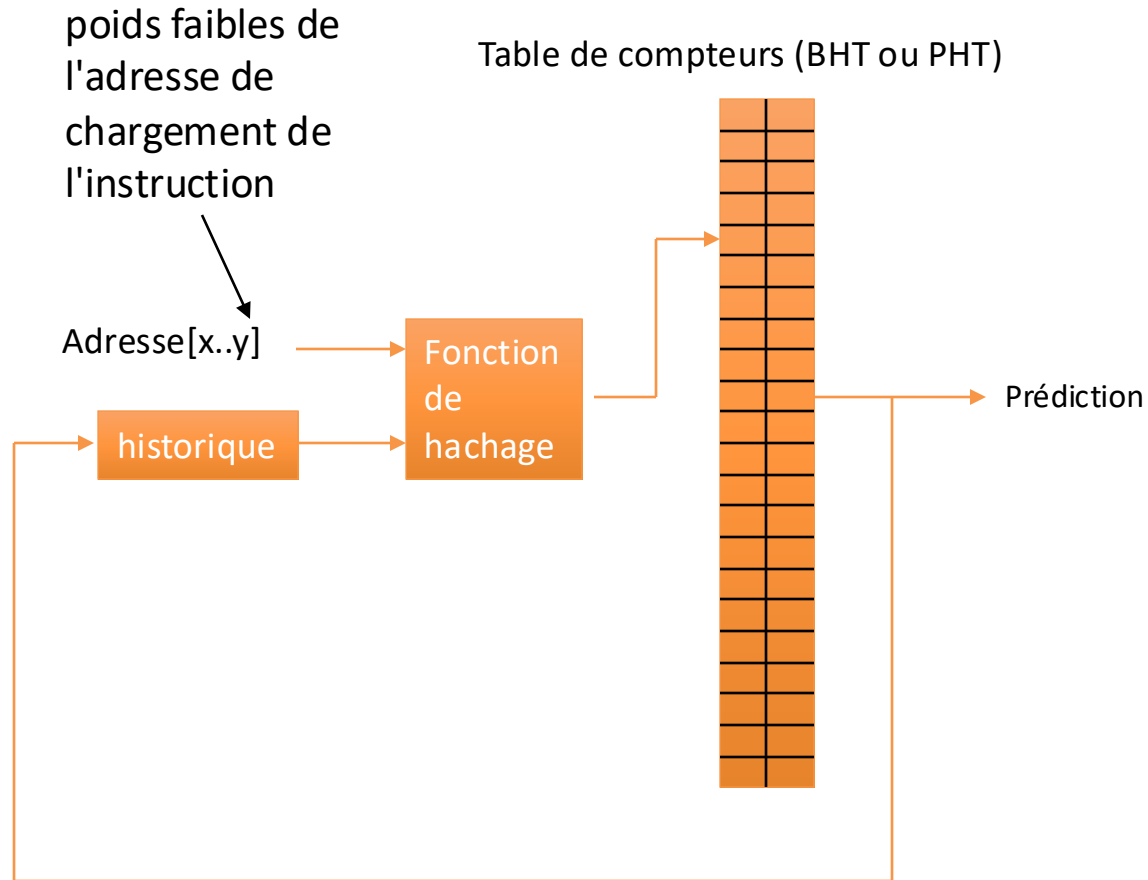
BHT = Branch History Table

PHT = Pattern History Table

Prédiction de la direction dynamique

Historique global (2)

Prédicteur caractérisé par (nombre de bits d'historique, nombre de bits d'un compteur)



Exemple de concaténation (4,2)

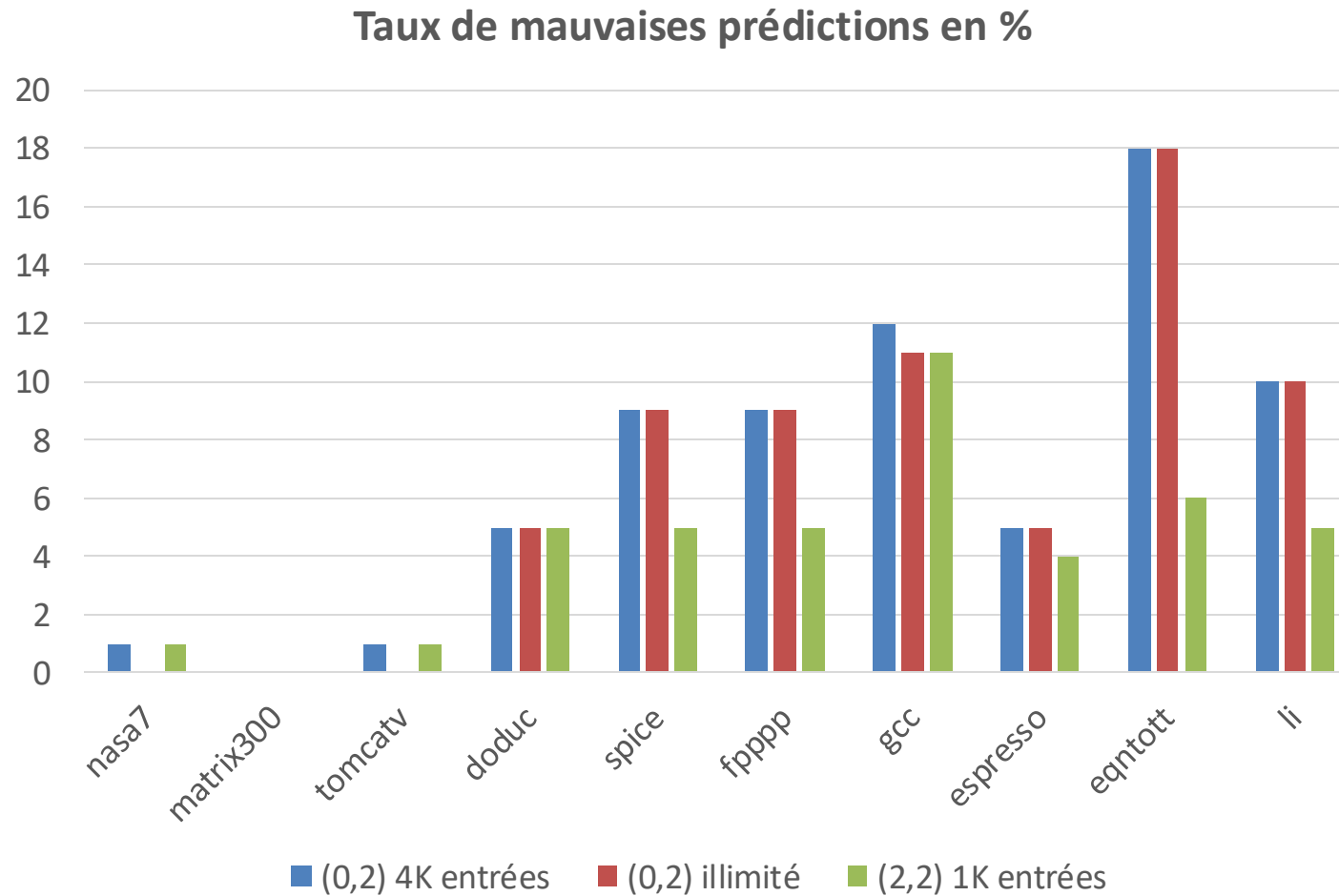
8 bits d'adresse
4 bits d'historique
donne accès à 2^{12} prédicteurs 2 bits
2 branchements se partagent un prédicteur s'ils sont éloignés de 2^8 instructions et qu'il y a le même historique global.

Exemple de XOR (12, 2)

12 bits d'adresse
12 bits d'historique
donne accès à 2^{12} prédicteurs 2 bits
Certains branchements distants de moins de 2^{12} instructions peuvent partager un prédicteur...

Prédiction de la direction dynamique

Historique global (3)



Exercice prédicteur à historique global (1)

- Soit le code suivant

```
int tab[9] = {8, 9, 10, 11, 12, 20, 29, 30, 31};
int a=0, b=0;

for (i=0 ; i<9 ; i++) {
    if (tab[i]%2 == 0)                branchement b1
        a++;                        [b1 pris]
    if (tab[i]%10 == 0)              branchement b2
        b++;                        [b2 pris]
}
```

- **Question 1.**

Les branchements sont prédits à partir de compteurs 1-bit, initialisés à 0 (*non pris*). Il n'y a pas de conflit dans la table de compteurs entre *b1* et *b2*. Chaque branchement met à jour son compteur avant que le branchement suivant ne soit exécuté.

Exercice prédicteur à historique global (2)

- **Question 1.**

Les branchements sont prédits à partir de compteurs 1-bit, initialisés à 0 (*non pris*). Il n'y a pas de conflit dans la table de compteurs entre *b1* et *b2*. Chaque branchement met à jour son compteur avant que le branchement suivant ne soit exécuté.

Indiquer, dans le tableau suivant, comment les deux branchements sont prédits et finalement exécutés puis indiquer le taux de bonnes prédictions de chacun des deux branchements puis le taux global.

Brouillon

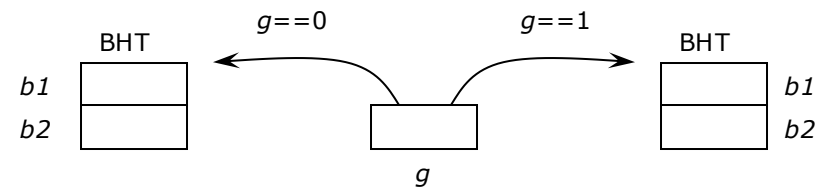
tab[i]	8	9	10	11	12	20	29	30	31
Préd. b1	NP								
Réel b1	P								
Préd b2	NP								
Réel b2	NP								

Solution

tab[i]	8	9	10	11	12	20	29	30	31
Préd. b1									
Réel b1									
Préd b2									
Réel b2									

Exercice prédicteur à historique global (3)

- On suppose maintenant qu'on a un prédicteur à deux niveaux : un historique global g (1 bit) mémorise la direction réelle du dernier branchement exécuté. Ce bit est utilisé pour choisir parmi deux tables de compteurs 1-bit (initialisés à 0 : *non pris*), comme montré ci-dessous :



- Selon la valeur de g (initialisé à 0), une des deux tables est utilisée pour prédire le branchement suivant (c'est cette même table qui est mise à jour lorsque le branchement est résolu). Il n'y a pas de conflit dans les tables de compteurs entre $b1$ et $b2$. Chaque branchement met à jour son compteur avant que le branchement suivant ne soit exécuté.
- Indiquer, dans le tableau diapo suivante, comment les deux branchements sont prédits et finalement exécutés (pour chaque branchement, ne remplir qu'une seule des deux tables, en fonction de la valeur de g)
- Quel est le taux de bonnes prédictions pour chacun des deux branchements ?
- Quel est le taux global de bonnes prédictions ?
- Quel est le taux de bonnes prédictions de $b2$ lorsque $g==0$? Expliquer pourquoi.

Exercice prédicteur à historique global (4)

Brouillon

		tab[i]	8	9	10	11	12	20	29	30	31
g==0	Préd b1	NP		P	NP						
	Réel b1	P		NP	P						
	Préd b2			NP							
	Réel b2			NP							
g==1	Préd b1										
	Réel b1										
	Préd b2	NP									
	Réel b2	NP									

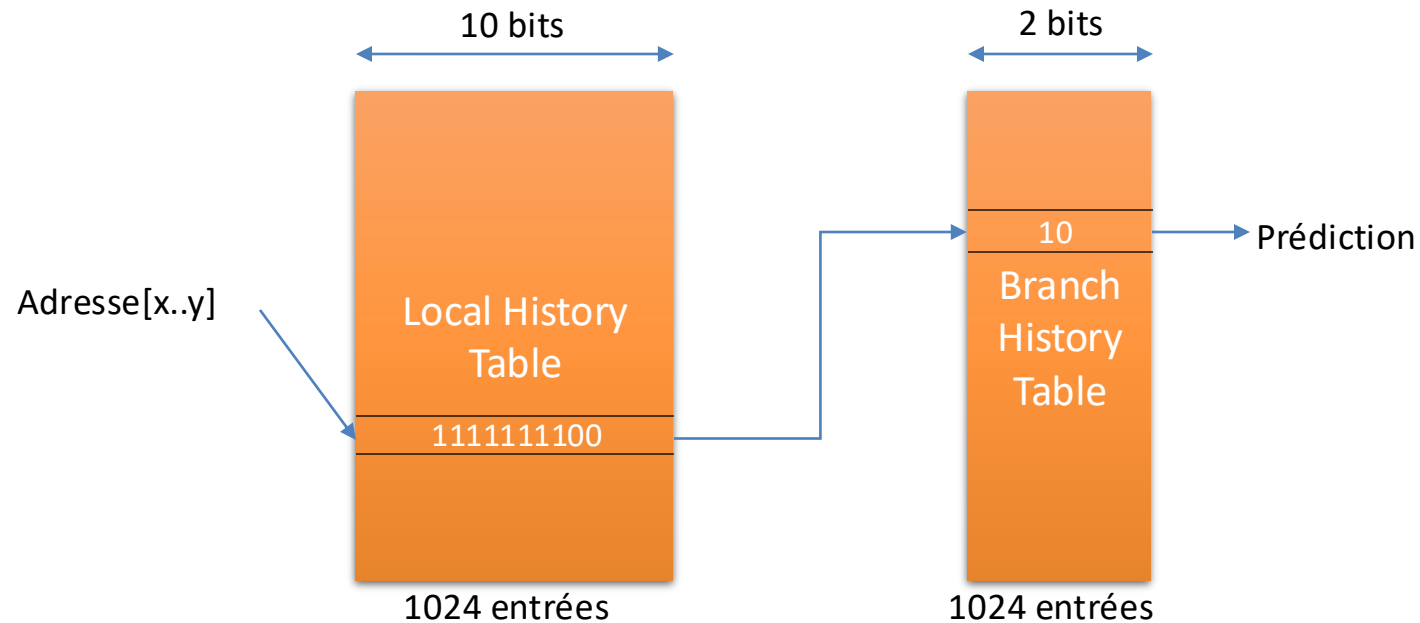
Solution

		tab[i]	8	9	10	11	12	20	29	30	31
g==0	Préd b1										
	Réel b1										
	Préd b2										
	Réel b2										
g==1	Préd b1										
	Réel b1										
	Préd b2										
	Réel b2										

Prédiction de la direction dynamique

Historique local

- Au lieu d'utiliser un historique représentant le comportement des derniers branchements, on utilise un historique représentant le comportement du branchement que l'on souhaite prédire. Il faut donc une table d'historiques qui va déterminer à quel compteur 2 bit on accède.



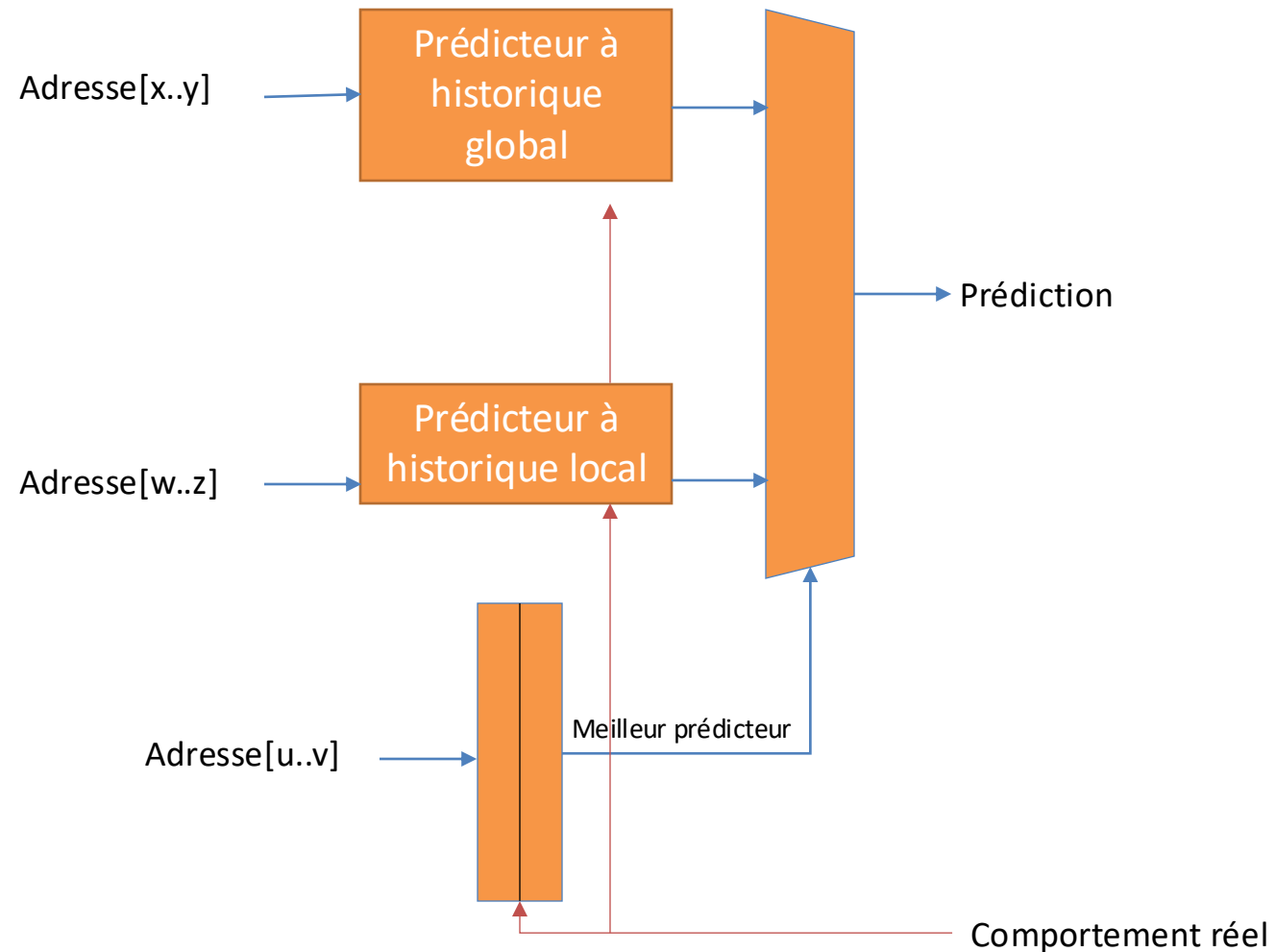
Prédiction de la direction dynamique

Table à accès direct

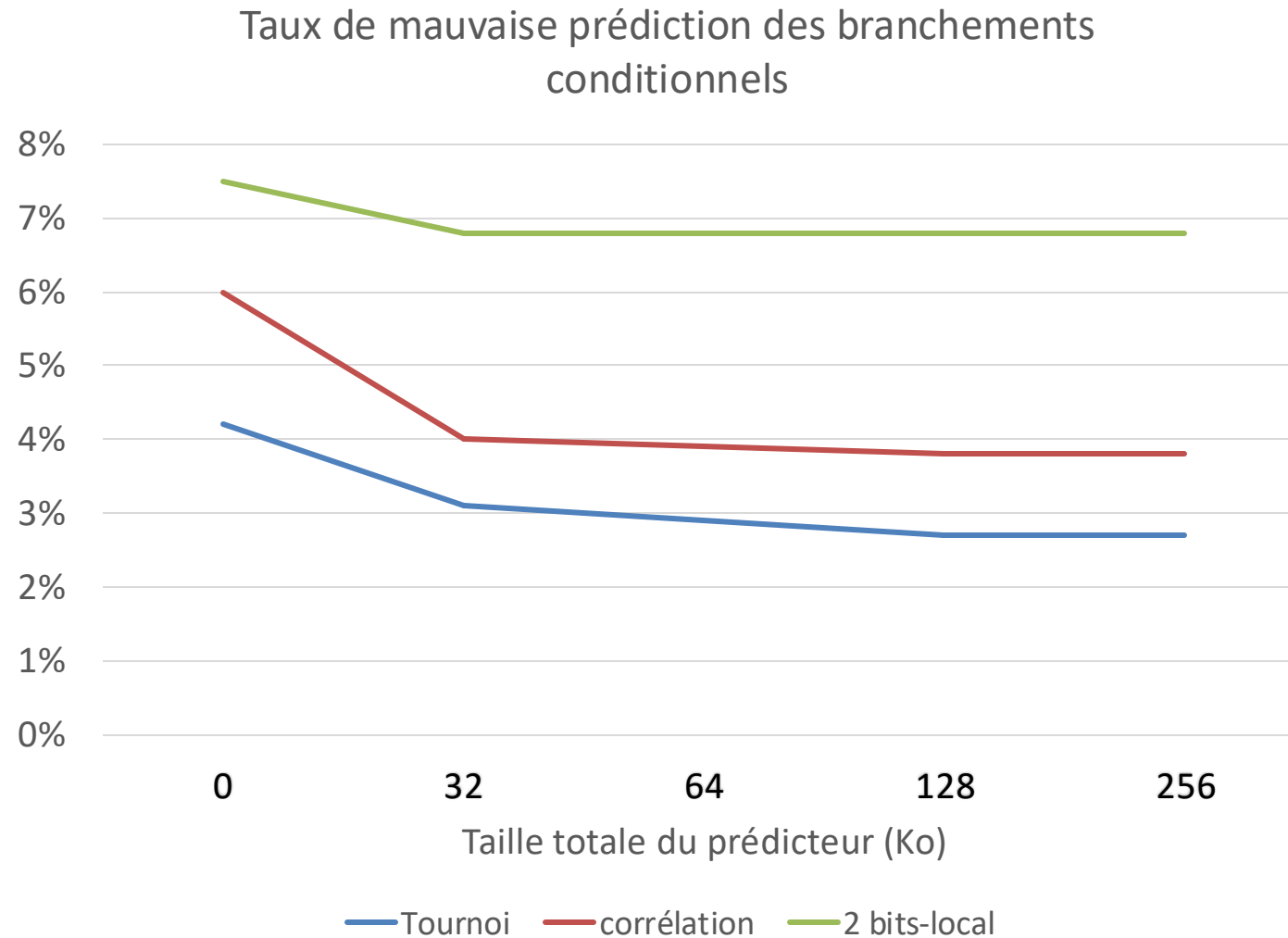
- La BHT (ou PHT) est à accès direct.
 - Le résultat étant 0 ou 1, ce n'est pas très gênant de partager des entrées entre plusieurs branchements. On a moins de 50% de chances de se tromper.
 - Il faut tout de même éviter de partager trop les entrées en adoptant une taille de BHT suffisamment grande (couramment 1024 voire 4096 entrées). On peut également essayer de trouver une fonction de hachage qui mélange mieux que le XOR mais elle doit rester simple pour que l'accès à la BHT se fasse en un cycle.
 - N'utiliser (mise à jour) la BHT que pour les branchements conditionnels. Les autres étant tout le temps pris, leur prédiction peut se faire différemment (voir plus loin la prédiction de l'adresse).
 - Des recherches continuent pour trouver de meilleures façons de prédire la direction des branchements conditionnels mais la qualité de prédiction étant déjà très bonne, il est difficile de l'améliorer sans ajouter un surcoût matériel important.

Prédiction de la direction dynamique

Prédicteur à tournoi

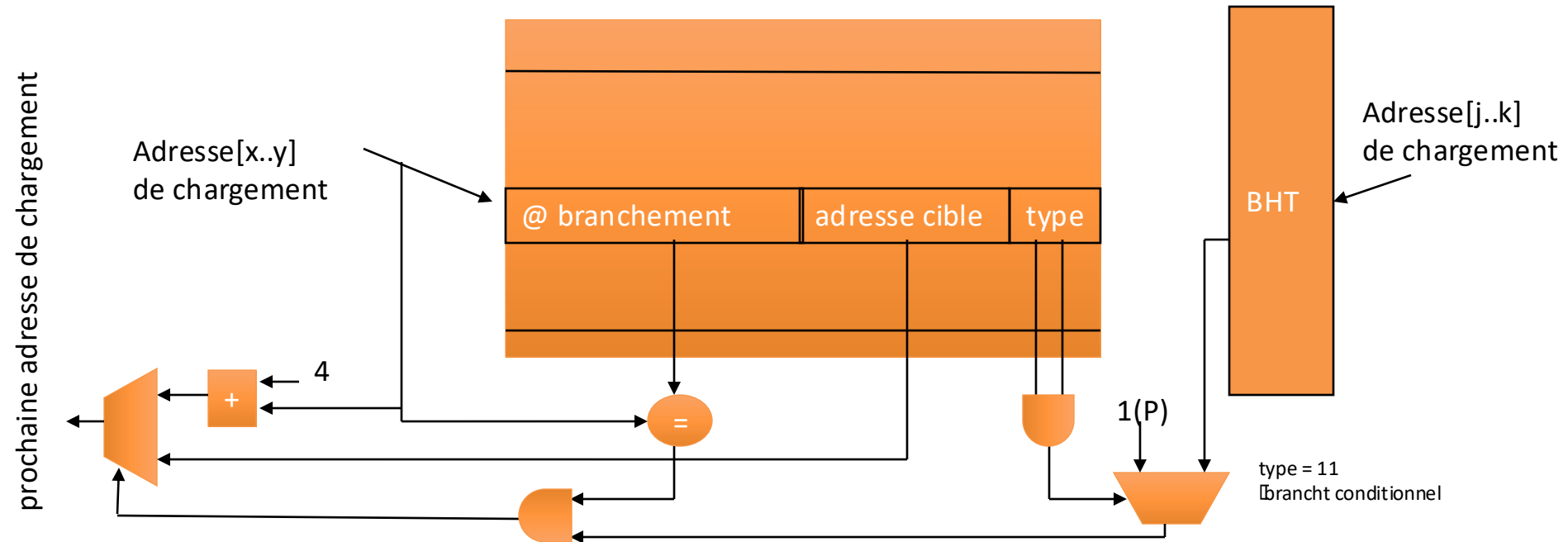


Performance prédicteurs de branchement



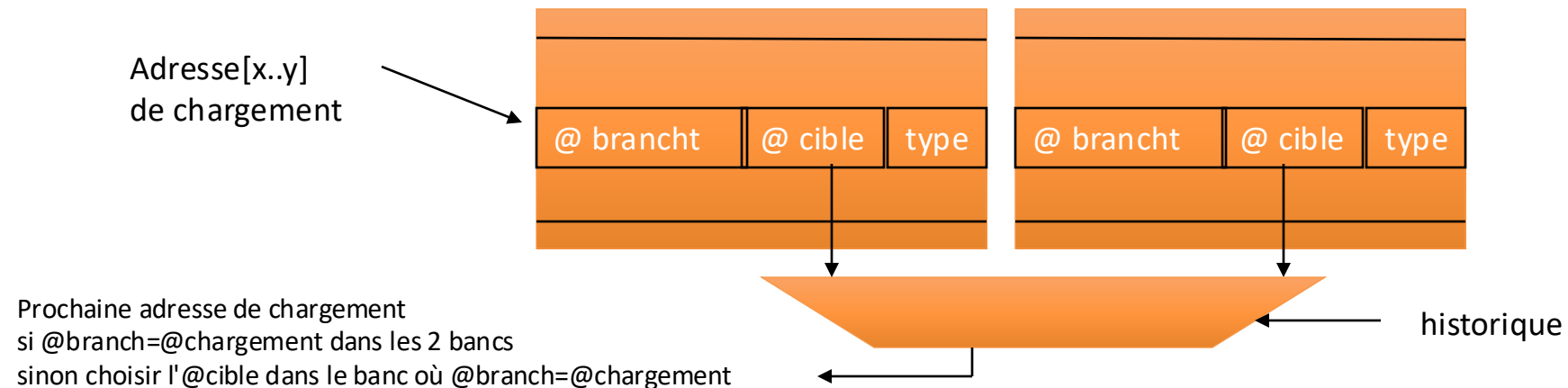
Prédiction de l'adresse

- Effectué avec une table à accès direct ou associatif par ensembles avec un tag (BTB pour Branch Target Buffer)
 - Tandis que l'on peut partager les entrées de la BHT car il n'y a que deux résultats possibles, prendre l'adresse cible d'un autre branchement conduit presque inévitablement à une erreur. Il faut donc s'assurer que l'adresse cible fournie correspond bien à l'adresse du branchement que l'on veut prédire.



Branchements indirects

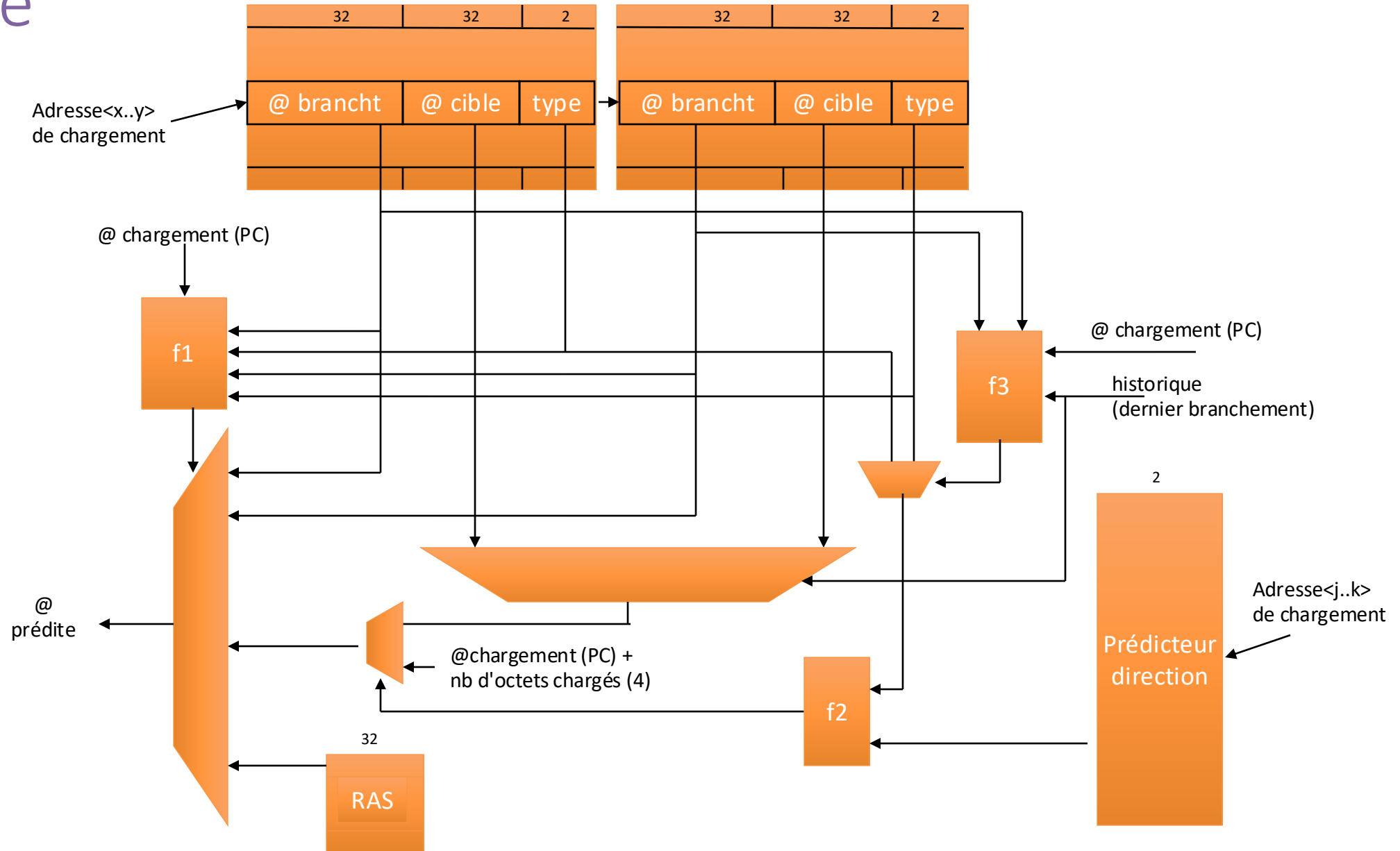
- Les branchements indirects trouvent l'adresse cible dans un registre.
 - Plusieurs adresses cibles possibles. Corrélée au chemin parcouru précédemment (direction des branchements précédents).
- BTB associative par ensembles
 - Le banc est choisi en fonction de l'historique global des branchements.



Sous-programmes

- Les appels de sous-programme se comportent comme des branchements inconditionnels.
- A l'exécution, les retours de sous-programme retournent à l'adresse du dernier BSR qui a été mémorisée en mémoire (pile).
- Pile interne au processeur
 - RAS: Return Address Stack
 - Exécution BSR: empiler dans la RAS adresse de retour (@BSR+4)
 - Chargement RTS: dépiler @ de chargement suivante de la RAS.

Vue globale



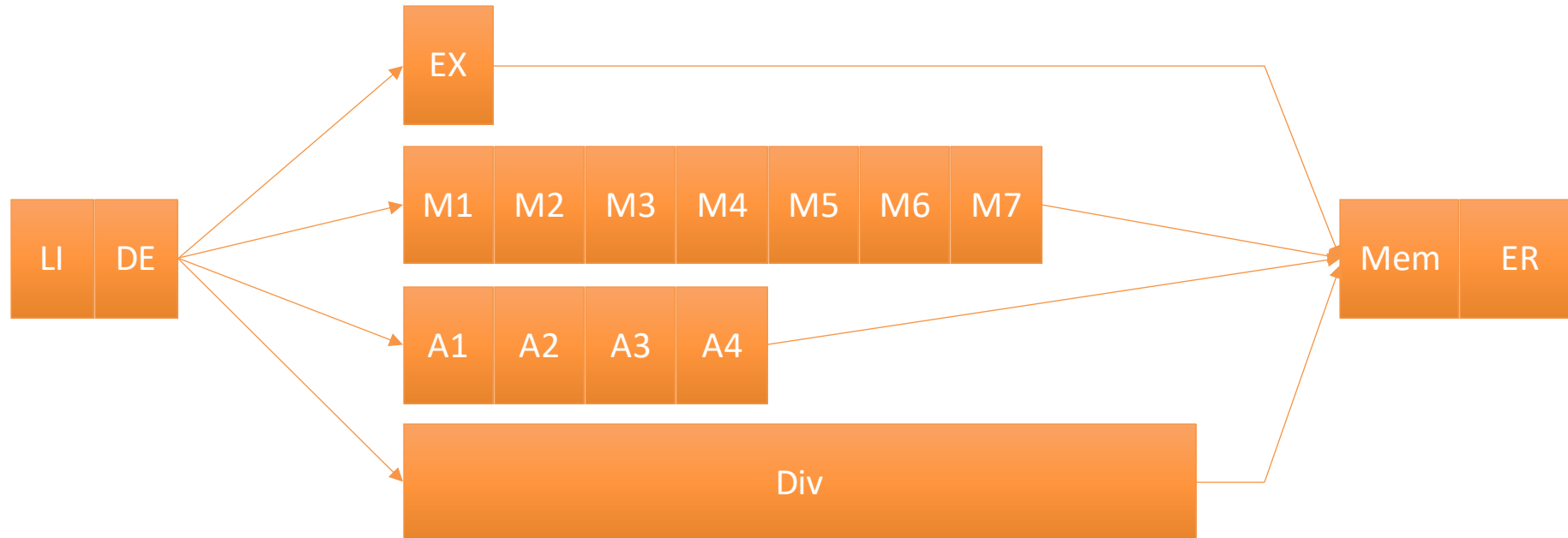
Exemples de prédicteurs de branchements réels

- Cortex A5
 - historique global, PHT 256 entrées, , BTB de 4 entrées pour les branchements indirects, RAS de 4 entrées
- Cortex A9
 - historique global, PHT 1024 compteurs 2 bits, BTB 2*256 entrées, RAS de 8 entrées

Exercice speedup BTB

- Supposons que l'on a un pipeline assez profond et on a une BTB pour les branchements conditionnels uniquement.
- Supposons que la pénalité de mauvaise prédiction est de 4 cycles et que la pénalité d'échec d'accès au BTB est de 3 cycles.
- Supposons un taux de succès de 90%, un taux de bonne prédiction de 90%, et une fréquence de branchements de 15%.
- Speedup de ce processeur par rapport à un processeur n'ayant pas de BTB et une pénalité de 2 cycles par branchement ?
- Supposons que le CPI d'un programme sans pénalité de branchement est de 1.

Pipeline et opérations multi-cycles



- M : multiplication entière et flottante
- A : additionneur flottant,
- Div rarement pipeliné

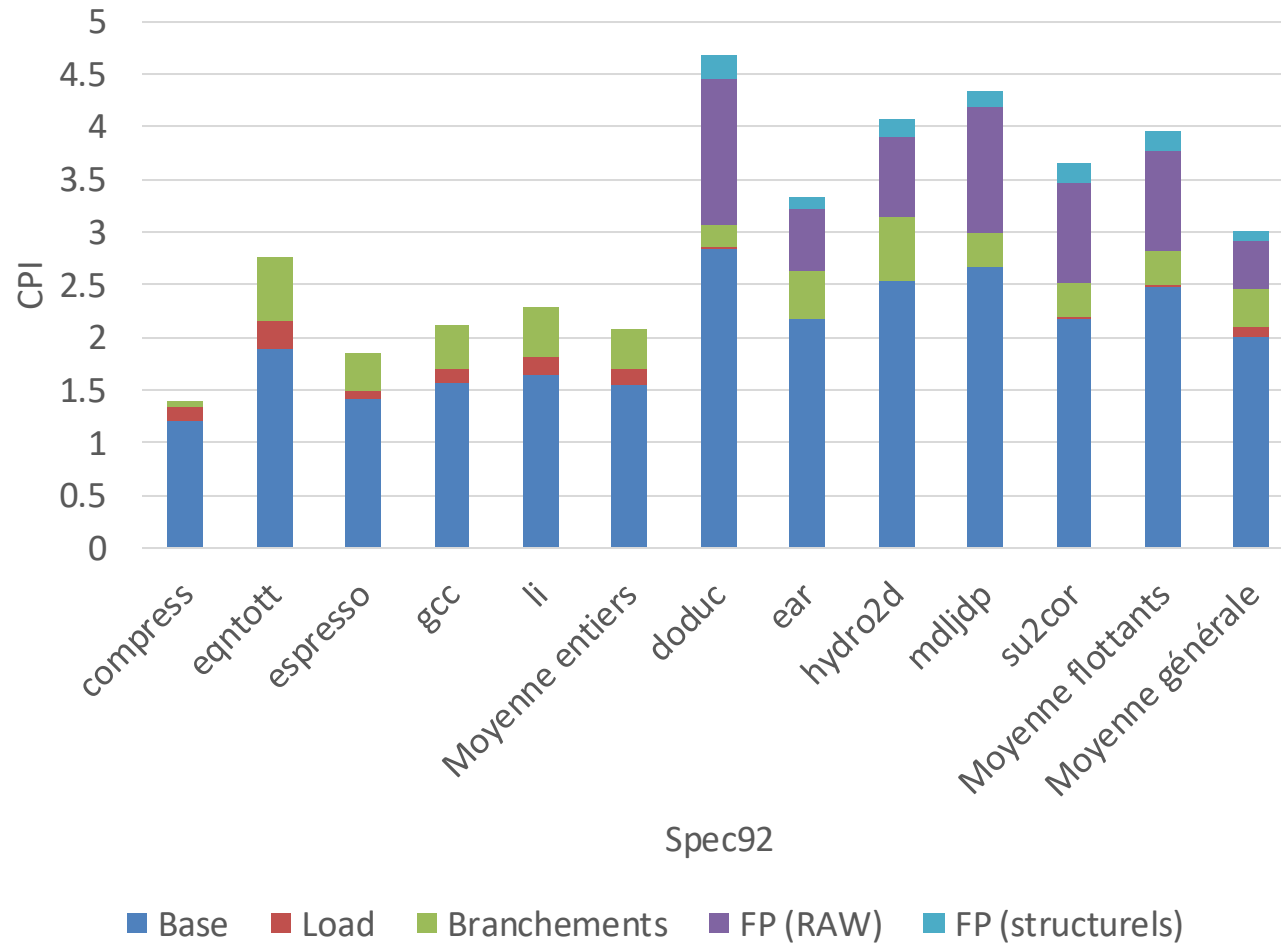
Unité fonctionnelle	Latence	Intervalle d'initiation
ALU entière (EX)	0	1
LDR	1	1
add flottant	3	1
mult	6	1
div	24	25

Pipeline et opérations multi-cycles (exemple aléas structurels)

	1	2	3	4	5	6	7	8	9	10	11
VMUL.F64 D0, D4, D6	LI	DI	M1	M2	M3	M4	M5	M6	M7	M	ER
ADD R0, R0, R2		LI	DI	EX	M	ER					
ADD R1, R0, R3			LI	DI	EX	M	ER				
VADD.F64 D2, D4, D6				LI	DI	A1	A2	A3	A4	M	ER
ADD R0, R0, R2					LI	DI	EX	M	ER		
ADD R1, R0, R3						LI	DI	EX	M	ER	
VLD R .64 D3, [R2]							LI	DI	EX	M	ER

- 3 instructions se terminent en même temps
 - 👎 Plus de ports sur le banc de registres mais nombre moyen = 1
 - 👍 Priorité à l'instruction la plus ancienne
- Les instructions se terminent dans le désordre
 - Pb pour les exceptions

Pipeline et opérations multi-cycles (performance)



Pipeline et opérations multi-cycles (ordonnancement)

- L'ordonnancement statique (par le compilateur) parvient difficilement à masquer les suspensions dues aux opérations à latence longue car l'ordonnancement statique ne doit pas violer les dépendances de contrôle (doit donc se faire à l'intérieur d'un bloc de base).
- En moyenne, 20% de branchements
 - Bloc de base = 5 instructions
- Solution logicielle : déroulage de boucles
- Solution matérielle : ordonnancement dynamique

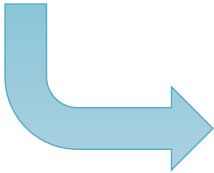
Déroutage de boucle (1)

```
for (i=9999; i>=0; i--)  
    X[i] = X[i] +S;
```

```
for:    VLDR.64    D0, [R1]  
        VADD.F64  D1, D0, D2  
        VSTR.64   D1, [R1]  
        SUBS      R1, R1, #8  
        BNE       for
```

```
for (i=9999; i>0; i-=2){  
    X[i] = X[i] +S;  
    X[i-1] = X[i-1] +S;  
}
```

Déroutage et renommage



```
for:    VLDR.64    D0, [R1]  
        VADD.F64  D1, D0, D2  
        VSTR.64   D1, [R1]  
        VLDR.64   D3, [R1, #-8]  
        VADD.F64  D4, D3, D2  
        VSTR.64   D4, [R1, #-8]  
        SUBS      R1, R1, #16  
        BNE       for
```

Ordonnancement

```
for:    VLDR.64    D0, [R1]  
        VLDR.64    D3, [R1, #-8]  
        VADD.F64  D1, D0, D2  
        VADD.F64  D4, D3, D2  
        SUBS      R1, R1, #16  
        VSTR.64   D1, [R1, #16]  
        VSTR.64   D4, [R1, #8]  
        BNE       for
```

Déroulage de boucle (2)

Hypothèses de latence

Instruction productrice	Instruction utilisatrice	Latence
opération UAL flottante	opération UAL flottante	3
opération UAL flottante	rangement double	2
chargement double	opération UAL flottante	1
chargement double	rangement double	0

```
for:  VLDR.64  D0, [R1]      1
      VADD.F64 D1, D0, D2   s3
      VSTR.64  D1, [R1]     ss6
      SUBS     R1, R1, #8    7
      BNE     for          8
```

10 000 * 8 = 80 000 cycles (idéal=50 000 cycles)

Ordonnancement simple

```
for:  VLDR.64  D0, [R1]      1
      VADD.F64 D1, D0, D2   s3
      SUBS     R1, R1, #8    4
      VSTR.64  D1, [R1]     s6
      BNE     for          7
```

10 000 * 7 = 70 000 cycles

Déroulage et réordonnancement

```
for:  VLDR.64  D0, [R1]      1
      VLDR.64  D3, [R1, #-8] 2
      VADD.F64 D1, D0, D2     3
      VADD.F64 D4, D3, D2     4
      SUBS     R1, R1, #16    5
      VSTR.64  D1, [R1, #16]  6
      VSTR.64  D4, [R1, #8]   7
      BNE     for            8
```

5 000 * 8 = 40 000 cycles

Exercice 1 : déroulage de boucle

Hypothèses de latence

Instruction productrice	Instruction utilisatrice	Latence
opération UAL flottante	opération UAL flottante	3
opération UAL flottante	rangement double	2
chargement double	opération UAL flottante	1
chargement double	rangement double	0

On considère la séquence de code suivante où X est un vecteur de nombres flottants 64 bits

```
for (i=N-2; i>0; i--)
```

```
    X[i+1] = X[i]+A;
```

1. En supposant que le vecteur est rangé à l'adresse 0, que le registre R1 est initialisé avec l'adresse de l'avant dernier élément du tableau (X[N-2]) et que D2 contient la valeur A, donner le code ARM de cette séquence.
2. Avec les hypothèses de latence, calculer le nombre de cycles nécessaires pour exécuter une itération de la boucle. On suppose qu'il n'y a pas d'aléas structurels.
3. Dérouler la boucle 4 fois, réordonner. Durée moyenne d'une itération ?

Exercice 1 : réponse q1

Exercice 1 : réponse q2

Exercice 1 : réponse q3

Exercice 2 : déroulage de boucle

Hypothèses de latence

Instruction productrice	Instruction utilisatrice	Latence
opération UAL flottante	opération UAL flottante	3
opération UAL flottante	rangement double	2
chargement double	opération UAL flottante	1
chargement double	rangement double	0

On considère la séquence de code suivante où X et Y sont des vecteurs de nombres flottants 64 bits

```
for (i=N; i>0; i--)
```

```
    Y[i] = A*X[i] + Y[i];
```

1. En supposant que le vecteur X est rangé à l'adresse 0, que les registres R1 et R2 sont initialisés avec l'adresse du dernier élément des tableaux X et Y et que D4 contient la valeur A, donner le code ARM de cette boucle.
2. Avec les hypothèses de latence, calculer le nombre de cycles nécessaire pour exécuter une itération de la boucle. On suppose qu'il n'y a pas d'aléas structurels.
3. Réordonner le code pour minimiser les suspensions.
4. Dérouler la boucle 2 fois, réordonner. Durée moyenne d'une itération ?

Exercice 2 : réponse q1

Exercice 2 : réponse q2

Exercice 2 : réponse q3

Exercice 2 : réponse q4

Exercice 3 : déroulage de boucle

Hypothèses de latence

Instruction productrice	Instruction utilisatrice	Latence
opération UAL flottante	opération UAL flottante	3
opération UAL flottante	rangement double	2
chargement double	opération UAL flottante	1
chargement double	rangement double	0

On considère la séquence de code suivante où A, B et C sont des vecteurs de nombres flottants 64 bits

```
for (i=0; i<N-1; i++){ A[i+1] = A[i]+ C[i]; B[i+1] = B[i] + A[i+1]; }
```

1. En supposant que les registres R1, R2 et R3 sont initialisés avec les adresses des tableaux A, B et C et que l'adresse du dernier élément est dans R4, donner le code ARM de cette boucle.
2. Avec les hypothèses de latence, calculer le nombre de cycles nécessaire pour exécuter une itération de la boucle. On suppose qu'il n'y a pas d'aléas structurels.
3. Modifier le code pour supprimer les suspensions.
4. Dérouler la boucle 2 fois, réordonner. Durée moyenne d'une itération ?

Exercice 3 : réponse q1

Exercice 3 : réponse q2

Exercice 3 : réponse q3

Exercice 3 : réponse q4